

Operating System

Chapter 1: Processes and Threads	7.3
Chapter 2: Interprocess Communication, Concurrency and Synchronization	7.17
Chapter 3: Deadlock and CPU Scheduling	7.35
Chapter 4: Memory Management and Virtual Memory	7.54
Chapter 5: File Systems, I/O Systems, Protection and Security	7.74

U

N

I

T

7

Chapter 1

Processes and Threads

LEARNING OBJECTIVES

- Basics of operating systems
- Services of OS
- Evolution of operating systems
- Processes
- Processes and process control blocks
- Process states
- Process creation
- Suspended processes
- OS control structures
- Process attributes
- Modes of execution
- Threads
- Thread functionality
- Thread synchronization

BASICS OF OPERATING SYSTEM

An operating system (OS) is a program that controls the execution of application programs and acts as an interface between applications and the computer hardware. The three objectives of an OS are as follows:

1. *Convenience*: An OS makes a computer more convenient to use.
2. *Efficiency*: An OS allows the computer system resources to be used in an efficient manner.
3. *Ability to evolve*: An OS should be constructed in such a way as to permit the effective development, testing and introduction of new system functions without interfering with service.

OS as a User–Computer Interface

Consider the below figure (Figure 1) which shows the hardware and software used in providing applications to a user in a layered fashion.

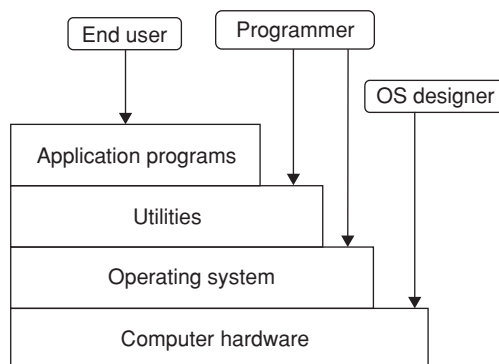


Figure 1 Layers and views of a computer system.

The end user of the application is not concerned with the details of computer hardware. Utilities implement the frequently used functions that assist in program creation, the management of files and control of input/output (I/O) devices. The most important collection of system programs comprises the OS. The OS masks the details of the hardware from the programmer and provides the programmer with a convenient interface for using the system.

Services of an OS

1. *Program development*: An OS provides a variety of facilities and services as editors and debuggers to assist programmers in creating programs.
2. *Program execution*: An OS handles the scheduling duties of program execution for the user.
3. *Access to I/O devices*: An OS provides a uniform interface that hides the details of I/O devices so that the programmers can access the I/O devices using simple reads and writes.
4. *Controlled access to files*: In a system with multiple users, an OS provides protection mechanism to control access to the files.
5. *System access*: For shared or public systems, an OS controls access to the system as a whole and to specific system resources.
6. *Error detection and response*: An OS must provide a response that clears the error condition with the least impact on running applications.
7. *Accounting*: A good OS will collect usage statistics for various resources and monitor performance parameters (viz., response time).

OS as Resource Manager

A computer is a set of resources for the movement, storage and processing of data and for the control of these functions. An OS is responsible for managing these resources.

1. An OS functions in the same way as ordinary computer software, that is, it is a program or suite of programs executed by the processor.
2. An OS frequently relinquishes control and must depend on the processor to allow it to regain control.

The OS directs the processor in the use of the other system resources and in the timing of its execution of other programs.

Figure 2 shows the resources that are managed by an OS.

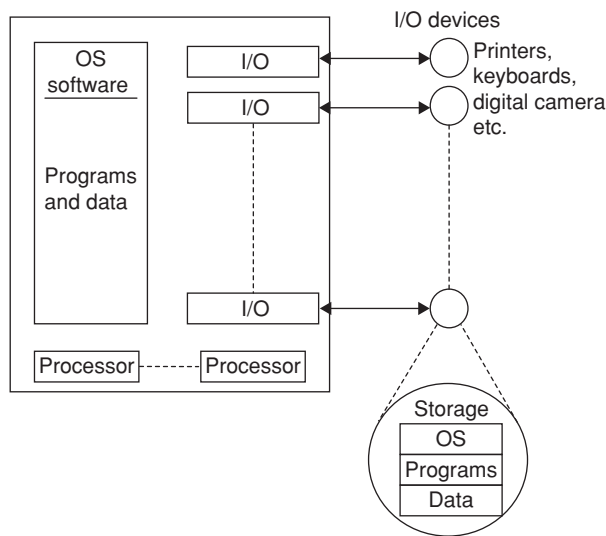


Figure 2 OS as a resource manager.

A portion of the OS lies in the main memory. This includes the kernel or nucleus, which contains the most frequently used functions in the OS and, at a given time, other portions of the OS currently in use. The remainder of main memory contains user programs and data. The allocation of this resource is controlled jointly by the OS and memory management hardware in the processor.

Evolution of OSs

Serial processing

With the earliest computers, the programmer interacted directly with the computer hardware. There was no OS. Programs in machine code were loaded via the input devices. This mode of operation is termed as *serial processing*. Problems with this system are scheduling and setup time.

Simple batch systems

- It requires the grouping up of similar jobs, which consist of programs, data and system commands.
- Users have no control over results of a program.
- Off-line debugging.

Multiprogrammed batch systems

In view of simultaneous execution of multiple programs, it improves system throughput and resource utilization.

Example: Windows XP, 98

- **Multitasking OS:** A running state of a program is called a *process* or a *task*. The concept of managing a multitude of simultaneously active programs, competing with each other for accessing the system resources is called *multitasking*.
- Serial multitasking or context switching is the simplest form of multitasking.

Example: Windows NT, Linux

- **Multuser OS:** It is defined as multiprogramming OS that supports simultaneous interaction with multiple users.

Example: Linux, Unix, a dedicated transaction processing system (viz., railway reservation system).

- **Multiprocessing OS:** The term *multiprocessing* means multiple CPUs performing more than one job at one time.

The term 'multiprogramming' means situation in which a single CPU divides its time between more than one job.

Time sharing systems

In this kind of OS, the processor time is shared among multiple users. The CPU switches rapidly from one user to another user; each user is given an impression that he/she has his own computer while it is actually one computer shared among many users.

If there are n users actively requesting service at one time, each user will only see on the average $1/n$ of the effective computer capacity, not counting OS overhead.

Bootstrap Bootstrap is an initial program which runs, when a computer is powered up (or) restarted. The task is to initialize system aspects (CPU registers to device controllers to memory contents). It is stored within the computer hardware known as *firm ware* (EEPROM).

PROCESSES

Processes and Process Control Blocks

Process A process is an instance of a program in execution. Two essential elements of a process are as follows:

1. Program code
2. Set of data

At any given point in time, while the program is executing, the process can be uniquely characterized by a number of elements, including the following:

1. Identifier
2. State
3. Priority

4. Program counter
5. Memory pointers
6. Context data
7. I/O status information
8. Accounting information

This information is stored in a data structure, typically called a *process control block*, that is created and managed by the OS.

Process control block (PCB) It contains sufficient information so that it is possible to interrupt a running process and later resume execution as if the interruption has not occurred.

Process States

The behaviour of an individual process can be characterized by listing the sequence of instructions that executed for that process. This listing is referred to as a *trace* of the process. Also the behaviour of a processor is shown by listing the traces of the various processes that are interleaved.

Dispatcher A dispatcher is a small program that switches the processor from one process to another.

Two-state process model In the simplest possible process model (Figure 3), at any time, a process is either being executed by a processor or not, that is, a processor may be in one of two states: *running* or *not running*.

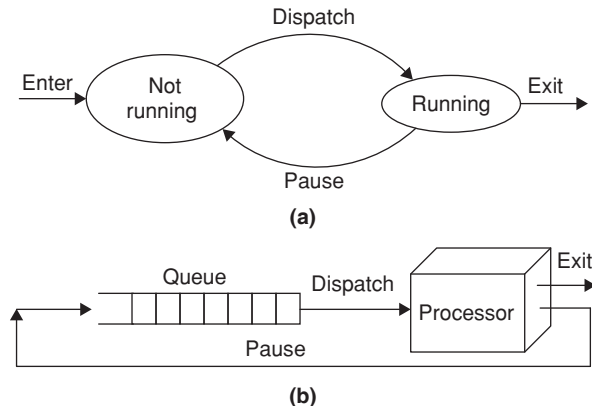


Figure 3 Two-state process model. (a) State transition diagram, (b) Queuing diagram

When the OS creates a new process, it creates the PCB for the process and enters that process into the system in the *not running* state. The process exists, is known to the OS, and is waiting for an opportunity to execute. From time to time, the currently running process will be interrupted and the dispatcher portion of the OS will select some other process to run. The former process moves from the *running* state to the *not running* state and one of the other processes moves to the *running* state.

Processes that are not running must be kept in some sort of queue, waiting their turn to execute. Figure 3(b) shows the structure. There is a single *queue* in which each entry is a pointer to the PCB of a particular process.

Creation and Termination of Processes

Process creation

When a new process is to be added to those currently being managed, the OS builds the data structures that are used to manage the process and allocates address space in main memory to the process. The common events which lead to process creation are as follows:

1. New batch job
2. Interactive logon
3. Created by OS to provide a service
4. Spawned by existing process

When the OS creates a process at the explicit request of another process, the action is referred to as *process spawning*. When one process spawns another, the former is referred to as the *parent process* and the spawned process is referred to as the *child process*.

Process termination

The following are the reasons for process termination:

1. *Normal completion*: Process executes an OS service call to indicate its completion.
2. *Time limit exceeded*: The process has run longer than the specified total time limit.
3. *Memory unavailable*: The process requires more memory than the system can provide.
4. *Bounds violation*: The process tries to access a memory location that it is not allowed to access.
5. *Protection error*: The process attempts to use a resource that is not allowed to access.
6. *Arithmetic error*: The process tries a prohibited computation.
7. *Time overrun*: The process has waited longer than a specified maximum for a certain event to occur.
8. *I/O failure*: Error occurs during input or output.
9. *Invalid instruction*: The process attempts to execute a non-existent instruction.
10. *Privileged instruction*: The process attempts to use an instruction reserved for OS.
11. *Data misuse*: A piece of data is of the wrong type or is not initialized.
12. *Operator or OS intervention*.
13. *Parent termination*.
14. *Parent request*.

Five-state model

The five states in Figure 4 are as follows:

1. *New*: The process is created but not admitted to the pool of executable processes.
2. *Running*: Process in execution, that is, it is using CPU.
3. *Blocked*: Waiting for some event to occur (i.e., I/O) before it can continue execution.

4. **Ready:** Process is ready for execution. Just it is waiting.
5. **Exit:** The process has been aborted by parent process or has finished its execution.

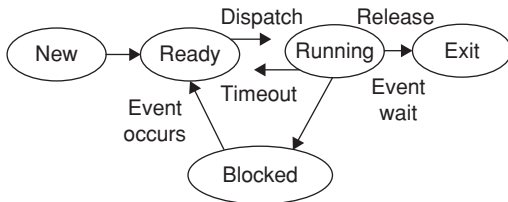


Figure 4 Process states.

Figure 4 indicates the types of events that lead to each state transition for a process. The possible transitions are as follows:

1. NULL → New: A new process is created to execute a program.
2. New → Ready: The OS will move a process from the New state to the Ready state when it is prepared to take on an additional process.
3. Ready → Running: When it is time to select a process to run, the OS chooses one of the processes in the Ready state.
4. Running → Exit: The currently running process is terminated by the OS if the process indicates that it has completed or if it aborts.
5. Running → Ready: The reasons for this transition are
 - Running process has reached the maximum allowable time for uninterrupted execution.
 - As the OS assigns different levels of priority to different processes, there will be pre-emption.
 - A process may voluntarily release control of the processor.
6. Running → Blocked: A process is put in the blocked state if it requests something for which it must wait.
7. Blocked → Ready: This transition occurs when the event for which the process has been waiting occurs.
8. Ready → Exit: A parent may terminate a child process at any time.
9. Blocked → Exit: Parent may terminate any blocked process.

Queuing model for five-state model

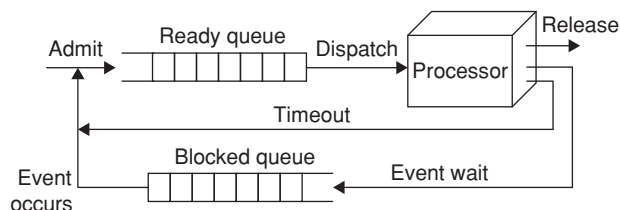


Figure 5 Single blocked queue.

Suspended Processes

Need for swapping In five-state process model using multiple blocked queues, the memory holds multiple processes. Moreover, the processor can move to another process when one process is blocked. But the processor is so much faster than I/O that it will be common for all of the processes in memory to be waiting for I/O. Thus, even with multiprogramming, a processor could be idle most of the time.

Then we can extend the main memory to accommodate more processes, but it is not an efficient solution. Another solution to this problem is swapping. Swapping involves moving part or all of a process from main memory to disk. When none of the processes in main memory is in the ready state, the OS swaps one of the blocked processes out onto disk into a suspend queue. The OS then brings in another process from the suspend queue or it honours a new process request. Then execution continues with the newly arrived process. With the use of swapping, another state is added to the process in the behaviour model.

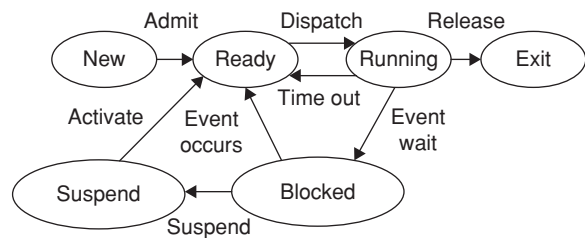


Figure 6 Process state-transition diagram with suspend state.

The four distinguishable states in this process model are as follows:

1. **Ready:** The process is in main memory and is available for execution.
2. **Blocked:** The process is in main memory and awaiting an event.
3. **Blocked/suspend:** The process is in secondary memory and awaiting an event.
4. **Ready/suspend:** The process is in secondary memory but is available for execution as soon as it is loaded into main memory.

Figure 7 shows the process state model with two suspend states:

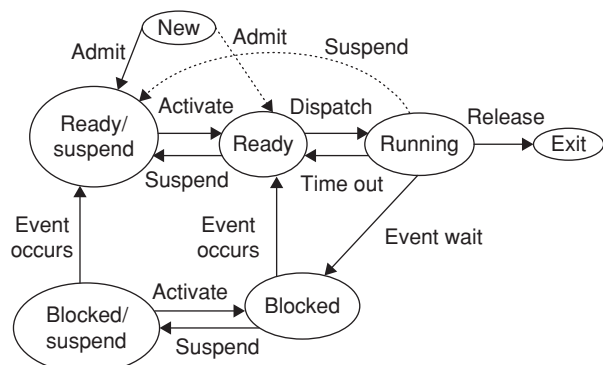


Figure 7 Process state transition diagram with suspend state.

Uses of Suspension

Characteristics of suspended process are as follows:

1. The process is not immediately available for execution.
2. The process may or may not wait on an event.
3. The process was placed in a suspend state by either itself, a parent or the OS.

Reasons for process suspension

- *Swapping*: To Release sufficient main memory.
- *Other OS reason*: OS may suspend a background process.
- *Interactive User Request*: A user may wish to suspend execution of a program.
- *Timing*: A process may be executed periodically and may be suspended.
- *Parent Process request*: A parent process may wish to suspend execution of a descendent.

OS CONTROL STRUCTURES

If the OS is to manage processes and resources, it must have information about the current status of each process and resources. The OS constructs and maintains tables of information about each entity that it is managing. Figure 8 shows the general structure of OS control tables:

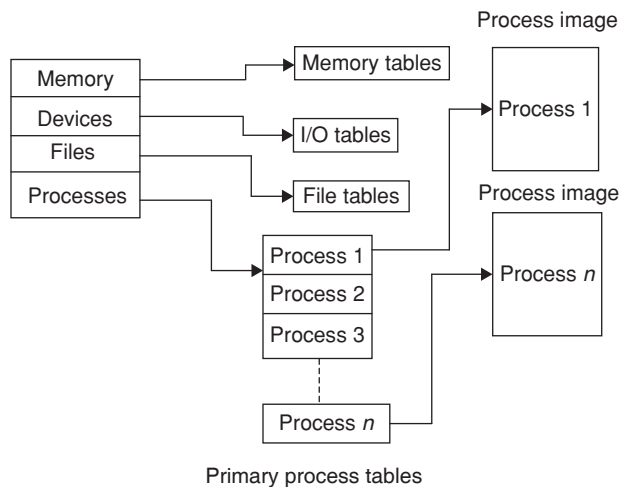


Figure 8 OS control tables.

Different tables maintained by the OS are

1. Memory
2. I/O
3. File
4. Process

Memory tables: These tables are used to keep track of both main and secondary memory.

I/O tables: These are used by the OS to manage the I/O devices and channels of the computer system.

File tables: These tables provide information about the existence of files, their location on secondary memory, their current status and other attributes.

Process tables: An OS must maintain process tables to manage processes.

PROCESS CONTROL STRUCTURES

The OS must know about

1. Process location
2. Process attributes

Process Location

The collection of program, data, stack and attributes is referred as process image.

The location of a process image will depend on the memory management scheme being used. The process image is maintained as a contiguous or continuous block of memory. This block is maintained in secondary memory, usually disk, so that the OS can manage the process, at least a small portion of its image must be maintained in main memory. To execute the process, the entire process image must be loaded into main memory or at least virtual memory. Thus the OS needs to know the location of each process on disk and for each such process that is in the main memory, the location of that process is in main memory.

For this, the OS maintains process tables. There is a primary process table with one entry for each process. Each entry contains, at least, a pointer to a process image.

Process Attributes

The typical information required by the OS for each process is as follows:

1. Process identification
2. Process state information
3. Process control information

Process identification Each process is assigned a unique numeric identifier, which may simply be an index into the primary process table. The identifier for a PCB includes the following:

1. Identifier of the process
2. Identifier of the process that created current process
3. User identifier

Process state information It consists of the contents of processor registers. It includes details of

1. User-visible register
2. Control and status registers
3. Stack pointers

Process control information It consists of the additional information needed by the OS to control and coordinate the various active processes. It includes the following:

1. Scheduling and state information
2. Data structuring
3. Interprocess communication

4. Process privileges
5. Memory management
6. Resource ownership and utilization

PROCESS CONTROL

Modes of Execution

Most processors support at least two modes of execution as follows:

1. More-privileged mode
2. Less-privileged mode

Two modes are required to protect the OS and key OS tables from interference by user programs.

1. **More-privileged mode:** This is also referred as *system mode*, *control mode* or *kernel mode*. Certain instructions can only be executed in Kernel mode (e.g., reading or altering a control register, viz., PSW, primitive I/O instructions, etc.).

The Kernel of the OS is a portion of the OS and encompasses the important system functions.

The functions of an OS kernel are as follows:

- Process management
- Memory management
- I/O Management
- Support functions

2. **Less-privileged mode:** This is also referred as *user mode*, because user programs typically would execute in this mode.

In this mode, the software has complete control of the processor and all its instructions, registers and memory.

Process creation If the OS decides to create a table, it has to proceed as follows:

1. Assign a unique process identifier to the new process.
2. Allocate space for the process.
3. Initialize the PCB
4. Set the appropriate linkages.
5. Create or expand other data structures.

Process switching In process switching, a running process is interrupted and the OS assigns another process to the running state and turns control over to that process. The design issues are as follows:

1. When to switch processes
2. Mode switching
3. Change of process state

When to switch processes A process switch may occur anytime that the OS has gained control from the currently running process. The mechanisms for interrupting the execution of a process are as follows:

1. Interrupt
2. Trap
3. Supervisor call

Interrupt When an interrupt occurs, the control is first transferred to an interrupt handler, which does some basic housekeeping and then branches to an OS routine that is concerned with the particular type of interrupt that has occurred (e.g., clock interrupt, I/O interrupt, memory fault, etc.).

Trap Trap related to an error or exception condition gets generated within the currently running process. If the error is fatal, the currently running process is moved to exit state and a process switch occurs, otherwise the action of the OS will depend on the nature of the error and design of the OS.

Supervisor call The OS may be activated by a supervisor call from the program being executed. The case of system call may place the user process in blocked state.

Mode switching If the processor identifies that any interrupt is pending, then

1. it sets the PC to the starting address of an interrupt handler program.
2. it switches from user mode to Kernel mode so that the interrupt processing code may include privileged instructions.

During this process, the context of the process, that has been interrupted, is saved into that PCB of the interrupted program. The context of a program includes PC, other processor registers and stack information.

The occurrence of an interrupt does not necessarily mean a process switch.

Change of process state The mode switch is a concept distinct from that of the process switch.

A mode switch may occur without changing the state of the process that is currently in Running state. In that case, the context saving and subsequent restoral involve little overhead. However, if the currently running process is to be moved to another state then the OS must make substantial changes in its environment. Thus, the process switch, which involves a state change, requires more effort than a mode switch.

System call In order to access the OS services, an interface is required which is provided by the system call.

All the system call routines are executed in Kernel mode. Whenever the system call is invoked, the process status word is changed from user mode to Kernel mode (0 → 1).

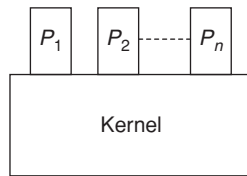
System calls are of six types as follows:

1. File system
2. Process
3. Scheduling
4. Interprocess communications
5. Socket
6. Miscellaneous

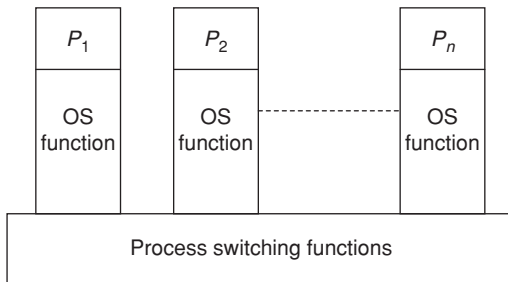
Execution of the OS

There are three possibilities to consider about OS execution:

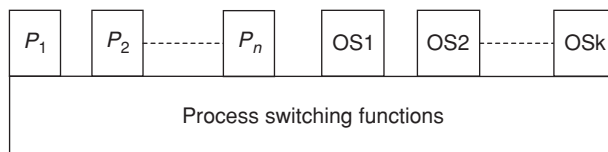
1. Separate Kernel (Figure 9)
2. OS functions execute within user processes (Figure 10)
3. OS functions execute as separate processes (Figure 11)

Separate Kernel**Figure 9** OS as separate Kernel.

Here the Kernel of the OS is executed outside of any process. When currently running process is interrupted or issues a supervisor call, the mode context of this process is saved and control is passed to the Kernel.

Execution within user process**Figure 10** OS functions execute within user processes.

We execute virtually all OS software in the context of user process. To pass control from a user program to the OS, the mode context is saved and a mode switch takes place to an OS routine, that is, a process switch is not performed, just a mode switch within the same process.

Process-based OS**Figure 11** OS functions execute as separate process.

Here the OS is a collection of system processes. This approach encourages the use of modular OS with minimal, clean interface between the modules.

THREADS

A *thread* is a basic unit of CPU utilization. It comprises a thread ID, a program counter, a register set and a stack.

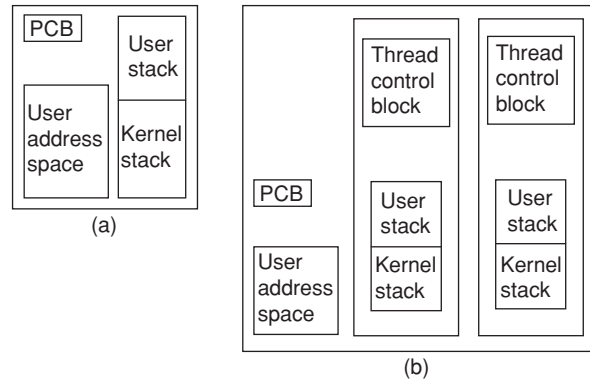
Multithreading

It refers to the ability of an OS to support multiple, concurrent paths of execution within a single process.

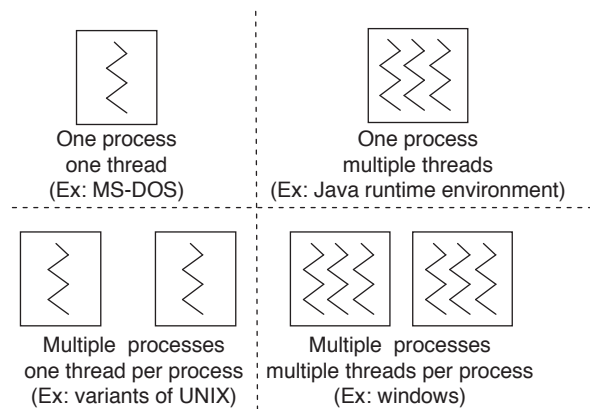
The threads which belong to same process can share their

1. Code section
2. Data section
3. Other OS resources

If a process has multiple threads of control, it can perform more than one task at a time. Figure 12 shows single threaded and multithreaded process models:

**Figure 12** Process models. (a) Single-thread process model
(b) Multithreaded process model.

As OS based on its design will be in one of the following manners (Figure 13):

**Figure 13** Threads and processes.

The threads of a process consist of the following:

1. Thread execution state.
2. Saved thread context when not running
3. An execution stack
4. Some pre-thread static storage for local variables.
5. Access to the memory and resources of its process, shared with all other threads in that process.

All the threads of a process share the state and resources of that process.

Benefits of multithreaded programming

1. *Responsiveness*: Multithreading an interactive application may allow program to continue running even if

part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user.

2. *Resource sharing*: Threads share the memory and the resources of the process to which they belong by default.
3. *Economy*: It is economical to create a new thread in an existing process than to create a brand-new process. It takes less time to context switch between two threads of same process than to switch between processes. Also the time to terminate a thread is less than process termination.
4. *Scalability*: Multithreading on a multi-CPU machine increases parallelism.

Applications that benefit from thread As the threads take advantages of multiple processors, image processing which can be done in parallel, will execute in threads.

Animation rendering is another thread application, where each frame can be rendered in parallel, as each one is independent of other GUI programming will execute at least two threads when it is processing large number of files.

Applications that cannot benefit from thread The main drawbacks of threads is if kernel is single threaded, system call of one thread will block the whole process, in which CPU will be idle during the blocking period.

The other major drawback is security as it is possible that a thread can overwrite the stack as the other thread, as they were meant to cooperate on a single task. Applications that are developed using PHP does not support multithreading at the server side.

THREAD FUNCTIONALITY

The key states for threads are as follows:

1. Running
2. Ready
3. Blocked

There are four basic thread operations associated with a change in thread state:

1. *Spawn*: When a new process is spawned, a thread for that process is also spawned.
Also, a thread within a process may spawn another thread within the same process.
2. *Block*: When a thread needs to wait for an event, it will block. Then the processor may turn to the execution of another ready thread in the same or different process.
3. *Unblock*: When an event for which a thread is blocked occurs, the thread is moved to Ready queue.
4. *Finish*: When a thread completes, its register context and stacks are deallocated.

Multithreading on a Uni-processor

On a uni-processor, multiprogramming enables the interleaving of multiple threads within multiple processes. For

example, consider the execution of three threads *A*, *B*, *C* in two processes on a single processor which are interleaved (Figure 14).

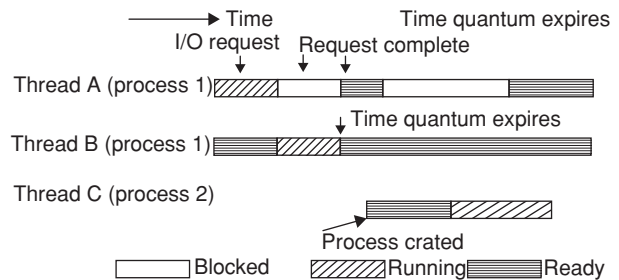


Figure 14 Multithreading on a uni-processor.

Execution passes from one thread to another, either when the currently running thread is blocked or its time slice is exhausted.

Resources used in thread creation and process creations

As process has heavy weight, when it is created, new address space is required, which includes stack, heap and data section, etc. If a process shares the memory, then the IPC is expensive.

The thread is a light-weight process, if it doesn't require any new resources, as it will share the process resources to which it belongs. The major benefit of this is, several threads belong to same activity and can run under same address space.

Thread Synchronization

All of the threads of a process share the same address space and other resources, such as open files. Any alteration of a resource by one thread affects the environment of the other threads in the same process. So, synchronization mechanism is required to coordinate the activities of all the threads within a process.

The techniques used for thread synchronization is the same as process synchronization techniques, which has been discussed later in this book.

TYPES OF THREADS

User-Level Threads

All thread management is done by the application. The Kernel is not aware of the existence of threads. Thread creation and scheduling are done in user space. User-level threads (Figure 15) are fast to create and manage. User-level library provides support for creating, managing and scheduling threads. In single-threaded Kernel, blocking system call from user level thread will block the entire process, even if other threads are ready to run.

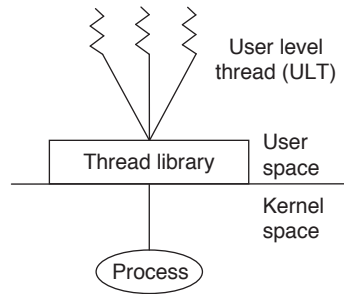


Figure 15 Pure user-level threads.

Advantages of ULTs

1. ULT creation does not require Kernel mode privileges.
2. ULT scheduling can be application specific.
3. ULTs can run on any OS.

Disadvantages of ULTs

1. When a ULT executes a system call, not only is that thread blocked, but also all of the threads within the process are blocked.
2. In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing.

Kernel-level Threads

Kernel-level threads (KLTs) are supported directly by the OS. The creation, scheduling, management are done by kernel in kernel space. They are slower to create and manage. In a multiprocessor, Kernel can schedule threads on different processors.

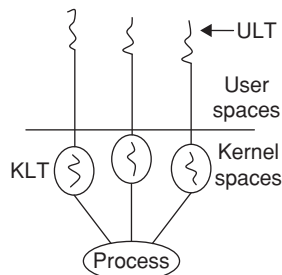


Figure 16 Pure kernel-level threads.

Advantages of KLTs

1. Kernel can simultaneously schedule multiple threads from the same process on multiple processors.
2. If one thread in a process is blocked, the kernel can schedule another thread of the same process.
3. Kernel routines themselves multithreaded.

Disadvantages of KLTs

The transfer of control from one thread to another within the same process requires a mode switch to the Kernel.

Combined Approach

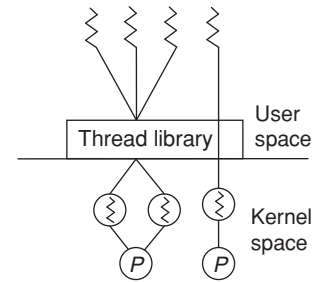


Figure 17 Combined approach.

In combined approach (Figure 17), multiple threads within the same application can run in parallel on multiple processors and a blocking system call need not block the entire process.

Relationship between the threads and processes

1. *One-to-one relationship*: Each thread of execution is a unique process with its own address and resources.
Example: Traditional UNIX.
2. *Many-to-one relationship*: A process defines an address space and dynamic resources ownership. Multiple threads may be created and executed within that process.
Example: Windows NT, Solaris, Linux.
3. *One-to-many relationship*: A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.
Example: Emerald
4. *Many-to-many relationship*: It combines the attributes of M:1 and 1:M cases.
Example: TRIX

THREADING ISSUES

fork() and exec() System Calls

A fork() system call is used to create a separate, duplicate process. In UNIX, each process is identified by its process identifier, which is a unique integer. A new process is created by fork() system call.

The new process consists of a copy of the address space of the original process. This mechanism allows the parent process to communicate easily with its child process. Both processes continue execution at the instruction after the fork(), with one difference: the return code for the fork() is zero for the new process, whereas the process identifier of the child is returned to the parent. The exec() system call is used after a fork() system call by one of the two processes to replace the processes memory space with a new program.

If one thread in a program calls `fork()`, then UNIX chooses two alternatives as follows:

1. Duplicates all the threads
2. Duplicates only the thread that invoked the `fork()` system call.

If a thread invokes the `exec()` system call, the program specified in the parameter to `exec()` will replace the entire process including all threads.

Cancellation

Thread cancellation is the task of terminating a thread before it has completed. A thread that is to be cancelled is often referred to as the *target thread*. Cancellation of a thread may occur in two different scenarios as follows:

1. *Asynchronous cancellation*: One thread immediately terminates the target thread.
2. *Deferred cancellation*: The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion.

Microkernels

Microkernel (Figure 18) is a small OS core that provides the foundation for modular extensions.

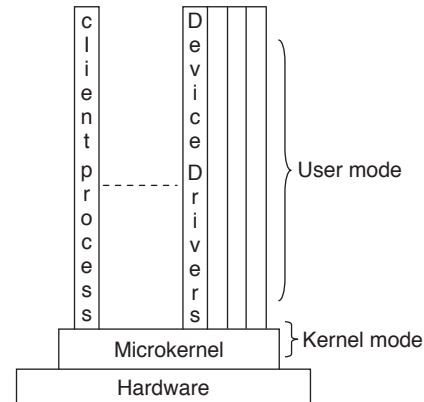


Figure 18 Microkernel architecture.

Advantages of Microkernel Organization

- Uniform interfaces
- Extensibility
- Flexibility
- Portability
- Reliability
- Distributed system support
- Support for object oriented OS

EXERCISES

Practice Problems I

Directions for questions 1 to 15: Select the correct alternative from the given choices.

1. In `fork()` system call, the return value to the parent process and the child process are respectively
 - (A) PID of child process, 1
 - (B) PID of child process, 0
 - (C) PID of child, PID of parent process
 - (D) PID of parent process, PID of child
2. Which of the following is not an advantage of thread?
 - (A) Inter process communication
 - (B) Less memory space occupied by thread
 - (C) Less time to create and terminate than a process
 - (D) Context switching is faster
3. A process executes the following segment of code


```
for (i = 1; i < 10; i++) fork();
```

 The number of new processes created is
 - (A) 1024
 - (B) 1023
 - (C) 1025
 - (D) 1028
4. For each of the following transitions, between process states, which transition is not possible?
 - (A) Running → Ready
 - (B) Blocked → Suspend
 - (C) Ready → Ready/Suspend
 - (D) Blocked → Running
5. An operating system can be mapped to a five-state process model. A new event has been designated as capable to pre-empt the existing processes in order to trigger a new process to complete. Select the correct statement from below:
 - (A) A new state need to be added to the existing transition model to accommodate the changes.
 - (B) The existing model still holds good.
 - (C) Both the states and transitions of the existing model have to be changed.
 - (D) Only the transitions need to be modified.
6. The advantage of having multiple threads over multiple processes is
 - (i) Less time for creation
 - (ii) Less time for termination
 - (iii) Less time for switching
 - (iv) Kernel not involved in communication among threads
 - (A) (i), (ii), (iii)
 - (B) (i), (ii), (iv)
 - (C) (ii), (iii), (iv)
 - (D) (i), (ii), (iii), (iv)
7. Select the correct sequence of steps taken by the processor when an interrupt occurs
 - (i) Switch from user mode to kernel mode.
 - (ii) Set the program counter to the first instruction of the interrupt handling routine.
 - (iii) Save the current context.

- (A) (i), (ii), (iii)
 (B) (i), (iii), (ii)
 (C) (iii), (ii), (i)
 (D) (ii), (i), (iii)

8. What are the necessary steps for a new process creation?

- Assign an identifier to the new process.
- Suspend all other processes.
- Allocate space for the process.
- Initialize process control block.
- Update process-related data structures.
- Update process state information wherever necessary.
- Set the process to user mode
- Notify all the machines in the network about the new process.
- Set the state of the new process as suspended

- (A) All except (ii), (v), (vi), (viii)
 (B) All except (ii), (vii), (viii), (ix)
 (C) All except (iv), (v), (vii), (ix)
 (D) All except (iii), (iv), (vii), (ix)

9. A processor, while executing the instruction sequence of user mode process, received n interrupts. If no other activity is reported to processor during the execution of the instruction sequence, what is the number of mode switches and process switches experienced?

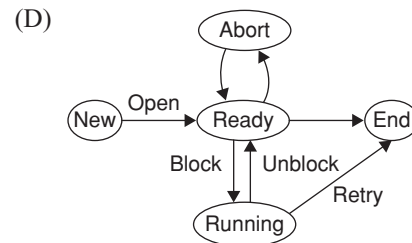
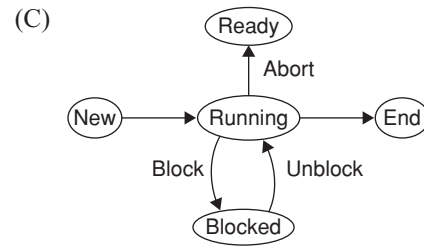
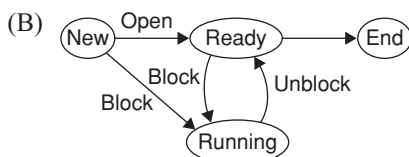
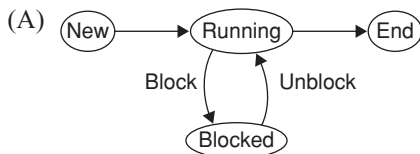
- (A) $2n, 2n$ (B) n, n
 (C) $2n, 0$ (D) $n, 0$

10. Assume a program needs to implement threads, what are the resources that need to be encompassed in critical section?

- Global variables
- Local variables
- Static variables
- Function parameters that are passed as reference pointers
- Global constants

- (A) (i), (ii), (iii), (v)
 (B) (i), (iv), (v)
 (C) (i), (iii), (v)
 (D) (i), (iii), (iv)

11. Which of the following is an appropriate four-state model for a process?



12. Which of the following are considered as disadvantage of user level thread (ULT)?

- Calls made from system will block all threads in a process
 - When scheduled in a multiprocessing environment only one thread per process can be executed at a time.
 - ULTs cannot communicate with each other in a process.
 - The cost of creating a thread is high
- (A) (i), (ii), (iii) (B) (i), (iii), (iv)
 (C) (i), (ii) only (D) (ii), (iii), (iv)

13. Choose from below, advantages of kernel-level threads:

- Kernel can simultaneously schedule multiple threads from the same process on multiple processors.
 - Kernel routines can be multithreaded.
 - If one thread of a process is blocked then kernel can schedule another thread from the same process.
- (A) (i) only (B) (i), (ii)
 (C) (i), (ii), (iii) (D) (i), (iii)

14. Assume that part of a program takes long time to execute. Select an option from below that can enhance performance:

- Implement the part that takes long time as a separate process and use the results as needed from the main program.
 - Implement both the parts as two different threads in the same process.
- (A) (i) only
 (B) (ii) only
 (C) Both (i) and (ii)
 (D) Neither (i) nor (ii)

15. What is a kernel-level thread?

- Threads that are spawned by OS Kernel
 - Threads that are launched by user by directly accessing the kernel
- (A) (i) only (B) (ii) only
 (C) Both (i), (ii) (D) Neither (i) nor (ii)

Practice Problems 2

Directions for questions 1 to 14: Select the correct alternative from the given choices.

1. Many-to-many multithreading model is used in which of the following operating system?
 - (A) Windows NT/2000 with Thread Fibre
 - (B) Windows 95
 - (C) Windows 98
 - (D) Solaris Green Threads
2. Which of the following does not interrupt a running process?
 - (A) Device
 - (B) Timer
 - (C) Scheduler
 - (D) Power failure
3. Which of the following need not be saved on a context switch between processes?
 - (A) General purpose registers
 - (B) Translation look aside buffer
 - (C) Program counter
 - (D) All of the above
4. Which of the following actions is/are typically not performed by the OS when switching context from process A to process B?
 - (A) Saving current register values and restoring the register values for process B
 - (B) Changing address translation tables
 - (C) Swapping out the memory image of process A to the disk
 - (D) Both (B) and (C)
5. For each thread in a multithreaded process, there is a separate
 - (A) Process control block
 - (B) User address space
 - (C) User and kernel stack
 - (D) Kernel space only
6. When a supervisor call is received
 - (A) Mode switch happens
 - (B) Process switch happens
 - (C) Both (A) and (B)
 - (D) Neither (A) nor (B)
7. What is the purpose of jacketing?
 - (A) Convert non-blocking system call to blocking system call
 - (B) Convert blocking system call to non-blocking system call
 - (C) Convert blocking system call into a new thread
 - (D) Convert non-blocking system call into a new thread
8. Which of the following statements is/are always true?
 - (i) Time taken for mode switch is always greater than process switch.
 - (ii) Time taken for mode switch is always less than process switch.
 - (iii) Time taken for mode switch is always equal to process switch.
 - (A) (i) and (iii)
 - (B) (ii) and (iii)
 - (C) Only (i)
 - (D) Only (ii)
9. Which of the following is the property of time sharing systems?
 - (i) Multiple user access
 - (ii) Multiprogramming
 - (A) (i) only
 - (B) (ii) only
 - (C) Both (i) and (ii)
 - (D) Neither (i) nor (ii)
10. Which of the following is/are not a valid reason for process creation?
 - (i) Created by OS
 - (ii) Interactive logon
 - (iii) Privileged instruction
 - (A) (i), (ii)
 - (B) (ii), (iii)
 - (C) (i), (iii)
 - (D) (iii) only
11. Which of the following is/are reason(s) for blocking a running process?
 - (i) A call from the running program to a procedure that is a part of OS code.
 - (ii) A running process may initiate an I/O operation.
 - (iii) A user may block a running process.
 - (A) (i), (ii) only
 - (B) (ii), (iii) only
 - (C) (i), (iii) only
 - (D) (iii) only
12. If the OS is pre-empting a running process because a higher priority process on blocked/suspend queue has just become unblocked, then the running process moved to _____ queue.
 - (A) Suspend
 - (B) Ready/suspend
 - (C) Blocked
 - (D) Blocked/suspend
13. Which of the following is used to call an OS function?
 - (A) Interrupt
 - (B) Trap
 - (C) Supervisor call
 - (D) All of these
14. Which of the following is a general component of a thread?
 - (i) Thread ID
 - (ii) Register set
 - (iii) User stack
 - (iv) Kernel stack
 - (A) (i), (iii), (iv)
 - (B) (i), (ii), (iv)
 - (C) (i), (ii), (iii)
 - (D) (i), (ii), (iii), (iv)

PREVIOUS YEARS' QUESTIONS

- Consider the following statements with respect to User-level threads and Kernel-supported threads. [2004]
 - Context switch is faster with Kernel-supported threads
 - For user-level threads, a system call can block the entire process
 - Kernel supported threads can be scheduled independently
 - User level threads are transparent to the Kernel
 Which of the above statements are true?
 - 2, 3 and 4 only
 - 2 and 3 only
 - 1 and 3 only
 - 1 and 2 only
- Which one of the following is true for a CPU having a single interrupt request line and a single interrupt grant line? [2005]
 - Neither vectored interrupt nor multiple interrupting devices are possible
 - Vectored interrupts are not possible but multiple interrupting devices are possible
 - Vectored interrupts and multiple interrupting devices are both possible
 - Vectored interrupt is possible but multiple interrupting devices are not possible
- Normally user programs are prevented from handling I/O directly by I/O instructions in them. For CPUs having explicit I/O instructions, such I/O protection is ensured by having the I/O instructions privileged. In a CPU with memory mapped I/O, there is no explicit I/O instruction. Which one of the following is true for a CPU with memory mapped I/O? [2005]
 - I/O protection is ensured by operating system routine(s)
 - I/O protection is ensured by a hardware trap
 - I/O protection is ensured during system configuration
 - I/O protection is not possible
- Consider the following code fragment: `if (fork ()==0)` [2005]


```
{ a = a + 5; printf("%d,%d\n", a, &a); }
else { a = a - 5; printf("%d,%d\n", a, &a); }
```

 Let u, v be the values printed by the parent process, and x, y be the values printed by the child process. Which one of the following is true?
 - $u = x + 10$ and $v = y$
 - $u = x + 10$ and $v \neq y$
 - $u + 10 = x$ and $v = y$
 - $u + 10 = x$ and $v \neq y$
- Consider the following statements about user-level threads and Kernel-level threads. Which one of the following statements is false? [2007]
 - Context switch time is longer for Kernel-level threads than for user level threads.
 - User level threads do not need any hardware support.
 - Related Kernel-level threads can be scheduled on different processors in a multi-processor system.
 - Blocking one Kernel-level thread blocks all related threads.
- Which of the following statements about synchronous and asynchronous I/O is NOT true? [2008]
 - An ISR is invoked on completion of I/O in synchronous I/O but not in asynchronous I/O
 - In both synchronous and asynchronous I/O, an ISR (Interrupt Service Routine) is invoked after completion of the I/O
 - A process making a synchronous I/O call waits until I/O is complete, but a process making an asynchronous I/O call does not wait for completion of the I/O
 - In the case of synchronous I/O, the process waiting for the completion of I/O is woken up by the ISR that is invoked after the completion of I/O
- A process executes the following code


```
for (i=0; i<n; i++) fork( );
```

 The total number of child processes created is [2008]
 - n
 - $2^n - 1$
 - 2^n
 - $2^{n+1} - 1$
- A CPU generally handles an interrupt by executing an interrupt service routine [2009]
 - As soon as an interrupt is raised.
 - By checking the interrupt register at the end of fetch cycle.
 - By checking the interrupt register after finishing the execution of the current instruction.
 - By checking the interrupt register at fixed time intervals.
- A thread is usually defined as 'light weight process' because an operating system (OS) maintains smaller data structures for a thread than for a process. In relation to this, which of the following is true? [2011]
 - On per-thread basis, the OS maintains only CPU register state
 - The OS does not maintain a separate stack for each thread
 - On per thread basis, the OS does not maintain virtual memory state.
 - On per thread basis, the OS maintains only scheduling and accounting information.
- Let the time taken to switch between user and kernel modes of execution be t_1 while the time taken to switch between two processes be t_2 . Which of the following is true? [2011]
 - Context switch time is longer for Kernel-level threads than for user level threads.
 - User level threads do not need any hardware support.
 - Related Kernel-level threads can be scheduled on different processors in a multi-processor system.
 - Blocking one Kernel-level thread blocks all related threads.

- (A) $t_1 > t_2$
 (B) $t_1 = t_2$
 (C) $t_1 < t_2$
 (D) Nothing can be said about the relation between t_1 and t_2

11. A process executes the code

```
fork();
fork();
fork();
```

The total number of **child** processes created is [2012]

- (A) 3 (B) 4
 (C) 7 (D) 8

12. Which one of the following is **FALSE**? [2014]

- (A) User level threads are not scheduled by the Kernel.
 (B) When a user level thread is blocked, all other threads of its process are blocked.

- (C) Context switching between user level threads is faster than context switching between Kernel-level threads.
 (D) Kernel-level threads cannot share the code segment.

13. Threads of a process share [2017]

- (A) global variables but not heap.
 (B) heap but not global variables.
 (C) neither global variables nor heap.
 (D) both heap and global variables.

14. Which of the following is/are shared by all the threads in a process? [2017]

- I. Program counter
 II. Stack
 III. Address space
 IV. Registers

- (A) I and II only (B) III only
 (C) IV only (D) III and IV only

ANSWER KEYS

EXERCISES

Practice Problems 1

1. B 2. A 3. B 4. D 5. D 6. D 7. C 8. B 9. C 10. D
 11. A 12. C 13. C 14. C 15. A

Practice Problems 2

1. A 2. C 3. B 4. C 5. C 6. A 7. B 8. D 9. C 10. D
 11. A 12. B 13. C 14. D

Previous Years' Questions

1. A 2. C 3. A 4. D 5. D 6. A 7. B 8. C 9. C 10. C
 11. C 12. D 13. D 14. B

Chapter 2

Interprocess Communication, Concurrency and Synchronization

LEARNING OBJECTIVES

- ▮ Principles of concurrency
- ▮ Process interaction
- ▮ Mutual exclusion
- ▮ Semaphores
- ▮ Binary semaphore
- ▮ Mutual exclusion using semaphores
- ▮ Progress using semaphores
- ▮ Classical problems of synchronization
- ▮ Dining philosophers problem
- ▮ Monitors
- ▮ Message passing
- ▮ Indirect addressing
- ▮ Mutual exclusion using message passing

BASIC CONCEPTS

Multiprogramming: It deals with the management of multiple processes within a uniprocessor system.

Multiprocessing: It deals with the management of multiple processes within a multiprocessor.

The fundamental operating system (OS) design is *concurrency*. Concurrency encompasses a host of design issues, including communication among processes, sharing of and competing for resources, synchronization of the activities of multiple processes and allocation of processor time to processes.

PRINCIPLES OF CONCURRENCY

There are two examples for concurrent processing as follows:

1. In a single-processor multiprogramming system, processes are interleaved in time to yield the appearance of simultaneous execution.
2. In a multiprocessor system, it is possible not only to interleave the execution of multiple processes but also to overlap them.

There are two problems with these techniques:

1. Problem with sharing of global resources
2. Problem with allocation of resources optimally.
3. Problem with locating a programming error as results is not deterministic and reproducible.

Example:

```
void process( )
{
    in = getchar( );
    out = in;
    putchar(out);
}
```

The procedure 'process' reads a character and prints it. Let us suppose that we have a uniprocessor system, with single user. Let the user running multiple applications and all applications use the procedure for reading and printing, that is, all the applications share common procedure for efficient and close interaction among them. But this sharing leads to problems. For example,

1. Let the process P_1 invokes 'process' and is interrupted immediately after 'getchar' returns its value and stores it in 'in'. Here the most recently entered character 'C' is stored in variable 'in'.
2. Now, suppose the process P_2 is activated and it invokes 'process', which runs to conclusion, inputting and then displaying a single character, D , on the screen.
3. The process P_1 is resumed. By this time, the value 'C' has been overwritten in 'in' and therefore lost. Instead 'in' contains 'D', which is transferred to 'out' and displayed.

Here the problem is with sharing a global variable. To avoid these types of problems, we impose some rules like, only one process

at a time may enter 'process' and that once in 'process' the procedure must run to completion before it is available for another process.

This problem is also applicable to multiprocessor systems.

Race condition: A race condition occurs when multiple processes or threads read and write data items so that the final result depends on the order of execution of instructions in the multiple processes.

Example: Let P_1 and P_2 be two processes that share global variables a and b , with initial values $a = 0$, $b = 1$. At some point in its execution, P_1 executes $a = a + b$ and P_2 executes $b = a + b$.

If P_1 executes before P_2 , then $a = 1$, $b = 2$.

If P_2 executes before P_1 , then $b = 1$, $a = 1$.

OS Concerns for Concurrency

1. The OS must be able to keep track of various processes using PCBs.
2. The OS must allocate and deallocate various resources for each active process.
3. The OS must protect the data and physical resources of each process.
4. The functioning of a process and the output it produces must be independent of the speed at which its execution is carried out relative to the speed of other concurrent processes.

PROCESS INTERACTION (IPC)

Process classification There are two types of processes as follows:

1. Independent/isolated
2. Cooperating

Independent Process: It cannot affect or be affected by the execution of another process.

Cooperating Process: It can affect or be affected by the execution of another process.

We can classify the ways in which processes interact on the basis of the degree to which they are aware of each other's existence. There are three types of process interaction as follows:

1. Process unaware of each other
2. Process indirectly aware of each other
3. Processes directly aware of each other

Competition Among Processes for Resources

1. This situation arises when processes unaware of each other.
2. There is no exchange of information between the competing processes.

3. But the execution of one process may affect the behaviour of competing processes.
4. With competing processes, there will be three control problems as follows:

Need for mutual exclusion

Example: Suppose two or more processes require access to a single non-sharable resource, such as a printer. Then that resource is referred as *critical resource* and the portion of the program that uses it is called *critical section* of the program. In the case of printer, only one process will have the control of printer while it prints an entire file.

Possibility of deadlock

Example: Two processes waiting for each other indefinitely for the release of resources.

Possibility of starvation

Example: One process is denied access to a particular resource which is required for the execution of that process.

Control of competition inevitably involves the OS, because it is the OS that allocates resources.

Cooperation Among Processes by Sharing

This situation arises when the processes are indirectly aware of each other. Processes may use and update the shared data without reference to other processes but know that other processes may have access to same data. So the control mechanism must ensure the integrity of the shared data. Problems with this type of sharing are

1. Mutual exclusion
2. Deadlock
3. Starvation
4. Data coherence

Data coherence

Suppose two items of data p and q are maintained in the relationship $p = q$, that is, any program that updates p and q values must maintain the relationship.

$$\begin{aligned} \text{Let } P_1 : p &= p + 1; \\ q &= q + 1; \\ P_2 : p &= p * 2; \\ q &= q * 2; \end{aligned}$$

Let initially the state is consistent, that is, $p = 2$, $q = 2$

Then the concurrent execution of P_1 and P_2 with mutual exclusion on p , q will be $p = p + 1$;

$$\begin{aligned} q &= q * 2; \\ q &= q + 1 \\ p &= p * 2; \end{aligned}$$

The final values of p and q will be $p = 6$, $q = 5$.

So the consistency is not maintained.

Cooperation Among Processes by Communication

This situation arises when processes are aware of each other directly. All the processes communicate with each other to synchronize or coordinate the various activities. Problems with this communication are as follows:

1. Deadlock
2. Starvation

BASIC DEFINITIONS

Atomic operations: A sequence of one or more statements that appears to be indivisible, that is, no process will interrupt the operation.

Critical section A section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code.

Deadlock A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.

Mutual exclusion The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.

Race condition A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.

Starvation A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.

CRITICAL SECTION

1. A section of code or set of operations, in which process may be changing shared variables, updating a common file or a table etc.
2. For the process that execute concurrently, it should ensure that execution of critical section should be made atomic. Atomic means that either an operation in the critical section should happen in its entirety or not at all.
3. Critical section of a process should not be executed concurrently with the critical section of another process.
4. To avoid Race Condition, we must have the following: 'if one process is in critical section, other competing process must be excluded to enter their critical sections, that is, a process must enter the critical section in a mutually exclusive way'.

This is called *problem of mutual exclusion*.

Region of code that updates or uses shared data to provide a consistent view of objects need to make sure an update is not in progress when reading the data.

5. Need to provide mutual exclusion for a critical section.

Requirements for Critical Section Problem

Mutual exclusion: No two contending processes should be simultaneously executing inside their critical section.

Bounded waiting: No process should have to wait forever to enter its critical section.

Progress: If no process is executing in its critical section and there exists some processes that wish to enter their critical sections, then only those processes that are not executing in the critical section can participate in the decision of which will enter its critical section next and this selection cannot be postponed indefinitely.

No Assumption: No assumption should be made about relative speed and properties of contending processes.

MUTUAL EXCLUSION

1. Only one process at a time can be updating shared objects.
2. Successful use of concurrency among processes requires the ability to define critical sections and enforce mutual exclusion.
3. Mutual exclusion in the use of a shared resource is provided by making its access mutually exclusive among the processes that share the resources.

Any facility that provides mutual exclusion should meet the following requirements:

1. No assumption regarding the relative speed of the process.
2. A process is in its critical section for a finite time only.
3. Only one process allowed in the critical section.
4. Process requesting access to critical section should not wait indefinitely.
5. A process waiting to enter critical section cannot be blocking a process in critical section or any other process.

Mutual exclusion can be satisfied in one of the following ways:

1. **Software approach:** This approach leaves the mutual exclusion responsibility with the process that wish to execute concurrently. This approach is prone to high processing overhead and bugs.
2. **Hardware support:** Use special-purpose machine instructions. This approach involves less overhead.
3. **Provide some level of support within the OS or a programming language:** Such techniques are as follows:
 - Semaphores
 - Monitors
 - Message passing

Hardware support for mutual exclusion: Use one of the following techniques:

1. Interrupt disabling
2. Special machine instructions
 - Compare and swap instructions
 - Exchange instructions

Interrupt Disabling

To provide mutual exclusion, it is sufficient to prevent a process from being interrupted. A process can enforce mutual exclusion in the following way:

```
while(true)
{
/* disable interrupts */;
/* critical section */;
/* enable interrupts */;
/* remainder */;
}
```

Here, the critical section is not interrupted, so mutual exclusion is guaranteed.

Problems with Interrupt Disabling

The efficiency of execution could be noticeably degraded, because the processor is limited in its ability to interleave processes. Interrupt disabling does not work on in a multi-processor architecture.

Special Machine Instructions

Two of the most commonly used special machine instructions are as follows:

1. Compare and swap
2. Exchange instruction

Compare and swap instructions It is defined as below:

```
int compare_and_swap(int *word, int
testval, int newval)
{
int oldval;
oldval = *word;
if(oldval == testval)
*word = newval;
return oldval;
}
```

- ‘Access to a memory location excludes any other access to that same location.’ On the basis of this principle, special machine instructions provide mutual exclusion.
- The above compare and swap instructions will check a memory location (*word) against a test value. If current value is test value, it is replaced with new value. Always the old value is returned.

- The below code provides mutual exclusion using compare and swap instructions:

```
/* Mutual Exclusion */
const int n = /* number of processes */
int S;
void P(int i)
{
while (true)
{
while (compare_and_swap(S, 0, 1) == 1)
/* do nothing */;
/* critical section */;
S = 0;
/* remainder */;
}
}
void main( )
{
S = 0;
begin (P(1), P(2) .... P(n));
}
```

Here, a shared variable ‘S’ is initialized to ‘0’. The only process that may enter its critical section is one that finds ‘S’ equal to 0.

All other processes that enter their critical sections go into a busy waiting mode.

Busy waiting or *spin waiting* is a technique in which a process can do nothing but continue to execute an instruction or set of instructions that tests the appropriate variable to gain entrance.

When a process leaves, its critical section sets ‘S’ to 0. Then the one of the waiting process will get access to enter its critical section.

Exchange instructions The exchange instructions can be defined as follows:

```
void exchange (int reg, int mem)
{
int temp;
temp = mem;
mem = reg;
reg = temp;
}
```

Mutual Exclusion Using Exchange Instructions

```
/* Mutual Exclusion */;
int const n = /* number of processes */;
int S;
void P(int i)
{
int ki = 1;
```

```

while (true)
{
do
exchange(ki, S);
while(ki! = 0);
/* critical section */;
S = 0;
/* remainder */;
}
}
void main( )
{
S = 0;
begin (P(1), P(2), ... P(n));
}

```

A shared variable 'S' is initialized to '0'. Each process uses a local variable *ki* that is initialized to 1. The only process that may enter its critical section is one that finds 'S' equal to 0. It excludes all other processes from critical section by setting 'S' to 1. When a process leaves its critical section, it resets 'S' to 0 allowing another process to its critical section.

Advantages of using machine instruction approach

1. Applicable to any number of processes on either a single processor or multiple processors, sharing the main memory.
2. Simple and easy to verify.
3. Used to support multiple critical sections.

Disadvantages

1. Busy waiting
2. Starvation is possible
3. Deadlock is possible

OTHER MECHANISMS FOR MUTUAL EXCLUSION

Let us discuss OS and programming language mechanisms that are used to provide concurrency.

Semaphores

Semaphore is an integer value used for signalling among processes.

There are two types of semaphores as follows:

1. Binary semaphore
2. Counting (or) general semaphore

Counting or general semaphore

Three operations may be performed on a semaphore all of which are atomic:

- Initialize
- Decrement
- Increment

The working of a counting semaphore with its operations is defined as below:

1. The semaphore may be initialized to a non-negative integer value.
2. The semwait operation decrements the semaphore value. If the value becomes negative, then the process executing the semwait is blocked, otherwise the process continues execution.
3. The semsignal operation increments the semaphore value. If the resulting value is less than or equal to zero, then one of the processes blocked by a semwait operation, if any, is unblocked.

Example: Let the semaphore value $S = 3$.

If the semaphore value is positive, then that value gives the number of processes that can issue a wait and immediately continue to execute.

Let five processes P_1, P_2, P_3, P_4, P_5 are going to execute a critical section code based on the semaphore value $S = 3$.

```

S = 3 ← Initially
    ↓ P1 sem wait
S = 2 ≥ 0, P1 executes
    ↓ P2 sem wait
S = 1 ≥ 0, P2 executes
    ↓ P3 sem wait
S = 0 ≥ 0, P3 executes
    ↓ P4 sem wait
S = -1 ≠ 0, P4 Blocked
    ↓ P5 sem wait
S = -2 ≠ 0, P5 Blocked

```

Initially, the 'S' value 3 means that at a time three processes can issue a 'wait' signal and continue execution.

Whenever S becomes 0, the next process which executes 'wait' operation will be blocked.

Here P_4 is blocked as it operates on the semaphore when $S = 0$.

If the semaphore value becomes negative, it specifies the number of processes waiting to be unblocked.

$S = -2$ means two processes are waiting to be unblocked.

Definition of semwait and semsignal operations

```

struct semaphore
{
int semvalue;
QueueType Queue;
};
void semwait(semaphore S)

```

```

{
S.semvalue--;
if (S.semvalue < 0)
{
/* place the process in S. Queue */;
/* Block this process */;
}
}
void semsignal(semaphore S)
{
S.semvalue++;
if (S.semvalue <= 0)
{
/* remove a process from S. Queue */;
/* Place the process in ready queue */;
}
}

```

Advantages

1. Because the waiting processes will be permitted to enter their Critical Section in a FCFS order, so the requirement of bounded waiting is met.
2. CPU cycles are saved here as waiting process does not perform any busy waiting.

Disadvantages

1. Complex to implement, since it involves implementation of FCFS.
2. Context switching is more, so more overheads are involved.

Binary semaphore It is a semaphore that takes on only the values 0 and 1.

The operations performed on a binary semaphore are as follows:

1. A binary semaphore may be initialized to 0 or 1.
2. The semwaitB operation checks the semaphore value. If the value is 0, then the process executing the semwaitB is blocked. If the value is 1, then the value is changed to 0 and the process continues execution.
3. The semsignalB operation checks to see if any processes are blocked on this semaphore. If so, then a process blocked by a semwaitB operation is unblocked. If no processes are blocked, then the value of the semaphore is set to 1.

Definition of semwaitB and semsignalB

```

struct binary-semaphore
{
enum {zero, one} value;
Queue-type Queue;
};
void semwaitB(binary-semaphore S)
{

```

```

if (S.value == one)
S.value = zero;
else
{
/* Place this process in S.Queue */;
/* Block this process */;
}
}
void semsignalB(binary-semaphore S)
{
if(S.Queue is empty)
S.value = one;
else
{
/* remove a process from S.Queue */;
/* Place process in ready list */;
}
}

```

Advantages

1. The implementation of binary semaphore is extremely simple.

Disadvantages

1. It does not meet the requirement of Bounded waiting.
2. A process, waiting to enter its Critical Section, will perform Busy waiting, thus wasting CPU cycles.

Notes:

1. Binary semaphores have the same expressive power as general semaphores.
2. **MUTEX**: It is similar to binary semaphore. The key difference between the two is that the process that locks the mutex must be the one to unlock it.
3. Both counting semaphores and binary semaphores use a queue to hold processes waiting on the semaphore. The order in which the processes removed from a Queue is FIFO, that is, the process that has been blocked the longest is released from Queue first.
4. A semaphore whose definition includes the order of removal of Blocked processes is referred as a strong semaphore otherwise it is a weak semaphore.
5. Strong semaphores guarantee freedom from starvation.

Mutual Exclusion Using Semaphores

```

const int n = /* number of processes */;
Semaphore S = 1;
void P(int i)
{
while (true)
{
Semwait(S);
/* critical section */;
Semsignal(S);

```

```

/* remainder */;
}
}
void main( )
{
begin (P(1), P(2), ... P(n));
}

```

If no process is executing in Critical Section, then the semaphore value is 1. The first process that is executing 'wait' operation will decrement value to 0 and enter its critical section. The process which execute 'wait' operation while a cooperating process is executing in its critical section, will find the semaphore value to 0 and keep looping in the 'while-loop' of 'wait' operation. Spinning of a waiting process in the while-loop, the binary semaphores are also known as spin locks. When the process executing in the CS makes an exit (from CS), it will execute the 'signal' operation and increment the semaphore value to 1.

At a time, only one of the cooperating processes can enter critical section with the condition that the wait operation is executed automatically. Mutual Exclusion is satisfied.

Example: Consider the following figure (Figure 1) which shows the possible sequence of three processes using mutual exclusion with a semaphore, *S*. Processes *P*, *Q*, *R* accesses a shared resource protected by the semaphore *S*.

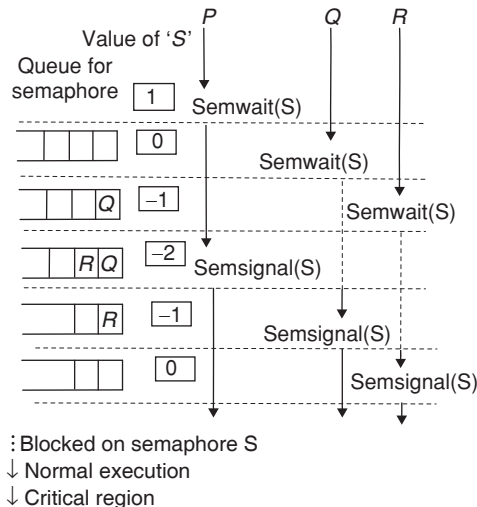


Figure 1 Mutual exclusion using semaphore.

Progress Using Semaphores

When no process is executing in the CS, the semaphore value will be 1. Then, one of the waiting process looping in the while loop of wait-operation will find the semaphore value of 1, exit from the while-loop, decrement the semaphore value to 0 and enter CS. Thus, if no process is executing in critical section and some are waiting to enter, then one of the waiting processes will enter its critical section immediately.

Bounded waiting using semaphores

Actually, one of the waiting processes will get entry into its CS when an operating process executing in its critical section exits. As this selection process is arbitrary, so a process waiting to enter its CS is likely to face starvation. So, the requirement of bounded waiting is not met.

CLASSICAL PROBLEMS OF SYNCHRONIZATION

We will discuss three problems of synchronization:

1. Bounded buffer problem
2. Readers/writers problem
3. Dining philosophers problem

Producer-Consumer Problem

1. Producer inserts item in the buffer
2. Updates Insertion pointer
3. Consumer consumes items in the buffer
4. Updates removal pointer
5. Both update information about how full, how empty the buffer.
6. Prevents buffer overflow, prevents buffer underflow, proper synchronization.

Producer	Consumer
repeat	repeat
produce item <i>v</i> ;	while (<i>in</i> <= <i>out</i>);
<i>b</i> [<i>in</i>] = <i>v</i> ;	<i>w</i> = <i>b</i> [<i>out</i>];
<i>in</i> = <i>in</i> + 1;	<i>out</i> = <i>out</i> + 1;
forever;	consume <i>w</i> ;
	forever;

Table 1 Producer consumer problem solution using semaphores

Producer	Consumer
repeat	repeat
produce item <i>v</i> ;	while (<i>in</i> <= <i>out</i>);
SemwaitB(<i>S</i>);	SemwaitB(<i>S</i>);
<i>b</i> [<i>in</i>] = <i>v</i> ;	<i>w</i> = <i>b</i> [<i>out</i>];
<i>in</i> = <i>in</i> + 1;	<i>out</i> = <i>out</i> + 1;
SemwaitB(<i>S</i>);	SemwaitB(<i>S</i>);
forever;	consume <i>w</i> ;
	forever;

If producer is slow or late, then consumer will busy at the while statement.

Table 2 Improved solution

Producer	Consumer
repeat	repeat
produce item <i>v</i> ;	Semwait(<i>n</i>);
Semwait(<i>S</i>);	Semwait(<i>S</i>);
<i>b</i> [<i>in</i>] = <i>v</i> ;	<i>w</i> = <i>b</i> [<i>out</i>];
<i>in</i> = <i>in</i> + 1;	<i>out</i> = <i>out</i> + 1;
Semwait(<i>S</i>);	Semwait(<i>S</i>);
Semwait(<i>n</i>);	consume <i>w</i> ;
forever;	forever;

The initial value of n and S are $n = 0$, $S = 1$. (n is the number of items in the buffer).

Table 3 Producer consumer bounded buffer problem

Producer	Consumer
repeat	repeat
produce item v ;	while($in == out$)
while($(in + 1) \% n == out$)	no operation;
no operation;	$w = b[out]$;
$b[in] = v$;	$out = (out + 1) \% n$;
$in = (in + 1) \% n$;	consume w ;
forever;	forever;

The buffer size is enforced using another counting semaphore.

Table 4 Producer consumer bounded buffer problem solution

Producer	Consumers
Repeat	repeat
Produce item v ;	Semwait(e);
Semwait(e);	Semwait(S);
Semwait(S);	$w = b[out]$;
$b[in] = v$;	$out = (out + 1) \% n$;
$in = (in + 1) \% n$	Semsignal(S);
Semsignal(S);	Semsignal(e);
Semsignal(e);	consume w ;
forever;	forever;

The initial value of buffer size, e is the size of the bounded buffer.

Observations on semaphores:

1. Semaphores are easy to use.
2. wait() and signal() are to be implemented as atomic operations.

Problems:

1. signal() and wait() may be exchanged by the programmer, this may result in deadlock or violation of mutual exclusion.

Readers/Writers Problem

1. A reader reads data.
2. A writer writes data.
3. Data is shared among a number of processes.
4. Multiple readers may read the data simultaneously, that is, concurrently.
5. Only one writer can write the data any time, that is, no reader should be present.
6. A reader and writer cannot access data simultaneously.
7. Locking table: Whether any two can be in the critical section simultaneously is shown in the table.

	Reader	Writer
Reader	OK	NO
Writer	NO	NO

Solution: Readers have priority; if a reader is in CS, any number of readers could enter irrespective of any writer waiting to enter critical section

Writer	Reader
while(true)	while(true)
{	{
Semwait(S);	Semwait(x);
writeunit();	Num = Num + 1;
Semsignal(S);	if (Num == 1)
}	Semwait(S);
	Semsignal(x);
	Readunit();
	Semwait(x);
	Num = Num - 1;
	if (Num == 0)
	Semsignal(S);
	Semsignal(x);
	}

Semaphore ' S ' is used to enforce mutual exclusion.

Semaphore ' x ' is used to assure that ' Num ' is updated properly.

Solution: If a writer wants critical section as soon as the critical section is available, writer enters it.

Dining Philosophers Problem

N philosophers are sitting around a dining table. There are N plates placed on the table such that each plate is in front of a philosopher and N forks placed between the plates. There is a bowl of Noodles placed at the centre of the table. Whenever a philosopher feels hungry, he tries to pick two forks which are shared with his nearest neighbour. If any of his neighbours happens to be eating at the time, the philosopher has to wait. Whenever a hungry philosopher gets two forks, he pours noodles into his plate. After he finishes, he places the chopsticks back onto the table and starts thinking. Now forks are available for neighbours.

Solution:

```
# define N 5 /* Number of philosophers*/
void philosopher(int i) /* philosopher number,
from 0 to 4*/
{
    while (true)
    {
        think(); /* philosopher is thinking*/
        take_fork(i); /*take left fork*/
        take_fork((i+1)% N); /* take right fork; %
is modulo operator*/
        eat( );
        put_fork( ); /* put left back on the table*/
        put_fork((i+1) % N); /* put right fork
back on the table */
    }
}
```

Notes:

1. This solution leads to deadlock.
2. Everyone picks the left fork and indefinitely wait for right fork causing starvation.

MONITORS

Monitor is a programming language construct that encapsulates variables, access procedures and initialization code within an abstract data type. The monitor's variable may only be accessed via its access procedures and only one process may be actively accessing the monitor at any one time. The access procedures are critical sections. A monitor may have a queue of processes that are waiting to access it.

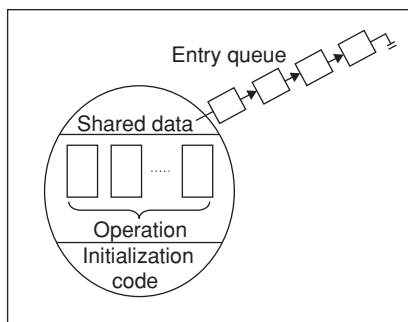
1. If the data in a monitor represent some resource, then the monitor provides a mutual exclusion facility for accessing the resource.
2. A monitor supports synchronization by the use of condition variables that are contained within the monitor and accessible only within the monitor.
3. Operations on conditional variables:
 - **cwait(c):** Suspend execution of calling process on condition *c*. The monitor is now available for use by another process.
 - **csignal(c):** Resume execution of some process blocked after a wait on the same condition. If there are several such processes, choose one of them; if there is no such process, do nothing.

Monitor syntax is as follows:

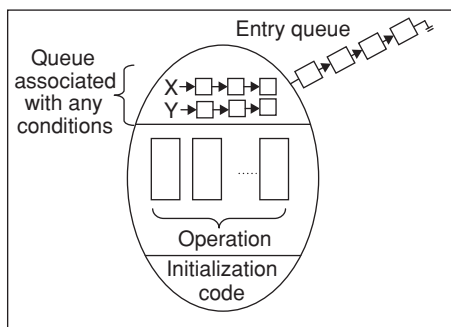
Monitor monitor - name

```
{
shared variable declarations;
Procedure body P1 (...) { }
Procedure body P2 (...) { }
Procedure body Pn (...) { }
initialization code { }
}
```

Schematic view of a monitor:



Schematic View of a monitor with condition variables:



MESSAGE PASSING

When processes interact with one another, two fundamental requirements must be satisfied:

1. Synchronization
2. Communication

One approach to provide both of these is *message passing*.

The primitive functions in message passing are send (destination, message) receive (source, message)

Design Characteristics of Message Systems for IPC and Synchronization

Synchronization There must be some synchronization existing between two processes to communicate with each other.

- **Send:** When a 'send' primitive is executed in a process, then the sender may
 - blocked or
 - non-blocked
 - until the message is received.
- **Receive:** When a process issues a 'Receive' primitive there are two possibilities:
 1. If a message has previously been sent, the message is received and execution continues.
 2. If there is no waiting message then either
 - (i) The process is blocked until a message arrives or
 - (ii) The process continues to execute, abandoning the attempt to receive.

Thus, both the sender and receiver may be in one of

1. Blocking send, blocking receive: Allows tight synchronization.
2. Non-blocking send, blocking receive:
 - Useful synchronization
 - Possibility of generating repeated messages
3. Non-blocking send, non-blocking receive: No need to wait

Addressing Two types of addressing methods:

1. Direct addressing
2. Indirect addressing

Direct Addressing

The send primitive includes a specific identifier of the destination process. The 'receive' primitive can be handled in one of two ways:

Explicit

The process must know ahead of time from which process a message is expected. Useful for cooperating concurrent processes.

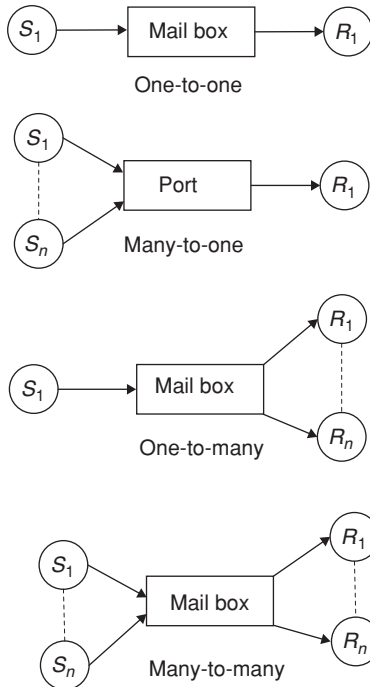
Implicit

The 'source' parameter of 'receive' primitive possesses a value returned when the receive operation has been performed. Example, Printer server.

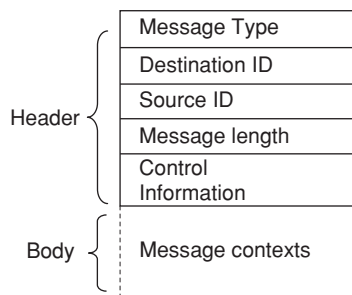
Indirect Addressing

Messages are not sent directly from sender to receiver but rather are sent to a shared data structure consisting of queues that can temporarily hold messages. These queues are referred to as mailboxes. The relationship between the sender and receiver is

- one-to-one
- many-to-one
- one-to-many
- many-to-many



Message format The general message format will be



Queuing discipline The queuing discipline may be

- FIFO
- Priority

Mutual Exclusion using message passing:

```
const int n = /* number of processes */;
void P(int i)
{
    message msg;
    while (true)
    {
        receive (box, msg);
        /* critical section */;
        send (box, msg);
        /* remainder */;
    }
}

void main( )
{
    create mailbox(box);
    send(box, null);
    begin (P(1), P(2) ... P(n));
}
```

- Here a set of concurrent processes that share a mailbox 'box', which can be used by all processes to send and receive.
- The mailbox is initialized to contain a single message with null content. A process wishing to enter its critical section first attempts to receive a message.
- If the mail box is empty, then the process is blocked.
- Once a process acquired the message, it performs its critical section and then places the message back into the mailbox.
- Hence, the message functions as a token that is passed from process to process.

EXERCISES

Practice Problems I

Directions for questions 1 to 17: Select the correct alternative from the given choices.

- The value of a counting semaphore is 7. Then 15 wait operations and 10 signal operations were completed on this semaphore. The resulting value of semaphore is
(A) 5 (B) 7
(C) 2 (D) 0
- At a particular time of computation, the value of counting semaphore is 7. Then 20 wait operations and 'x' signal operations were completed on this semaphore. If the final value of the semaphore is 5, what is x?
(A) 18 (B) 13
(C) 5 (D) 0
- A process using a semaphore has a start value of 1 for its semaphore. Since the start of execution of the program, 12 signal operations were completed. How many wait operations have been completed so far if the current value of semaphore is 6?
(A) 1 (B) 5
(C) 7 (D) 11

4. It is found that a program has multiple critical sections. Choose correct statements from below:
- (i) Multiple semaphores are needed for handling the situation.
 - (ii) A single semaphore that uncompresses all the critical section is sufficient and is also more efficient.
 - (iii) To get better control of the code, monitors need to be implemented.
- (A) (i) and (ii) (B) (ii) and (iii)
 (C) (i) and (iii) (D) (i), (ii), (iii)

5. Consider the below pseudocode:

```
semaphore S = 1;
semaphore E = 1;
if(thread_count++ < 100)
spawnnewthread( );
wait(E);

// critical section - begin
-----
-----
// critical section - end
signal(S);
```

Assume that above pseudocode gets called a hundred times, what is the count of semaphore *E*?

- (A) 0 (B) 1
 (C) -99 (D) -100
6. Consider the below code for a process *i*:

```
-----
-----
flag[i] = true;
if(turn == i and flag[i] == true)
/* critical section begin */
counter++;
/* critical section end */
turn = x;
-----
-----
```

If the value of a counter started, what would be the value of 'counter' count at the end of the program:

- (A) Semaphore count
 (B) Thread count
 (C) Concurrency count
 (D) Deadlock process count

7. Consider the below pseudocode:

```
function waitB(s)
{
if(s.value == 1)
s.value = 0;
else
place the process in the Queue;
```

```
}
function signalB(s)
{
s.value = 1;
}
```

What does the code most likely behave as

- (A) general semaphore (B) weak semaphore
 (C) binary semaphore (D) mutex

8. A shared variable *x*, initialized to 0 is operated on by four concurrent processes *P*, *Q*, *R*, *S* as follows:

<pre>P(x) { wait(); read(x); increment x by 1; store(x); signal(); } R(x) { wait(); read(x); decrement x by 2; store(x); signal(); }</pre>	<pre>Q(x) { wait(); read(x); increment x by 1; store(x); signal(); } S(x) { wait(); read(x); decrement x by 2; store(x); signal(); }</pre>
---	--

A counting semaphore '*N*' is used by the processes whose value is initialized to 2. What is the maximum possible value of '*x*' after all processes complete execution?

- (A) -2 (B) -1
 (C) 1 (D) 2

9. Consider the following code:

```
Program concurrency;
Var x: Integer (: = 0);
    y: Integer (: = 0);
Procedure threadA( ) ;
begin
x = 1; /*S1*/
y = y + x; /*S2*/
end;
Procedure threadB( ) ;
begin
y = 4; /*S3*/
x = x + 5; /*S4*/
end;
begin /*mainprogram*/
parbegin
threadA( );
threadB( );
parend;
end.
```

Suppose a process has two concurrent threads: one thread executes statements S_1 and S_2 , and the other thread executes statements S_3 and S_4 . What are the maximum possible values of x and y when the code finishes execution? (All the statements S_1, S_2, S_3 and S_4 are atomic).

- (A) $x = 6, y = 4$ (B) $x = 6, y = 5$
 (C) $x = 1, y = 5$ (D) $x = 6, y = 10$

10. Consider the following program:

```
boolean lock[2];
int turn;
void P(int id)
{
    while(true)
    {
        lock[id] = true;
        while (turn != id)
        {
            while (lock [1 - id])
            /*do nothing*/
            turn = id;
        }
        /*critical section*/
        lock[id] = false;
        /*remainder*/
    }
}
void main( )
{
    lock[0] = false;
    lock[1] = false;
    turn = 0;
    parbegin (P(0), P(1));
}
```

Which of the following statements is correct for two processes executing this code?

- (A) Given program provides mutual exclusion.
 (B) Given program does not provide mutual exclusion.
 (C) Given program provides mutual exclusion and also solves starvation problem.
 (D) Given program provides mutual exclusion but does not prevent from starvation.

11. Consider two process P_0 and P_1 which share the following variables:

```
boolean flag [2]; /*initially false*/
int turn;
```

These two processes, $P_i (i = 0 \text{ or } 1)$, $P_j (j = 1 \text{ or } 0)$ execute the following code:

```
do
{
    flag[i] = TRUE;
    while(flag [j])
```

```
{
    if (turn == j)
    {
        flag[i] = false;
        while (turn == j);
        flag [i] = TRUE;
    }
}
// critical section
turn = j;
flag[i] = FALSE;
// remainder.
}
```

while(TRUE);

The code satisfies

- (i) Mutual exclusion
 (ii) Progress
 (iii) Bounded waiting
 (A) (i), (ii) only (B) (ii), (iii) only
 (C) (i), (iii) only (D) (i), (ii), (iii)

12. Which of the following statement(s) is false?

- (i) Spinlocks are not appropriate for single-processor systems.
 (ii) Mailboxes may be used for synchronization.
 (iii) Message passing and semaphores do not have equivalent functionality.
 (A) (i) only (B) (iii) only
 (C) (i), (iii) (D) (i), (ii), (iii)

13. Consider the following code:

```
signal (mutex);
.....
Critical section
.....
```

```
wait (mutex);
```

Here 'mutex' is a semaphore variable, which is initialized to 1. Then

- (A) Mutual exclusion is provided
 (B) Mutual exclusion violated, if several processes are simultaneously active in their critical section.
 (C) Deadlock will occur
 (D) Starvation is possible

14. Which of the following sequence of 'wait' and 'signal' operations leads to deadlock?

(Here 'mutex' is a semaphore variable initialized to 1.)

- (A) wait (mutex);

 Critical section

 Signal (mutex);

```
(B) wait (mutex);
.....
Critical section
.....
Wait (mutex);
(C) Signal (mutex);
.....
Critical section
.....
Wait (mutex);
(D) signal (mutex);
.....
Critical section
.....
Signal (mutex);
```

15. Which of the following situation arises if a process omits the wait(S) or the signal(S) on a semaphore variable 'S' (Initially $S = 1$).

(i) Mutual exclusion violated
 (ii) Deadlock will occur
 (A) (i) only (B) (ii) only
 (C) Both (i) and (ii) (D) Neither (i) nor (ii)

16. consider the following shared data and code:

```
data:
int turn;
Boolean flag[2];
Code:
do
{
flag [i] = TRUE;
```

```
turn = j;
while (flag [j] && turn == j);
//critical section
flag [i] = FALSE;
//remainder
}
```

```
while (TRUE);
```

Let two processes P_i ($i = 0$ or 1) and P_j ($j = 1$ or 0) use the shared data and executes the code. Then the code provides

(A) a solution to critical section problem
 (B) mutual exclusion but not progress.
 (C) progress but not mutual exclusion
 (D) both mutual exclusion, progress but no bounded waiting.

17. Consider the following code that shows the structure of a process in an algorithm to solve the critical section problem for two processes.

```
var flag[2] of Boolean; /* initialized to false */
repeat
flag[i] = true;
while flag[j] do no-op;
//critical section
flag[i] = false;
// remainder
until false
```

Then which of the following statements is true?

(A) The algorithm satisfies all the requirements of critical section problem.
 (B) The algorithm satisfies only mutual exclusion and progress.
 (C) The algorithm only satisfies progress requirement.
 (D) The algorithm does not satisfy critical section problem requirements.

Practice Problems 2

Directions for questions 1 to 15: Select the correct alternative from the given choices.

- The result of a computation depends upon the speed of the processes involved, is said to be:
 (A) Cycle stealing (B) Race condition
 (C) A time lock (D) A deadlock
- A relation between processes such that each has some part which must not be executed, while the critical section of another is being executed is known as
 (A) Mutual exclusion (B) Semaphore
 (C) Multi-tasking (D) Multi-programming
- Producer-consumer problem can be solved using
 (A) Semaphores (B) Event counters
 (C) Monitors (D) All of the above
- To avoid the race condition, the number of processes allowed in critical section is
 (A) 0 (B) 1
 (C) 2 (D) 3

- Mutual exclusion problem occurs between
 (A) Two disjoint processes that unaware of each other
 (B) Processes that share resources
 (C) Processes directly aware of each other.
 (D) Both (A) and (B)
- Semaphores are used to solve the problem of
 (A) Race condition (B) Multitasking
 (C) Mutual exclusion (D) Both (A) and (C)
- At a particular time, the value of a counting semaphore is 10. It will become 7 after
 (A) 3 signal operations
 (B) 3 wait operations
 (C) 5 signal operations and 2 wait operations
 (D) None of the above
- Critical region is
 (A) A part of the OS which is not allowed to be accessed by any process
 (B) A set of instructions that accesses common shared resource, which exclude one another in time

- (C) The portion of main memory, which can be accessed only by one process at a time
(D) Both (A) and (C)
9. Concurrent processes are:
(A) Processes that don't overlap in time
(B) Processes that overlap in time
(C) Processes that are executed
(D) Processes that are executed by a processor at the same time
10. Semaphore operations are atomic because they are implemented within the _____.
(A) Kernel (B) Shell
(C) User process (D) Normal process space
11. The programming language construct that provides equivalent functionality of a semaphore and better control is
(A) Signal (B) Monitor
(C) Mutex (D) Critical section.
12. What is the ideal way of emptying the queue of a strong semaphore?
(A) Random (B) LIFO
(C) FIFO (D) binary
13. What are the disadvantages of machine instruction approach?
(i) While a process is waiting for entering a critical section, process still consumes resources.
(ii) There could be starvation
(iii) There could be deadlocks
(A) (i), (ii) only (B) (ii), (iii) only
(C) (iii), (i) only (D) (i), (ii), (iii)
14. Select from below the advantages of Machine Instruction approach?
(i) Applicable to any number of processes either uni-processor or multi-processor system
(ii) Simple and easy to verify
(iii) Supports multiple critical sections
(A) (i), (ii) only (B) (ii), (iii) only
(C) (iii), (i) only (D) (i), (ii), (iii)
15. Which of the below are requirements for mutual exclusion?
(i) Only one process is allowed into critical section.
(ii) A process remains inside its critical section for finite time only.
(iii) It must be possible for a process accessing critical section to be delayed indefinitely.
(iv) A process halting in critical section must do so without interfering with other processes.
(A) (i), (ii), (iii) (B) (ii), (i), (iv)
(C) (i), (ii), (iv) (D) (i), (ii), (iii), (iv)

PREVIOUS YEARS' QUESTIONS

1. Consider these two functions and two statements S_1 and S_2 about them: [2006]

<pre>int work1 (int *a,int i, int j) { int x=a[i+2]; a[j]=x+1; return a[i+2]-3; }</pre>	<pre>int work2 (int *a,int i, int j) { int t1=i+2; int t2=a[t1]; a[j]=t2+1; return t2 - 3; }</pre>
---	--

- S1:** The transformation from work1 to work2 is valid, that is, for any program state and input arguments, work 2 will compute the same output and have the same effect on program state as work 1
- S2:** All the transformations applied to work 1 to get work 2 will always improve the performance (i.e., reduce CPU time) of work 2 compared to work 1
- (A) S_1 is false and S_2 is false
(B) S_1 is false and S_2 is true
(C) S_1 is true and S_2 is false
(D) S_1 is true and S_2 is true
2. The atomic fetch-and-set x, y instructions unconditionally sets the memory location x to 1 and fetches the old value of the of x in y without allowing any

intervening access to the memory location x . Consider the following implementation of P and V functions on binary semaphore S .

```
void P (binary-semaphore *s) {
    unsigned y;
    unsigned *x = & (s → value);
    do {
        fetch-and-set x, y ;
    } while (y) ;
}

void V (binary-semaphore *s) {
    s → value = 0;
}
```

Which one of the following is true?

[2006]

- (A) The implementation may not work if context switching is disabled in P
(B) Instead of using fetch-and-set, a pair of normal load/store can be used
(C) The implementation of V is wrong
(D) The code does not implement a binary semaphore
3. The P and V operations on counting semaphores, where s is a counting semaphore, are defined as follows:

```

P(s) : s = s - 1;
        if s < 0 then wait;
V(s) : s = s + 1;

```

if $s \leq 0$ then wake up a process waiting on s ;

Assume that P_b and V_b , the wait and signal operations on binary semaphores, are provided. Two binary semaphores x_b and y_b are used to implement the semaphore operations $P(s)$ and $V(s)$ as follows:

```

P(s) :      P_b (x_b) ;
           s = s - 1;
           if (s < 0) {
               V_b (x_b) ;
               P_b (y_b) ;
           }
           else V_b (x_b) ;

```

```

V(s) :      P_b (x_b) ;
           s = s + 1;
           if (s <= 0) V_b (y_b) ;
           V_b (x_b) ;

```

The initial values of x_b and y_b are respectively

[2008]

- (A) 0 and 0 (B) 0 and 1
(C) 1 and 0 (D) 1 and 1

4. Consider a system with four types of resources R_1 (3 units), R_2 (2 units), R_3 (3 units), R_4 (2 units). A non-pre-emptive resource allocation policy is used. At any given instance, a request is not entertained if it cannot be completely satisfied. Three processes P_1, P_2, P_3 request the resources as follows if executed independently.

[2009]

Process P_1 :	Process P_2 :	Process P_3 :
$t=0$: requests 2 units of R_2	$t=0$: requests 2 units of R_3	$t=0$: requests 1 unit of R_4
$t=1$: requests 1 unit of R_3	$t=2$: requests 1 unit of R_4	$t=2$: requests 2 units of R_1
$t=3$: requests 2 units of R_1	$t=4$: requests 1 unit of R_1	$t=5$: releases 2 units of R_1
$t=5$: releases 1 unit of R_2 and 1 unit of R_1 .	$t=6$: releases 1 unit of R_3	$t=7$: requests 1 unit of R_2
$t=7$: releases 1 unit of R_3	$t=8$: Finishes	$t=8$: requests 1 unit of R_3
$t=8$: requests 2 units of R_4		$t=9$: Finishes
$t=10$: Finishes		

Which one of the following statements is true if all three processes run concurrently starting at time $t=0$?

- (A) All processes will finish without any deadlock
(B) Only P_1 and P_2 will be in deadlock.
(C) Only P_1 and P_3 will be in a deadlock.
(D) All three processes will be in deadlock.

5. The enter_CS() and leave_CS() functions to implement critical section of a process are realized using test-and-set instruction as follows: [2009]

```

void enter_CS(X)
{
    while (test-and-set(X)) ;
}
void (leave_CS(X))
{
    X=0;
}

```

In the above solution, X is a memory location associated with the CS and is initialized to 0. Now consider the following statements:

- (i) The above solution to CS problem is deadlock-free.
(ii) The solution is starvation free.
(iii) The processes enter CS in FIFO order.
(iv) More than one process can enter CS at the same time.

Which of the above statements is true?

- (A) (i) only
(B) (i) and (ii)

- (C) (ii) and (iii)
(D) (iv) only

6. Consider the methods used by processes P_1 and P_2 for accessing their critical sections whenever needed, as given below. The initial values of shared Boolean variables S_1 and S_2 are randomly assigned.

[2010]

Method used by P_1	Method used by P_2
while ($S_1 = S_2$); Critical section $S_1 = S_2$;	while ($S_1 \neq S_2$); Critical section $S_2 = \text{not } (S_1)$;

Which one of the following statements describes the properties achieved?

- (A) Mutual exclusion but not progress
(B) Progress but not mutual exclusion
(C) Neither mutual exclusion nor progress
(D) Both mutual exclusion and progress

7. The following program consists of three concurrent processes and three binary semaphores. The semaphores are initialized as $S_0 = 1, S_1 = 0, S_2 = 0$.

Process P_0	Process P_1	Process P_2
while (true) { wait (S_0); print '0' release (S_1); release (S_2); }	wait (S_1); Release (S_0);	wait (S_2); release (S_0);

How many times will process P_0 print '0'? [2010]

- (A) At least twice (B) Exactly twice
(C) Exactly thrice (D) Exactly once

8. Fetch_And_Add(X, i) is an atomic Read-Modify-Write instruction that reads the value of memory location X , increments it by the value i , and returns the old value of X . It is used in the pseudocode shown below to implement a busy wait lock. L is an unsigned integer shared variable initialized to 0. The value of 0 corresponds to lock being available, while any non-zero value corresponds to the lock being not available.

[2012]

```
AcquireLock(L) {
    while(Fetch_And_Add(L, 1))
        L = 1;
}
ReleaseLock (L) {
    L = 0;
}
```

This implementation

- (A) fails as L can overflow
(B) fails as L can take on a non-zero value when the lock is actually available
(C) works correctly but may starve some processes
(D) works correctly without starvation
9. A shared variable x , initialized to 0, is operated on by four concurrent processes W, X, Y, Z as follows. Each of the processes W and X reads x from memory, increments by one, stores it to memory, and then terminates. Each of the processes Y and Z reads x from memory, decrements by two, stores it to memory, and then terminates. Each process before reading x invokes the P operation (i.e., wait) on a counting semaphore S and invokes the V operation (i.e., signal) on the semaphore S after storing x to memory. Semaphore S is initialized to 2. What is the maximum possible value of x after all processes complete execution? [2013]
- (A) -2 (B) -1
(C) 1 (D) 2

10. A certain computation generates two arrays ' a ' and ' b ' such that $a[i] = f(i)$ for $0 \leq i < n$ and $b[i] = g(a[i])$ for $0 \leq i < n$. Suppose this computation is decomposed into two concurrent processes X and Y such that X computes the array ' a ' and Y computes the array ' b '. The processes employ two binary semaphores R and S , both initialized to zero. The array ' a ' is shared by the two processes. The structures of the processes are shown below.

Process X:

```
private i;
for (i=0; i<n; i++) {
    a[i] = f(i);
    ExitX(R, S);
}
```

Process Y:

```
private i;
for (i=0; i<n; i++) {
    EntryY(R, S);
    b[i] = g(a[i]);
}
```

Which one of the following represents the correct implementations of ExitX and EntryY?

[2013]

- (A) ExitX(R, S) {
 P(R);
 V(S);
}
EntryY(R, S) {
 P(S);
 V(R);
}
- (B) ExitX(R, S) {
 V(R);
 V(S);
}
EntryY(R, S) {
 P(R);
 P(S);
}
- (C) ExitX (R, S) {
 P(S);
 V(R);
}
EntryY(R, S) {
 V(S);
 P(R);
}
- (D) ExitX (R, S) {
 V(R);
 P(S);
}
EntryY(R, S) {
 V(S);
 P(R);
}

11. Consider the procedure below for the *producer-consumer* problem which uses semaphores; [2014]

```
semaphore n = 0;
semaphore s = 1;
void producer ()
{
    while (true)
    {
        produce ( )
        semWait (s);
        addToBuffer ();
        semSignal (s);
        semSignal (n);
    }
}
void consumer ()
{
    while (true)
    {
        semWait (s);
        semWait (n);
        remove FromBuffer ();
        semSignal(s);
        consume ( ) ;
    }
}
```

Which one of the following is true?

- (A) The producer will be able to add an item to the buffer, but the consumer can never consume it.
(B) The consumer will remove no more than one item from the buffer.
(C) Deadlock occurs if the consumer succeeds in acquiring semaphore s when the buffer is empty.
(D) The starting value for the semaphore n must be 1 and not 0 for deadlock free operation.

12. The following two function $P1$ and $P2$ that share a variable B with an initial value of 2 execute concurrently. [2015]

```

P1 ( ) {
    C = B - 1;
    B = 2 * C;
}

P2 ( ) {
    D = 2 * B;
    B = D - 1;
}

```

The number of distinct values that B can possibly take after the execution is _____

13. Two processes X and Y need to access a critical section. Consider the following synchronization construct used by both the processes [2015]

Process X	Process Y
<pre> /* other code for process X */ while (true) { varP = true; while(varQ == true) { /* Critical Section */ varP = false; } } /* other code for process X */ </pre>	<pre> /* other code for process Y */ while (true) { varQ = true; while (varP == true) { /* Critical Section */ varQ = false; } } /* other code for process Y */ </pre>

Here, $varP$ and $varQ$ are shared variables and both are initialized to false. Which one of the following statements is true?

- (A) The proposed solution prevents deadlock but fails to guarantee mutual exclusion.
 (B) The proposed solution guarantees mutual exclusion but fails to prevent deadlock.
 (C) The proposed solution guarantees mutual exclusion and prevents deadlock.
 (D) The proposed solution fails to prevent deadlock and fails to guarantee mutual exclusion.
14. Consider the following proposed solution for the critical section problem. There are n process: $P_0 \dots P_{n-1}$. In the code, function $pmax$ returns an integer not smaller than any of its arguments. For all i , $t[i]$ is initialized to zero. [2016]
- ```

do {
 c[i] = 1; t[i] = pmax (t[i], ...,
 t[n-1]) + 1; c[i] = 0;
 for every $j \neq i$ in $(0, \dots, n-1)$ {
 while (c[j]);
 while (t[j] != 0 && t[j] <= t[i]);
 }
 Critical Section;
 t[i] = 0;
 Remainder Section;
} while (true);

```

Which one of the following is TRUE about the above solution?

- (A) At most one process can be in the critical section at any time.  
 (B) The bounded wait condition is satisfied.  
 (C) The progress condition is satisfied.  
 (D) It cannot cause a deadlock.
15. Consider the following two - process synchronization Solution.

| Process 0                     | Process 1                     |
|-------------------------------|-------------------------------|
| Entry: loop while (turn = 1); | Entry: loop while (turn = 0); |
| (Critical section)            | (Critical section)            |
| Exit: turn = 1;               | Exit: turn = 0;               |

The shared variable  $turn$  is initialized to zero. Which one of the following is TRUE? [2016]

- (A) This is a correct two - process synchronization Solution.  
 (B) This Solution violates mutual exclusion requirement.  
 (C) This Solution violates progress requirement.  
 (D) This Solution violates bounded wait requirement.
16. Consider a non-negative counting semaphore  $S$ . The operation  $P(S)$  decrements  $S$ , and  $V(S)$  increments  $S$ . During an execution, 20  $P(S)$  operations and 12  $V(S)$  operations are issued in some order. The largest initial value of  $S$  for which at least one  $P(S)$  operation will remain blocked is \_\_\_\_\_. [2016]
17. A multithreaded program  $P$  executes with  $x$  number of threads and uses  $y$  number of locks for ensuring mutual exclusion while operating on shared-memory locations. All locks in the program are *non-reentrant*, i.e., if a thread holds a lock  $l$ , then it cannot re-acquire lock  $l$  without releasing it. If a thread is unable to acquire a lock, it blocks until the lock becomes available. The *minimum* value of  $x$  and the *minimum* value of  $y$  together for which execution of  $P$  can result in a deadlock are: [2017]
- (A)  $x = 1, y = 2$  (B)  $x = 2, y = 1$   
 (C)  $x = 2, y = 2$  (D)  $x = 1, y = 1$

18. Consider the following solution to the producer-consumer synchronization problem. The shared buffer size is  $N$ . Three semaphores  $empty$ ,  $full$  and  $mutex$  are defined with respective initial values of 0,  $N$  and 1. Semaphore  $empty$  denotes the number of available slots in the buffer, for the consumer to read from. Semaphore  $full$  denotes the number of available slots in the buffer, for the producer to write to. The placeholder variables, denoted by  $P$ ,  $Q$ ,  $R$ , and  $S$ , in the code below can be assigned either  $empty$  or  $full$ . The valid semaphore operations are:  $wait()$  and  $signal()$ .

| Producer                                                                                                                     | Consumer                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <pre>do {     wait(P);     wait (mutex);     //Add item to     buffer     signal (mutex);     signal (Q); } while (1);</pre> | <pre>do {     wait(R);     wait (mutex);     //Consume item     from buffer     signal (mutex);     signal (S); } while (1);</pre> |

Which one of the following assignments to  $P$ ,  $Q$ ,  $R$  and  $S$  will yield the correct solution? [2018]

- (A)  $P$ : full,  $Q$ : full,  $R$ : empty,  $S$ : empty  
 (B)  $P$ : empty,  $Q$ : empty,  $R$ : full,  $S$ : full  
 (C)  $P$ : full,  $Q$ : empty,  $R$ : empty,  $S$ : full  
 (D)  $P$ : empty,  $Q$ : full,  $R$ : full,  $S$ : empty

### ANSWER KEYS

#### Practice Problems 1

1. C    2. A    3. C    4. D    5. C    6. C    7. C    8. D    9. D    10. B  
 11. D    12. B    13. B    14. B    15. C    16. A    17. D

#### Practice Problems 2

1. B    2. A    3. D    4. B    5. D    6. C    7. B    8. B    9. B    10. A  
 11. B    12. C    13. D    14. D    15. C

#### Previous Years' Questions

1. D    2. A    3. C    4. A    5. A    6. A    7. A    8. B    9. D    10. C  
 11. C    12. 3    13. A    14. A    15. C    16. 7    17. D    18. C

# Chapter 3

## Deadlock and CPU Scheduling

### LEARNING OBJECTIVES

- Deadlock
- System model
- Bridge crossing
- Resources
- Resource allocation graph
- Methods of handling deadlocks
- Deadlock prevention
- Resource allocation denial (OR) banker's algorithm
- Deadlock detection
- Dining philosophers problem
- Scheduling algorithms
- Scheduling policies
- Round Robin scheduling
- Shortest remaining time
- Highest response ratio next
- Multilevel feedback queue scheduling

### DEADLOCK

It is a situation where a process or set of processes is blocked, waiting for some resource that is held by other waiting processes.

### System Model

Let the resource types be  $R_1, R_2 \dots R_m$  (like CPU cycles, memory space, input/output (I/O) devices, etc.). Each resource type  $R_i$  has  $W_i$  instances; each process utilizes a resource as follows:

**Request** A process, needing a resource, will request the operating system (OS) for assignment of the needed resource. Then the process waits, till operating system assigns it an instance of the requested resource.

**Assignment** The OS will assign to the requesting process an instance of the requested resource, whenever, it is available. Then, the process comes out of its waiting state.

**Use** The process will use the assigned resource. In case, the resource is non-sharable, the process will have exclusive access to it.

**Release** After the process finished with the use assigned resource, it will return the resource to the system pool. The released resource can now be assigned to another waiting process.

**Example:**

### Bridge crossing



Traffic is allowed only in one direction. Each section of a bridge can be viewed as a resource.

If a deadlock occurs, it can be resolved if one car backs up (pre-empt resources and rollback). Several cars may have to be backed up if a deadlock occurs.

Problem of starvation (infinite wait) is possible.

### Resources

Types of resources:

- Reusable resources
- Consumable resources

**Reusable resources** These resources can be safely used by only one process at a time, and are not depleted by that use.

**Examples:** Processors, I/O channels, main and secondary memory, devices and files, etc.

Consider two processes  $P$  and  $Q$  that compete for exclusive access to a disk file  $D$  and tape drive  $T$ . Let their implementation is as shown below:

Table 1 Process P

| Step  | Action           |
|-------|------------------|
| $P_0$ | Request ( $D$ )  |
| $P_1$ | Lock ( $D$ )     |
| $P_2$ | Request ( $T$ )  |
| $P_3$ | Lock ( $T$ )     |
| $P_4$ | Perform function |
| $P_5$ | Unlock ( $D$ )   |
| $P_6$ | Unlock ( $T$ )   |

Table 2 Process Q

| Step  | Action           |
|-------|------------------|
| $Q_0$ | Request ( $T$ )  |
| $Q_1$ | Lock ( $T$ )     |
| $Q_2$ | Request ( $D$ )  |
| $Q_3$ | Lock ( $D$ )     |
| $Q_4$ | Perform Function |
| $Q_5$ | Unlock ( $T$ )   |
| $Q_6$ | Unlock ( $D$ )   |

$P$  and  $Q$  are executing on a single processor in interleaved fashion. Then deadlock occurs if each process holds one resource and requests the other.

For example, deadlock occurs if the multiprogramming system interleaves the execution of the two processes as follows:

$$P_0, P_1, Q_0, Q_1, P_2, Q_2$$

One strategy to deal with this type of deadlocks is to impose system design constraints concerning the order in which resources can be requested.

**Consumable resources** A consumable resource is one that can be created and destroyed. There is no limit on the number of consumable resources of a particular type.

**Examples:** Interrupts, signals, messages, etc.

Consider the following pair of processes, in which each process attempts to receive a message from the other process and then send a message to the other process:

| $P_1$                   | $P_2$                   |
|-------------------------|-------------------------|
| .....                   | .....                   |
| receive ( $P_2$ );      | receive ( $P_1$ );      |
| .....                   | .....                   |
| send ( $P_2$ , $M_1$ ); | send ( $P_1$ , $M_2$ ); |

Deadlock occurs in above case, if the receive is blocking. There is no single effective strategy that can deal with all types of deadlocks.

## Deadlock Characteristics

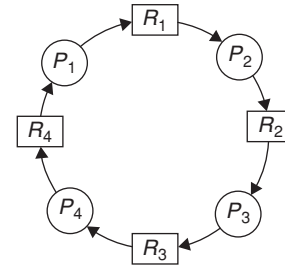
Deadlock is an undesirable state of the system. The following are the four conditions that must hold simultaneously for a deadlock to occur:

**Mutual exclusion** A resource can be used by only one process at a time. If another process requests for that resource then the requesting process must be delayed until the resource has been released.

**Hold-and-wait** Some processes must be holding some resources in a non-sharable mode and at the same time must be waiting to acquire some more resources, which are currently held by other processes in a non-sharable mode.

**No pre-emption** Resources granted to a process can be released back to the system only as a result of the voluntary action of that process, after the process has completed its task.

**Circular wait** Deadlocked processes are involved in a circular chain such that each process holds one or more resources being requested by the next process in the chain.



## Resource Allocation Graph

A deadlock is described in terms of a directed graph called a system Resource Allocation Graph (RAG). It consists of two sets:

1. The set of vertices,  $V$
2. The set of edges,  $E$

The set of vertices is again divided into two categories.

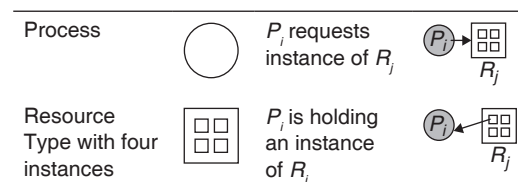
The set of all active processes in the system is  $P = \{P_1, P_2, \dots, P_n\}$  and the set of all different type of resources i.e.,  $R = \{R_1, R_2, \dots, R_m\}$

There are two types of edges in the RAG:

1. A directed edge from the process  $P_i$  to resources type  $R_j$  and is denoted by  $P_i \rightarrow R_j$ . It signifies that the  $i$ th process is requesting one unit of the resource type  $j$ . This edge is request edge.
2. A directed edge from the resource  $R_i$  to process  $P_j$  denoted by  $R_i \rightarrow P_j$ . It signifies that one unit of  $i$ th resource is held by the process  $j$ . This edge is also called as an *allocation edge/assignment edge*.

## Notations Used in RAGs

We denote a process by a circle and each resource by a rectangle.



However, if we have more number of instances of a resource type, then it is denoted by more dots.

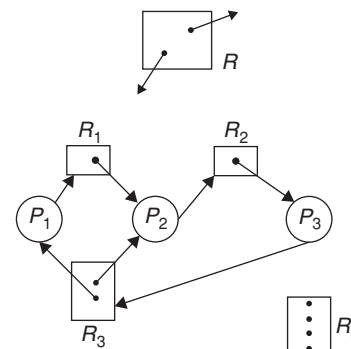
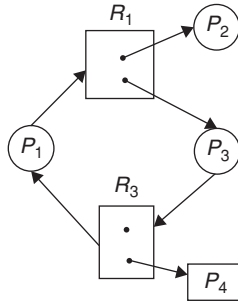


Figure 1 RAG with deadlock.

**Notes:**

1. If a cycle exists in the RAG, there may or may not be a deadlock.
2. Acyclic RAG implies no deadlock.
3. No deadlock implies acyclic RAG. This means that cycles can be there even if there is no deadlock.



**Figure 2** RAG with no deadlock but contains cycle.

**Notes:**

1. If graph contains no cycles, then no deadlock.
2. If graph contains a cycle

If there is only one instance per resource type, then deadlock occurs.

If there are several instances per resource type, deadlock may occur.

## METHODS OF HANDLING DEADLOCKS

There are three approaches to deal with deadlocks. They are

1. Deadlock prevention
2. Deadlock avoidance
3. Deadlock detection

### Deadlock Prevention

1. The strategy of deadlock prevention is to design a system in such a way that the possibility of deadlock is excluded.
2. Two classes of deadlock prevention are
  - Indirect method
  - Direct method

**Indirect method** Prevent the occurrence of one of three necessary conditions of deadlock i.e., mutual exclusion, No pre-emption and hold and wait.

**Direct method** Prevent the occurrence of circular wait.

### Prevention Techniques

**Mutual exclusion** This is supported by the OS.

**Hold and wait**

1. This condition can be prevented by requiring that a process request all of its required resources at one time and blocking the process until all requests can be granted simultaneously.
2. But this prevention does not yield good results because

- Long waiting time required
- Not efficient use of allocated resources
- A process may not know all the required resources in advance.

### Advantages

1. Works well for processes that perform a single burst of activity.
2. No pre-emption necessary.

**No pre-emption** Prevention strategies for 'no pre-emption' are

1. If a process that is holding some resources, requests another resource that cannot be immediately allocated to it, then all resources currently being held are released and if necessary request them again together with the additional resources.
2. If a process requests a resource that is currently held by another process, the OS may pre-empt the second process and require it to release its resources. This technique works only when two processes do not have same priority.

### Advantages

Convenient when applied to resources whose state can be saved and restored easily.

### Disadvantage

Pre-empts more often than necessary.

**Circular wait** One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resource in an increasing order of enumeration, i.e., if a process has been allocated resources of type  $R$ , then it may subsequently request only those resources of types following  $R$  in the ordering.

### Advantages

1. Feasible to enforce via compile time checks.
2. No run-time computation required.

### Disadvantages

1. Disallows incremental resources requests.

**Note:** The deadlock prevention strategies are conservative and undercommits resources.

### Deadlock Avoidance

1. This approach allows the three necessary conditions of deadlock but makes judicious choices to assure that deadlock point is never reached.
2. Deadlock avoidance allows more concurrency than prevention.
3. A decision is made dynamically whether the current resource allocation request will, if granted, potentially lead to a deadlock.

4. It requires the knowledge of future process requests.
5. Two techniques to avoid deadlock:
  - Process Initiation Denial
  - Resource Allocation Denial

**Process initiation denial** In this technique, do not start a process if its demands might lead to deadlock.

Consider a system of 'n' processes and 'm' different types of resources. Let us define the following vectors and matrices:

Resources  $R = (R_1, R_2, \dots, R_m)$

$R_1$ : amount of type 1 resources

$R_2$ : amount of type 2 resources

Available  $= V = (V_1, V_2, \dots, V_m)$

'V' specifies total amount of each resource not allocated to any process.

$$\text{Claim } C = \begin{matrix} P_1 \\ P_2 \\ \vdots \\ P_n \end{matrix} \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nm} \end{bmatrix}$$

$C_{ij}$  = requirement of process  $i$  for resources  $j$

$$\text{Allocation } A = \begin{matrix} P_1 \\ P_2 \\ \vdots \\ P_n \end{matrix} \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix}$$

$A_{ij}$  = Current allocation to process  $i$  of resource  $j$ .

The following relationships must hold:

1. All resources are either available or allocated, that is,

$$R_j = V_j + \sum_{i=1}^n A_{ij}, \forall j$$

2. No process can claim more than the total amount of resources in the system, that is,

$$C_{ij} \leq R_j, \forall i, j$$

3. No process is allocated more resources of any type than the process originally claimed to need, that is,

$$A_{ij} \leq C_{ij}, \forall i, j$$

With these properties satisfied, we can define a deadlock avoidance policy that refuses to start a new process if its resource requirements might lead to deadlock. Start a new process  $P_{n+1}$  only if

$$R_j \geq C_{(n+1)j} + \sum_{i=1}^n C_{ij}, \forall j$$

that is, a process is only started if the maximum claim of all current processes plus those of the new process can be met.

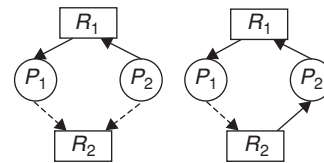
### Resource allocation denial (OR) banker's algorithm

Consider a system with a fixed number of processes and a fixed number of resources. At any time, a process may have zero or more resources allocated to it.

**State:** The state of a system reflects the current allocation of resources to processes.

### Safe state

1. When a process requests an available resource, the system must decide if immediate allocation leaves the system in a safe state.
2. A state is safe if the system can allocate resources to each process in some order and still avoid deadlock.
3. More formally, a system is in safe state if there exists a safe sequence of all processes.
4. A sequence of processes  $\langle P_1, P_2, \dots, P_n \rangle$  is a safe sequence for the current allocation state, if for each  $P_i$ , the resource requests that  $P_i$  can still make can be satisfied by the currently available resources plus the resources held by all the  $P_j$  with  $j < i$ .
5. When  $P_j$  is finished,  $P_i$  can obtain the needed resources, completed its designated task, return its allocated resources and terminates.
6. When  $P_i$  terminates,  $P_{i+1}$  can obtain its needed resources and so on.
7. If a system is in safe state, no deadlock occurs.
8. If a system is in unsafe state deadlock may occur.
9. Avoidance ensures that a system will never enter an unsafe state.



10. The dotted line in the above graph represents a claim edge, i.e., a process may request that resource sometime in the future.
11. A request can only be granted if it does not result in the formation of a cycle in the graph.
12. If  $P_2$  request  $R_2$ , we cannot allocate it, since this would create a cycle.
13. A cycle indicates the system is in *unsafe state*.

**Example 1:** Consider the following state of a system consisting of three processes and two resources:

$$R = (R_1, R_2) = (5, 3)$$

$$V = (V_1, V_2) = (2, 1)$$

$$C = \begin{matrix} & R_1 & R_2 \\ \begin{matrix} P_1 \\ P_2 \\ P_3 \end{matrix} & \begin{bmatrix} 4 & 3 \\ 2 & 1 \\ 3 & 3 \end{bmatrix} \end{matrix}$$

$$A = \begin{matrix} & R_1 & R_2 \\ \begin{matrix} P_1 \\ P_2 \\ P_3 \end{matrix} & \begin{bmatrix} 2 & 1 \\ 1 & 1 \\ 0 & 0 \end{bmatrix} \end{matrix}$$

Is this is a safe state?

**Solution:**

To check whether the state is safe or not, identify whether any one of the three process can run to completion with the resources available, that is,  $C_{ij} - A_{ij} \leq V_j, \forall j$

$$C - A = \begin{matrix} & R_1 & R_2 \\ \begin{matrix} P_1 \\ P_2 \\ P_3 \end{matrix} & \begin{pmatrix} 2 & 2 \\ 1 & 0 \\ 3 & 3 \end{pmatrix} \end{matrix}$$

We can identify that

$[1 \ 0] < [2 \ 1]$ .

$\therefore P_2$  can execute first.

After  $P_2$  execution it will release all its resources then

$V = (2 \ 1) + (1 \ 1) = (3 \ 2)$ .

Now  $P_1$  can execute as  $(2 \ 2) < (3 \ 2)$ .

After that  $P_1$  can release its resources then

$V = (3 \ 2) + (2 \ 1) = (5 \ 3)$

Now  $P_3$  can execute and release the resources after completion.

Hence, the safe sequence is  $\langle P_2, P_1, P_3 \rangle$ .

**Example 2:** Now suppose for the above system the allocation matrix

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 1 \\ 2 & 1 \end{pmatrix} \text{ and } V = (0 \ 0)$$

Then no process can run to completion as no  $C_{ij} - A_{ij} \leq V_j, \forall j$ .  
Hence, the system is in unsafe state.

## Detection Algorithm for Several Instances of a Resource Type

### Safety algorithm

To find out whether or not a system is in a safe state.

**Step I:** Let 'work' and 'finish' be the two vectors of length  $m$  and  $n$ .

Initialize: Work = Available and Finish  $[i] = \text{false}$ ;

**Step II:** Find  $i$  such that both:

(a) Finish  $[i] = \text{false}$ ;

(b) Need  $\leq$  Work (Need = claim – Allocation)

If no such  $i$  exists, go to step 4.

**Step III:** Work = Work + Allocation

Finish  $[i] = \text{true}$

Go to step 2

**Step IV:** If Finish  $[i] = \text{true}$  for all  $i$ , then the system is in safe state else it is in unsafe state. This algorithm takes  $O(m \rightarrow n^2)$  operations to decide whether a state is safe.

### Resource–Request algorithm

Let Request $_i$  be the request vector for process,  $P_i$ . If Request $_i[j] = k$ , the process  $P_i$  wants  $k$  instances of resource

type  $R_j$ . When a request for resources is made by process,  $P_i$ , the following actions are taken.

**Step I:** If Request $_i \leq$  Need $_i$ , go to step 2. Else raise an exception (error) as the process has exceeded its maximum claim.

**Step II:** If Request $_i \leq$  Available, go to step 3. Else  $P_i$  must wait, since the resources are not available.

**Step III:** Have the system pretend to have allocated the requested resources to process,  $P_i$ , by modifying the state as follows.

Available = Available – Request $_i$

Allocation = Allocation + Request $_i$

Need $_i$  = Need $_i$  – Request $_i$

If the resulting resource–allocation is safe, then the transaction is completed and process,  $P_i$  is allocated its resources. However, if the new state is unsafe then  $P_i$  must wait for Request $_i$  and the old resource allocation state is restored.

### Advantages of deadlock avoidance technique

1. Not necessary to pre-empt and rollback processes.
2. Less restrictive than deadlock prevention.

### Disadvantages

1. Future resource requirement must be known in advance.
2. Processes can be blocked for long periods.
3. Exists fixed number of resources for allocation.

## Deadlock Detection

1. This technique does not limit resource access or restrict process action.
2. Requested resources are granted to processes whenever possible.

Deadlock detection is used by employing an algorithm that tracks the circular waiting and killing one or more processes so that the deadlock is removed.

The system state is examined periodically to determine if a set of processes is deadlocked.

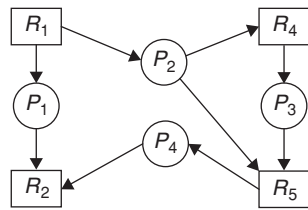
A deadlock is resolved by aborting and restarting a process, relinquishing all the resources that the process held.

**For single instance of each resource type** If in the RAG, every resource has only one instance (or single instance) then we define a deadlock detection algorithm that uses a variant of the RAG and is called a wait-for-graph.

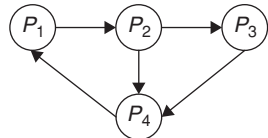
How can we get this graph from RAG? : We can get this by removing the nodes of type resource and collapsing the appropriate edges. Wait-for-graph has a cycle, then there is deadlock in the system.

To detect deadlocks, the system needs to maintain the wait-for-graph and to periodically invoke an algorithm. The complexity of this algorithm is  $O(n^2)$  where  $n$  is the number of vertices in the graph.

Consider the RAG:



We draw the wait-for-graph by removing all nodes that represents resources and collapsing their edges.



Wait-for-graph

The system is in deadlock state.

Cycle  $P_1, P_2, P_4, P_1$

Cycle  $P_1, P_2, P_3, P_4, P_1$

### Deadlock Detection Algorithm for Several Instances of Resource Type

Consider the Allocation matrix,  $A$ , Request matrix  $Q$  ( $Q_{ij}$  represents the amount of resource of type  $j$  requested by process  $i$ ), Resource vector  $R$  and available vector  $V$ .

The algorithm proceeds by marking processes that are not deadlocked. Initially all processes are unmarked.

1. Mark each process that has a row in the Allocation matrix of all zeros.
2. Initialize a temporary vector  $W$  to equal the Available vector.
3. Find an index  $i$  such that process  $i$  is currently unmarked and the  $i$ th row of  $Q$  is less than or equal to  $W$ , that is,  $Q_{ik} \leq W_k$ , for  $1 \leq k \leq M$ . If no such row is found, terminate the algorithm.
4. If such a row is found, mark process  $i$  and add the corresponding row of the allocation matrix to  $W$ . That is, set  $W_k = W_k + A_{ik}$  for  $1 \leq k \leq M$ . Return to step 3.

A deadlock exists if and only if there are unmarked processes at the end of the algorithm. Each unmarked process is deadlocked.

**Example 3:** Let the

Request matrix  $Q =$

|       | $R_1$ | $R_2$ | $R_3$ |
|-------|-------|-------|-------|
| $P_1$ | 1     | 1     | 1     |
| $P_2$ | 1     | 0     | 0     |
| $P_3$ | 1     | 1     | 1     |
| $P_4$ | 1     | 1     | 1     |

Allocation matrix  $A =$

|       | $R_1$ | $R_2$ | $R_3$ |
|-------|-------|-------|-------|
| $P_1$ | 1     | 1     | 0     |
| $P_2$ | 1     | 0     | 0     |
| $P_3$ | 0     | 1     | 1     |
| $P_4$ | 0     | 1     | 1     |

Resource vector  $R = (3 \ 3 \ 3)$

Available Vector  $V = (1 \ 0 \ 1)$

Is deadlock existing in this system?

**Solution:**

$W = (1 \ 0 \ 1)$

The request of  $P_2$  is less than  $W$ . So  $W = W + (1 \ 0 \ 0) = (2 \ 0 \ 1)$ . So mark  $P_2$ . No other unmarked process has a row  $Q$  that is less than or equal to  $W$ .

Terminate the algorithm.

$\therefore P_1, P_3, P_4$  are in deadlock.

### Advantages

1. Never delays process initiation
2. Facilitates online handling

### Disadvantages

1. Inherent pre-emption losses

### Deadlock Recovery

The possible deadlock recovery strategies are as follows:

1. Abort all deadlocked processes.
2. Back up each deadlocked process to some previously defined checkpoint and restart all processes.
3. Successively abort deadlocked processes until deadlock no longer exists.
4. Successively pre-empt resources until deadlock no longer exists.

### DINING PHILOSOPHERS PROBLEM

Consider the following solution for dining philosophers problem using semaphores:

```

Semaphore fork[5] = {1};
int i;
void philosopher(int i)
{
 while (true)
 {
 think();
 wait(fork[i]);
 wait(fork[(i + 1) mod 5]);
 eat();
 signal(fork[(i + 1) mod 5]);
 signal(fork[i]);
 }
}

void main()
{
 Begin (Philosopher(0), Philosopher(1),
 Philosopher(2), Philosopher(3),
 Philosopher(4));
}

```

Here, each philosopher picks up first the fork on the left and then the fork on the right. After the philosopher is finished eating, the two forks are placed on the table. But this

solution leads to deadlock, if all of the philosophers are hungry at the same time, they all sit down, they all pick up the fork on their left and they all reach out for the other fork, which is not there.

A refined solution to dining philosophers problem which is deadlock free is shown below:

```
Semaphore fork[5] = {1};
Semaphore room = {4};

int i;
void philosopher(int i)
{
 while(true)
 {
 think();
 wait(room);
 wait(fork[i]);
 wait(fork[(i + 1) mod 5]);
 eat();
 signal(fork[(i + 1) mod 5]);
 signal(fork[i]);
 signal(room);
 }
}

void main ()
{
 Begin
 {
 (Philosopher (0), Philosopher (1), Philosopher
 (2), Philosopher (3), Philosopher (4));
 }
}
```

This solution is free from deadlock and starvation.

## CPU SCHEDULING

1. The objective of multi programmed OS is to maximize CPU utilization by having some process running at all times.
2. The objective of time shared OS is to switch the CPU among processes so frequently that the users can interact with each program while it is executing.
3. When there are more than one process ready to execute with the processor, a selection decision needs to be made to pick a process for execution from among the ready processes. This activity is called *process scheduling*.

**Scheduling queue:** It maintains information of all ready processes for CPU devices. It is maintained as a linked list.

## Types of Scheduling Queue

1. **Job queue:** It consists of all processes in the system.
2. **Ready queue:** It consists of all processes that are residing in the main memory and are ready but waiting to execute on CPU.
3. **Device queue:** It consists of processes waiting for a particular I/O device. Each device has its own queue.

## Process CPU-I/O Burst Cycle

The execution of process consists of CPU burst and I/O burst. The execution of process starts with CPU burst and I/O burst, which are executed alternatively.

The alternating sequence of CPU and I/O burst are shown below:

```
Read a }
Inc a } CPU Burst
Read x }
```

I/O waiting } I/O Burst

```
Dec x }
Store x } CPU Burst
```

I/O waiting } I/O Burst

·  
·  
·  
·

There should be proper balance between CPU bound process and I/O bound process in a schedule.

## Scheduler

A process migrates between various scheduling queues throughout its lifetime. The process of selecting processes from the queues is carried out by scheduler.

## TYPES OF PROCESSOR SCHEDULING

There are three types of processor scheduling:

1. Long-term scheduling
2. Medium-term scheduling
3. Short-term scheduling

The following figure relates the scheduling functions to the process state transition diagram:

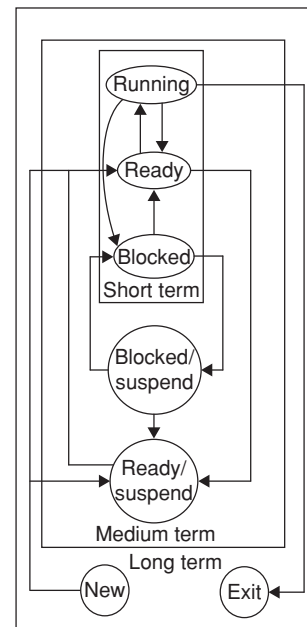


Figure 3 Levels of Scheduling

## Long-term Scheduling

1. This is performed when a new process is created. This is also called *job scheduling*.
2. This is a decision whether to add a new process to the set of process that is currently active.
3. It controls the degree of multiprogramming.
4. The long-term scheduler creates processes from the queue when it can.
5. This involves two decisions:
  - The Scheduler must decide when the OS can take on one or more additional processes.
  - The scheduler must decide which job(s) to accept and turn into processes.

## Medium-term Scheduling

1. It is a part of swapping function.
2. This is a decision whether to add a process to those that are at least partially in main memory and therefore avail for execution.

## Short-term Scheduling

1. It is the decision regarding which ready process to be executed next.
2. This is also known as CPU scheduler.
3. This is invoked whenever an event occurs that may lead to the blocking of the current process or that may provide an opportunity to pre-empt a currently running process in favour of another.
4. Another term involved in short-term scheduling is *dispatcher* which is a module that gives control of the CPU to the process selected by short-term scheduler.

**Examples:** Clock interrupts, I/O interrupts, OS calls, etc.

## SCHEDULING ALGORITHMS

### Scheduling Criteria

The commonly used scheduling criteria can be categorized along two dimensions:

1. User oriented versus system oriented.
2. Performance related versus others.

### User-oriented, performance-related criteria

1. **Turnaround time:** It is the time taken to execute a process. It is calculated as the interval from the time of submission of a process to the time of completion.   
 Turnaround time = waiting time + execution time + time spent in I/O + time spent to get into memory
2. **Response time:** Amount of time it takes from when a request was submitted until the first response is produced. It should be minimum.
3. **Deadlines:** When process completion deadlines can be specified, the scheduling discipline should subordinate other goals to that of maximizing the percentage of deadlines met.

**Waiting time:** It is the amount of time that a process spends in ready queue and doing I/O, and it should be minimum.

### User-oriented, other criteria

**Predictability:** A given job should run in about the same amount of time and at about the same cost regardless of the load on the system.

### System-oriented, performance-related criteria

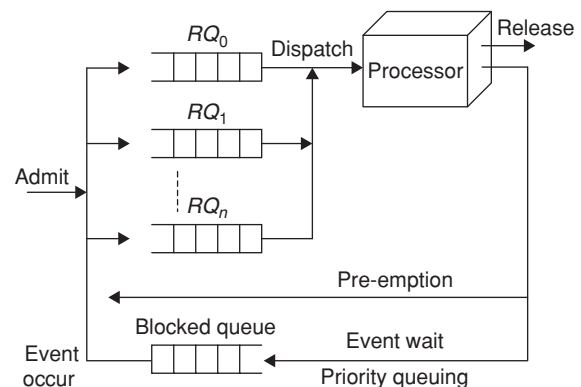
1. **Throughput:** The scheduling policy should attempt to maximize the number of processes completed per unit of time. This is a measure of how much work is being performed.
2. **Processor (CPU) utilization:** This is the percentage of time that the processor is busy. In real system, it should range from 40–90%.

### System-oriented, other criteria

1. **Fairness:** No process should suffer from starvation.
2. **Enforcing priorities:** Should favour higher priority processes and use Aging technique in order to increase the priority of processes is that wait in the system for long time.
3. **Balancing resources:** Should keep the resources of the system busy.

### Use of priorities

1. Each process is assigned a priority, and the scheduler will always choose a process of higher priority over one of lower priority.



Here  $RQ_0, RQ_1, \dots, RQ_n$  are ready queues with priority  $(RQ_i) > \text{priority}(RQ_j)$  for  $i < j$ . The scheduler starts with  $RQ_0$  processes, if it is empty choose a process from  $RQ_1$  and so on.

### Scheduling Policies

We will discuss the following scheduling algorithms:

- |                |             |
|----------------|-------------|
| 1. FCFS        | 4. SRN      |
| 2. Round Robin | 5. HRRN     |
| 3. SPN         | 6. Feedback |

Before discussing these algorithms, let us discuss some basic concepts:

### Selection function

1. Determines which process among ready process, is selected next for execution.
2. This may depend on priority, resource requirement, execution characteristics of process.
3. Execution characteristics of a process include  
 $w$  = waiting time  
 $e$  = execution time,  
 $S = w + e$

**Decision mode** It is of the following two types:

1. Non-pre-emptive
2. Pre-emptive
  - In non-pre-emptive scheduling, if once the CPU has been allocated to a process, the process can keep the CPU until it releases it, either by terminating or switching to waiting state.
  - In pre-emptive scheduling, CPU can be taken away from a process during execution.

### Comparison of non-pre-emptive and pre-emptive scheduling

|    | Non-pre-emptive Scheduling                                                                                                   | Pre-emptive Scheduling                                                                                                                                        |
|----|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | In non-pre-emptive scheduling, if once a process has been allocated CPU then the CPU cannot be taken away from that process. | In pre-emptive scheduling, the CPU can be taken away before the completion of the process.                                                                    |
| 2. | No preference is given when a higher priority job comes.                                                                     | It is useful when a higher priority job comes as here the CPU can be snatched from a lower priority process.                                                  |
| 3. | The treatment of all processes is fairer.                                                                                    | The treatment of all processes is not fairer as CPU snatching is done either due to constraints or due to higher priority, process request for its execution. |
| 4. | It is a cheaper scheduling method. First come first served is an example.                                                    | It is a costlier scheduling method. Round Robin is an example.                                                                                                |

**First come, first served scheduling (FCFS)** The process that requests the CPU first is allocated the CPU first. It is non-pre-emptive scheduling and average waiting time is quite long.

**Example 4:** Consider the following processes:

| Process | Cpu Burst Time (Millisecond) |
|---------|------------------------------|
| $P_1$   | 20                           |
| $P_2$   | 5                            |
| $P_3$   | 3                            |

Find the average waiting time.

### Solution:

Suppose they are in the order  $P_1, P_2, P_3$  at time 0. So, Gantt chart is

|       |       |       |
|-------|-------|-------|
| $P_1$ | $P_2$ | $P_3$ |
| 0     | 20    | 25 28 |

$$\text{Average waiting time} = \frac{(0 + 20 + 25)}{3} = 15 \text{ ms}$$

If they arrived in order  $P_3, P_2, P_1$  then, Gantt chart is

|       |       |       |
|-------|-------|-------|
| $P_3$ | $P_2$ | $P_1$ |
| 0     | 3 8   | 28    |

$$\text{Average waiting time} = \frac{0 + 3 + 8}{3} = \frac{11}{3} = 3.67 \text{ ms}$$

### Notes:

1. Throughput is not that much emphasized.
2. Response time may be high especially if there is a large variance in process execution times.
3. Minimum overhead required.
4. It penalizes short processes, also penalizes I/O bound processes.
5. There is no possibility of starvation.

### Advantages

1. Simple and brutally fair.
2. It is suitable for batch systems.

### Disadvantages

1. The average waiting time is not minimal.
2. Not suitable for time sharing systems like Unix.
3. **Convoy effect:** Short process behind long process results in lower CPU utilization.

### Round Robin scheduling

1. It is designed for time sharing system.
2. Similar to FCFS with pre-emption added.
3. Each process gets a small central CPU time (a time slice) usually 10 – 100 ms.
4. After time slice has elapsed and added to the end of the ready queue.
5. The scheduler picks the first process from the ready queue, sets a timer to interrupt after one time quantum and then dispatches the process. One of the following happens.
6. The process may have a CPU burst of less than 1 time quantum. (or)
7. CPU burst time of the currently executing process is longer than one time quantum. In this case, the timer will go off, cause an interrupt, a context switch is then

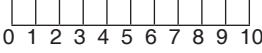
## 7.44 | Unit 7 • Operating System

executed and the process is put at the tail of the ready queue.

8. Average waiting time is quite long.

### Performance of Round Robin scheduling

Let us assume that we have only one process of 10 time units.

| Process time = 10                                                                 | Quantum | Switch |
|-----------------------------------------------------------------------------------|---------|--------|
|  | 12      | 0      |
|  | 6       | 1      |
|  | 1       | 9      |

**Example 5:** Consider the following processes, arrival times and CPU processing requirements with Round Robin scheduling algorithm.

| Process | CPU time | Arrival time |
|---------|----------|--------------|
| A       | 8        | 0            |
| B       | 1        | 1            |
| C       | 2        | 3            |
| D       | 1        | 4            |
| E       | 5        | 2            |

What will be the mean turnaround time if time quantum is 4 msec?

**Solution:**

Plotting the Gantt chart

|          |          |          |          |          |          |          |    |
|----------|----------|----------|----------|----------|----------|----------|----|
| <i>A</i> | <i>B</i> | <i>E</i> | <i>C</i> | <i>D</i> | <i>A</i> | <i>E</i> |    |
| 0        | 4        | 5        | 9        | 11       | 12       | 16       | 17 |

Turnaround time = Finish time – Arrival time

TAT of A = 16 – 0 = 16

B = 5 – 1 = 4

C = 11 – 3 = 8

D = 12 – 4 = 8

E = 17 – 2 = 15

$$\text{Mean Turnaround time} = \frac{16 + 4 + 8 + 8 + 15}{5} = 10.2 \text{ msec}$$

- If there are  $n$  processes in the ready queue and time quantum  $q$ , then each process gets  $\frac{1}{n}$  of the CPU time in chunks of at most  $q$  time units at once.
- No process waits more than  $(n - 1)q$  time units until the next time quantum.
- The performance of Round Robin depends on time slice. If it is larger it is same as FCFS. If  $q$  is very small overhead is too high as the number of context switches increases.

**Notes:**

- Throughput is low if quantum is too small.
- Provides good response time for short processes.

- Minimum overhead.
- All processes treated fairly.
- No starvation.

### Shortest process next (SPN)

- This is a non-pre-emptive policy in which the process with the shortest expected processing time is selected next.
- A short process will jump to the head of the queue past longer jobs.

**Example 6:** Consider the following process, such that all have arrived at time = 0

| Process | Burst time |
|---------|------------|
| $P_1$   | 5          |
| $P_2$   | 9          |
| $P_3$   | 6          |
| $P_4$   | 3          |

Find the average waiting time using SPN.

**Solution:** Gantt chart is

|       |       |       |       |    |
|-------|-------|-------|-------|----|
| $P_4$ | $P_1$ | $P_3$ | $P_2$ |    |
| 0     | 3     | 8     | 14    | 23 |

$$\text{Average waiting time} = \frac{0 + 3 + 8 + 14}{4} = 6.25 \text{ ms}$$

**Notes:**

- Difficulty with this policy is that we need to know or at least estimate the required processing time of each process.
- High throughput is possible.
- Provides good response time for short processes.
- High overhead.
- It penalizes long processes.
- There is a possibility of starvation.

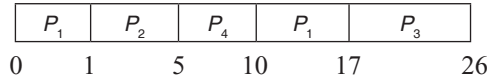
### Shortest remaining time (SRT)

- It is a pre-emptive version of SPN.
- Here the scheduler always chooses the process that has the shortest expected remaining processing time.
- When a new process joins the ready queue, it may in fact have a shorter remaining time than the currently running process.
- Accordingly, the scheduler may pre-empt the current process when a new process becomes ready.

**Example 7:** Consider the following process. Find the average waiting time for SRT.

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| $P_1$   | 0            | 8          |
| $P_2$   | 1            | 4          |
| $P_3$   | 2            | 9          |
| $P_4$   | 3            | 5          |

**Solution:** Gantt chart



So average waiting time

$$\frac{(10-1) + (1-1) + (17-2) + (5-3)}{4}$$

$$\frac{9+0+15+2}{4} = \frac{26}{4}$$

$$= 6.5 \text{ ms}$$

Here  $P_3$  starts at time 17 but the arrival time was at 2. So waiting time of  $P_3$  will be  $(17-2)$ .

**Notes:**

1. High throughput.
2. Provides good response time.
3. High overhead.
4. Penalizes long processes.
5. Starvation is possible.

### Highest Response Ratio Next (HRRN)

This algorithm works on the principle the executes the job first which has the highest response ratio. We define response ratio as the ratio between turnaround time and response time.

$$\text{Response Ratio} = \frac{(W+S)}{S}, \text{ where}$$

$W$  – Time spend waiting for the processor

$S$  – Service time

This response ratio is also named as normalized turnaround time.

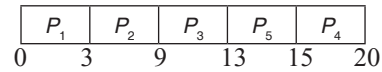
**Example 8:** Consider 5 processes with their arrival and service times:

| Process | Arrival time | Service time |
|---------|--------------|--------------|
| $P_1$   | 0            | 3            |
| $P_2$   | 2            | 6            |
| $P_3$   | 4            | 4            |
| $P_4$   | 6            | 5            |
| $P_5$   | 8            | 2            |

What is the average turnaround time using HRRN technique?

**Solution:**

Gantt chart



Average turnaround time

$$= \frac{3 + (6+1) + (4+5) + (5+9) + (2+5)}{5} = \frac{40}{5} = 8 \text{ ms}$$

**Notes:**

1. It is a non-pre-emptive scheduling algorithm.
2. High throughput.
3. Provides good response time.
4. High overhead.
5. Good balance of any type processes.
6. No starvation.

### Multilevel feedback queue scheduling

1. In multilevel queue scheduling algorithm, processes are permanently assigned to a queue on entry to the system. Processes cannot move between queues.
2. Processes can move between queues. If a process uses too much CPU time, it will be moved to a lower priority queue.
3. I/O bound and interactive processes are put into higher priority queue.
4. A process that waits too long in a lower priority queue may be moved to a higher priority queue. This form of aging prevents starvation.

In general, a multilevel feedback queue scheduler is defined by following parameters:

1. The number of queues.
2. The scheduling algorithm for each queue.
3. The method used to determine when to upgrade a process to a higher priority queue.
4. The method used to determine when to denote a process to a lower priority queue.
5. The method used to determine in which queue a process will enter when process needs service.

**Notes:**

1. Pre-emptive at time quantum.
2. Throughput is not that much emphasized.
3. Response time is not that much emphasized.
4. High overhead.
5. Favours I/O bound processes.
6. Starvation is possible.

## EXERCISES

## Practice Problem I

**Directions for questions 1 to 19:** Select the correct alternative from the given choices.

1. Consider the following processes; find the average waiting time using non-pre-emptive priority scheduling?

| Process | Arrival time (ms) | Burst time (ms) | Priority |
|---------|-------------------|-----------------|----------|
| $P_0$   | 0                 | 10              | 5        |
| $P_1$   | 1                 | 6               | 4        |
| $P_2$   | 3                 | 2               | 2        |
| $P_3$   | 5                 | 4               | 0        |

- (A) 2.36 ms (B) 0.31 ms  
(C) 7.75 ms (D) 13.25 ms
2. Consider a set of five processes whose arrival time, CPU times needed are given below.

| Process | CPU time (in msec) | Arrival time (in msec) |
|---------|--------------------|------------------------|
| $P_1$   | 10                 | 5                      |
| $P_2$   | 5                  | 2                      |
| $P_3$   | 3                  | 0                      |
| $P_4$   | 20                 | 4                      |
| $P_5$   | 2                  | 3                      |

If the CPU scheduling policy is SJF, find the average waiting time (with pre-emption).

- (A) 4.8 ms (B) 5.6 ms  
(C) 2.16 ms (D) 2.8 ms
3. Consider the following snapshot of a system:

|       | Allocation |   |   |   | Max |   |   |   | Available |   |   |   |
|-------|------------|---|---|---|-----|---|---|---|-----------|---|---|---|
|       | A          | B | C | D | A   | B | C | D | A         | B | C | D |
| $P_0$ | 0          | 0 | 1 | 2 | 0   | 0 | 1 | 2 | 1         | 5 | 2 | 0 |
| $P_1$ | 1          | 0 | 0 | 0 | 1   | 7 | 5 | 0 |           |   |   |   |
| $P_2$ | 1          | 3 | 5 | 4 | 2   | 3 | 5 | 6 |           |   |   |   |
| $P_3$ | 0          | 6 | 3 | 2 | 0   | 6 | 5 | 2 |           |   |   |   |
| $P_4$ | 0          | 0 | 1 | 4 | 0   | 6 | 5 | 6 |           |   |   |   |

Which of the following is true?

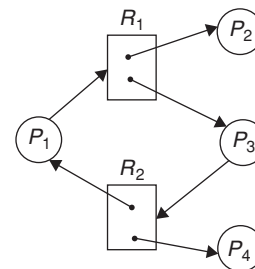
- (i) The system is in a safe state.  
(ii) If a request from process  $P$ , arrives for (0, 4, 2, 0). Then the request can be granted.
- (A) Only (i) (B) Only (ii)  
(C) Both (i) and (ii) (D) Neither (i) nor (ii)
4. In a Round Robin scheduling context switch time is 4 units, the average process running time before blocking is 6 units then CPU efficiency is
- (A) 0.2 (B) 0.4  
(C) 0.6 (D) 0.1
5. A short-term scheduler executes at least once every 20 msec. If it takes 2 msec to decide to execute a process

for 2 msec, what is the percentage of CPU time wasted?

- (A) 8% (B) 9%  
(C) 10% (D) 11%
6. Consider a system which has  $n$  resources of the same type. The  $n$  resources are shared among three processes  $A, B, C$ , which have high demands of 3, 5, 6, respectively. For what value of  $n$  will deadlock not occur?
- (A) 11 (B) 10  
(C) 9 (D) 15
7. A comparative study of scheduling algorithm was performed, the average arrival time in the queue is 5 msec and waiting time of the processes is 10 msec. What is the average queue length of the waiting processes?
- (A) 50 (B) 60  
(C) 70 (D) 80
8. A CPU scheduling algorithm determines an order for the execution of its scheduled processes. Given five processes to be scheduled on one processor, how many possible different schedules are there?
- (A) 50 (B) 100  
(C) 120 (D) 150
9. Consider the following set of jobs (processes) along with their Arrival Time (AT), start time (ST) and Finish Time (FT). Find weighted turnaround time.

| Job no. | AT   | ST   | FT   |
|---------|------|------|------|
| 1       | 10.0 | 10.0 | 10.3 |
| 2       | 10.2 | 10.3 | 10.8 |
| 3       | 10.4 | 10.8 | 10.9 |
| 4       | 10.5 | 10.9 | 11.3 |
| 5       | 10.8 | 11.3 | 11.4 |

- (A) 3.04 (B) 2.04  
(C) 4.04 (D) 0.56
10. Is the following resource allocation graph in a deadlock state?



- (A) Yes (B) No  
(C) Not predictable (D) Insufficient data
11. Starvation of longer jobs happens in one of the following scheduling algorithm?
- (A) Shortest run remaining time first  
(B) Round Robin  
(C) Highest response ratio next  
(D) First-come first-served

12. Suppose  $n$  processes,  $P_1 \dots P_n$  share  $n$  identical resource units, which can be reserved and released one at a time. The maximum resource requirement of process  $P_i$  is  $S_i$ , where  $S_i > 0$ . Which one of the following is a sufficient condition for ensuring that deadlock does not occur?

- (A)  $\sum_{i=1}^n S_i < m$  (B)  $\sum_{i=1}^n S_i < n$   
 (C)  $\sum_{i=1}^n S_i < (m+n)$  (D)  $\sum_{i=1}^n S_i < (m*n)$

13. A system with following processes and resources exists. Check the system for safe state and find the safe sequence of processes

|       | Allocation |   |   | Max |   |   | Available |   |   |
|-------|------------|---|---|-----|---|---|-----------|---|---|
|       | X          | Y | Z | X   | Y | Z | X         | Y | Z |
| $P_0$ | 0          | 1 | 0 | 7   | 5 | 3 | 3         | 3 | 2 |
| $P_1$ | 2          | 0 | 0 | 3   | 2 | 2 |           |   |   |
| $P_2$ | 3          | 0 | 2 | 9   | 0 | 2 |           |   |   |
| $P_3$ | 2          | 1 | 1 | 2   | 2 | 2 |           |   |   |
| $P_4$ | 0          | 0 | 2 | 4   | 3 | 3 |           |   |   |

- (A)  $\langle P_1, P_3, P_4, P_2, P_0 \rangle$   
 (B)  $\langle P_3, P_4, P_2, P_0, P_1 \rangle$   
 (C)  $\langle P_2, P_4, P_0, P_1, P_3 \rangle$   
 (D) The system is in unsafe state.
14. Does the below statements be executed concurrently?  
 $S_1: a = x + y$   
 $S_2: b = z + 1$   
 (A) Yes (B) No  
 (C) Not predictable (D) None of the above
15. Let  $A, B, C$  be three jobs. Their arrival time and execution time are shown below. By applying monoprogramming and multiprogramming (use Round Robin with time slice 1 unit) approaches, calculate the amount of reduction in turnaround time?

| Job | Arrival time | Execution time |
|-----|--------------|----------------|
| A   | 1            | 2              |
| B   | 2            | 6              |
| C   | 3            | 1              |

- (A) 3.33 (B) 4.33  
 (C) 5.33 (D) 2.33
16. Consider a system with three processes  $A, B, C$  with 15 tape drivers. Process A has 4 tape drives but requires 14 tape drives.

Process B has 5 tape drives but requires 9 tape drives.  
 Process C has 3 tape drives but requires 7 tape drives.  
 Among the following processes which will enter the deadlock state?

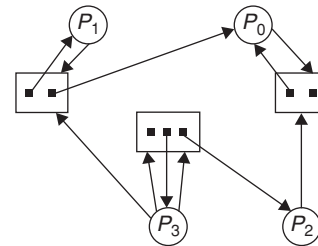
- (A) A, B only (B) A, B, C  
 (C) A, C only (D) B, C only
17. Assume that the following jobs are to be executed on a uniprocessor system:

| Job id | CPU burst time |
|--------|----------------|
| P      | 4              |
| Q      | 1              |
| R      | 8              |
| S      | 1              |
| T      | 2              |

The jobs are assumed to have arrived at 0, and in the order P, Q, R, S and T. Calculate the departure with time slice (completion time) for job P if scheduling is Round Robin with time slices of 1 unit (slice).

- (A) 4 (B) 10  
 (C) 11 (D) 12

**Common data for questions 18 and 19:** Consider the following Resource Allocation Graph:



18. The system is in a deadlock state. This remark is:  
 (A) True (B) False  
 (C) Impossible to determine (D) Unpredictable
19. Which one is a safe sequence?  
 (A)  $P_0, P_1, P_2, P_3$  (B)  $P_1, P_0, P_2, P_3$   
 (C)  $P_2, P_0, P_1, P_3$  (D) Both (A) and (C)

### Practice Problem 2

**Directions for questions 1 to 15:** Select the correct alternative from the given choices.

1. Let there be five processes ( $P_1$  to  $P_5$ ) and three resource types A, B, C.  
 Resource type A has 10 instances,  
 Resource type B has 5 instances,  
 Resource type C has 7 instances.

Suppose that at time  $T_0$ , the following snapshot of the system has been taken.

|       | Allocation |   |   | Max |   |   | Available |   |   |
|-------|------------|---|---|-----|---|---|-----------|---|---|
| $P_1$ | 0          | 1 | 0 | 7   | 5 | 3 | 3         | 3 | 2 |
| $P_2$ | 2          | 0 | 0 | 3   | 2 | 2 |           |   |   |
| $P_3$ | 3          | 0 | 2 | 9   | 0 | 2 |           |   |   |
| $P_4$ | 2          | 1 | 1 | 2   | 8 | 2 |           |   |   |
| $P_5$ | 0          | 0 | 2 | 4   | 3 | 3 |           |   |   |

Which of the following statement is true?

- (A) The system is in safe state.  
 (B) The system has process initiation denial problem.  
 (C) No process causes initiation denial  
 (D) Both (A) and (C)
2. Consider three CPU intensive processes, which require 20, 30 and 40 time units and arrive at times 0, 2 and 4, respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end.  
 (A) 0 (B) 1  
 (C) 2 (D) 3
3. Consider the set of processes  $P_1$  to  $P_5$  with the following CPU burst times. Find the average turnaround time using shortest remaining time first.

| Process | CPU burst time | Arrival time |
|---------|----------------|--------------|
| $P_1$   | 3              | 0            |
| $P_2$   | 6              | 2            |
| $P_3$   | 4              | 4            |
| $P_4$   | 5              | 6            |
| $P_5$   | 2              | 8            |

- (A) 1.3 ms (B) 3.5 ms  
 (C) 5.8 ms (D) 7.2 ms
4. All processes are arriving at time 0, find the average waiting time.

| Process | Burst time | Priority |
|---------|------------|----------|
| $P_1$   | 10         | 3        |
| $P_2$   | 1          | 1        |
| $P_3$   | 2          | 3        |
| $P_4$   | 1          | 4        |
| $P_5$   | 5          | 2        |

- (A) 8.2 ms (B) 4.1 ms  
 (C) 2.0 ms (D) 1.3 ms
5. Consider a set of three processes  $P_1$ ,  $P_2$  and  $P_3$  with their priorities and arrival times as given below.

| Process | Burst time | Priority   | Arrival time |
|---------|------------|------------|--------------|
| $P_1$   | 10         | 3          | 0            |
| $P_2$   | 5          | 2          | 1            |
| $P_3$   | 2          | 1(highest) | 2            |

Find the average waiting time.

- (A) 1 ms (B) 2 ms  
 (C) 3 ms (D) 4 ms
6. The portion of the process scheduler in an OS that dispatches processes is concerned with:  
 (A) assigning ready processes to the CPU  
 (B) activating suspended I/O bound processes  
 (C) temporarily suspending processes when the CPU load is too great.  
 (D) All the above
7. In a time-sharing OS, when the time slot given to a process is completed, the process goes from the running state to the

- (A) blocked state (B) ready state  
 (C) suspended state (D) terminated state

8. On a system with  $n$  CPUs, what is the maximum number of processes that can be in the ready state?  
 (A)  $n$  processes  
 (B) No process can be in ready state  
 (C) There is no limit to the number of processes in the ready state  
 (D) None of the above
9. Consider a set of  $n$  tasks with known runtimes  $r_1, r_2, \dots, r_n$  to be run on a uniprocessor machine. Which of the following processor scheduling algorithms will result in the maximum throughput?  
 (A) Round Robin  
 (B) SJF  
 (C) Highest response ratio next  
 (D) First-come first-served

10. Match the following:

|                   |                           |
|-------------------|---------------------------|
| A Critical region | I Hoare's monitor         |
| B Wait/signal     | II Mutual exclusion       |
| C Working set     | III Principle of locality |
| D Deadlock        | IV Circular wait          |

- (A) A – II, B – I, C – III, D – IV  
 (B) A – I, B – II, C – III, D – IV  
 (C) A – II, B – I, C – IV, D – III  
 (D) A – I, B – II, C – IV, D – III

11. Consider three processes  $A$ ,  $B$ ,  $C$  to be scheduled as per SRT algorithm.  $A$  is known to be scheduled first and when  $A$  has been running for 7 units of time,  $C$  has arrived.  $C$  has run for 1 unit of time when  $B$  has arrived and completed running in 2 units of time, what could be the minimum time of executions for  $A$  and  $C$ ?  
 (A) 11 and 4 (B) 11 and 3  
 (C) 12 and 3 (D) 12 and 4

12. Select the correct statements from below:

- (i) SRT and SPN can cause starvation for larger processes.  
 (ii) FCFS can potentially block small processes in favour of much larger processes.  
 (iii) Round Robin algorithm gives fair treatment to all the processes.  
 (iv) FCFS is a pre-emptive algorithm.  
 (v) The throughput for Round Robin is high even for small time slices.  
 (A) (i), (ii), (iii) (B) (i), (ii), (iv)  
 (C) (iii), (iv), (v) (D) (i), (ii), (iii), (v)

13. Three processes share four resource units that can be reserved and released only one at a time. Each process needs a maximum of two units. Then  
 (A) there is a possibility of deadlock  
 (B) no deadlock will occur

- (C) there will be a circular wait  
(D) nothing can be predicted about dead lock.
14.  $N$  processes share  $M$  resource units that can be reserved and released only one at a time. The maximum need of each process does not exceed  $M$  and the sum of all maximum needs is less than  $M + N$ , then  
(A) there is a possibility of deadlock  
(B) there will be no deadlock
- (C) circular wait exists  
(D) nothing can be predicted about dead lock.
15. Which of the following scheduling algorithms could result in starvation?  
(A) First-come, first-served  
(B) Shortest job first  
(C) Round Robin  
(D) Highest response ratio next

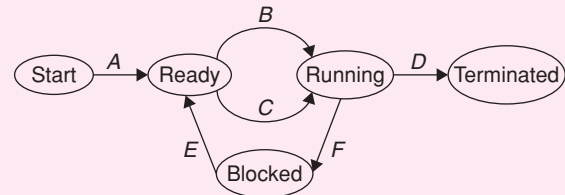
### PREVIOUS YEARS' QUESTIONS

1. Consider three processes (process id 0, 1, 2, respectively) with compute bursts 2, 4 and 8 time units. All processes arrive at time zero. Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. The average turnaround time is: [2006]  
(A) 13 units (B) 14 units  
(C) 15 units (D) 16 units
2. Consider three processes, all arriving at zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle? [2006]  
(A) 0% (B) 10.6%  
(C) 30.0% (D) 89.4%
3. A single processor system has three resource types  $X$ ,  $Y$  and  $Z$ , which are shared by three processes. There are five units of each resource type. Consider the following scenario, where the column *alloc* denotes the number of units of each resource type allocated to each process, and the column *request* denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish *last*?

|       | alloc |     |     | request |     |     |
|-------|-------|-----|-----|---------|-----|-----|
|       | $X$   | $Y$ | $Z$ | $X$     | $Y$ | $Z$ |
| $P_0$ | 1     | 2   | 1   | 1       | 0   | 3   |
| $P_1$ | 2     | 0   | 1   | 0       | 1   | 2   |
| $P_2$ | 2     | 2   | 1   | 1       | 2   | 0   |

- (A)  $P_0$   
(B)  $P_1$   
(C)  $P_2$   
(D) None of the above, since the system is in a deadlock.
- [2007]

4. Which of the following is *not* true of deadlock prevention and deadlock avoidance schemes? [2008]  
(A) In deadlock prevention, the request for resources is always granted if the resulting state is safe  
(B) In deadlock avoidance, the request for resources is always granted if the result state is safe  
(C) Deadlock avoidance is less restrictive than deadlock prevention  
(D) Deadlock avoidance requires knowledge of resource requirements *a priori*
5. In the following process state transition diagram for a uniprocessor system, assume that there are always some processes in the ready state: [2009]



Now consider the following statements:

- I. If a process makes a transition  $D$ , it would result in another process making transition  $A$  immediately.  
II. A process  $P_2$  in blocked state can make transition  $E$ , while another process  $P_1$  is in running state.  
III. The OS uses pre-emptive scheduling.  
IV. The OS uses non-pre-emptive scheduling.
- Which of the above statements are true?  
(A) I and II (B) I and III  
(C) II and III (D) II and IV
6. Which of the following statements are true?  
I. Shortest remaining time first scheduling may cause starvation  
II. Pre-emptive scheduling may cause starvation  
III. Round Robin is better than FCFS in terms of response time [2010]  
(A) I only (B) I and III only  
(C) II and III only (D) I, II and III
7. A system has  $n$  resources  $R_0, \dots, R_{n-1}$ , and  $k$  processes  $P_0, \dots, P_{k-1}$ . The implementation of the resource request logic of each process  $P_i$  is as follows:

```

if (i%2 == 0) {
if (i<n) request Ri;
if (i+2<n) request Ri+2;
}
else {
if (i<n) request Rn-i;
if (i+2<n) request Rn-i-2;
}

```

In which one of the following situations is a deadlock possible? [2010]

- (A)  $n = 40, k = 26$  (B)  $n = 21, k = 12$   
 (C)  $n = 20, k = 10$  (D)  $n = 41, k = 19$

8. Consider the following table of arrival time and burst time for three processes  $P_0, P_1$  and  $P_2$ . [2011]

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| $P_0$   | 0 ms         | 9 ms       |
| $P_1$   | 1 ms         | 4 ms       |
| $P_2$   | 2 ms         | 9 ms       |

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

- (A) 5.0 ms (B) 4.33 ms  
 (C) 6.33 ms (D) 7.33 ms

9. Consider the three processes,  $P_1, P_2$  and  $P_3$  as shown in the table. [2012]

| Process | Arrival time | Time units required |
|---------|--------------|---------------------|
| $P_1$   | 0            | 5                   |
| $P_2$   | 1            | 7                   |
| $P_3$   | 3            | 4                   |

The completion order of the three processes under the policies FCFS and  $RR_2$  (Round Robin scheduling with CPU quantum of 2 time units) are

- (A) **FCFS:**  $P_1, P_2, P_3$  **RR2:**  $P_1, P_2, P_3$   
 (B) **FCFS:**  $P_1, P_3, P_2$  **RR2:**  $P_1, P_3, P_2$   
 (C) **FCFS:**  $P_1, P_2, P_3$  **RR2:**  $P_1, P_3, P_2$   
 (D) **FCFS:**  $P_1, P_3, P_2$  **RR2:**  $P_1, P_2, P_3$

10. A scheduling algorithm assigns priority proportional to the waiting time of a process. Every process starts with priority zero (the lowest priority). The scheduler re-evaluates the process priorities every  $T$  time units and decides the next process to schedule. Which one of the following is TRUE if the processes have no I/O operations and all arrive at time zero? [2013]

- (A) This algorithm is equivalent to the first-come-first-serve algorithm.  
 (B) This algorithm is equivalent to the Round Robin algorithm.

(C) This algorithm is equivalent to the shortest-job-first algorithm.

(D) This algorithm is equivalent to the shortest-remaining-time-first algorithm.

11. An operating system uses the *Banker's algorithm* for deadlock avoidance when managing the allocation of three resource types  $X, Y$  and  $Z$  to three processes  $P_0, P_1$ , and  $P_2$ . The table given below presents the current system state. Here, the *allocation* matrix shows the current number of resources of each type allocated to each process and the *Max* matrix shows the maximum number of resources of each type required by each process during its execution. [2014]

|       | Allocation |   |   | Max |   |   |
|-------|------------|---|---|-----|---|---|
|       | X          | Y | Z | X   | Y | Z |
| $P_0$ | 0          | 0 | 1 | 8   | 4 | 3 |
| $P_1$ | 3          | 2 | 0 | 6   | 2 | 0 |
| $P_2$ | 2          | 1 | 1 | 3   | 3 | 3 |

There are three units of type  $X$ , two units of type  $Y$  and two units of type  $Z$  still available. The system is currently in a *safe* state. Consider the following independent requests for additional resources in the current state:

REQ1:  $P_0$  requests 0 units of  $X$ , 0 units of  $Y$  and two units of  $Z$

REQ 2:  $P_1$  requests two units of  $X$ , 0 units of  $Y$  and 0 units of  $Z$

Which one of the following is true?

- (A) Only REQ1 can be permitted  
 (B) Only REQ2 can be permitted  
 (C) Both REQ1 and REQ2 can be permitted  
 (D) Neither REQ1 nor REQ2 can be permitted

12. Consider the following set of processes that need to be scheduled on a single CPU. All the times are given in milliseconds.

| Process name | Arrival time | Execution time |
|--------------|--------------|----------------|
| A            | 0            | 6              |
| B            | 3            | 2              |
| C            | 5            | 4              |
| D            | 7            | 6              |
| E            | 10           | 3              |

Using the *shortest remaining time first* scheduling algorithm, the average process turnaround time (in msec) is \_\_\_\_\_. [2014]

13. Three processes  $A, B$  and  $C$  each execute a loop of 100 iterations. In each iteration of the loop, a process performs a single computation that requires  $t_c$  CPU milliseconds and then initiates a single I/O operation that lasts for  $t_{io}$  milliseconds. It is assumed that the computer where the processes execute has sufficient number of I/O devices and the OS of the computer assigns different I/O devices to each process. Also, the scheduling overhead of the OS is negligible. The processes have the following characteristics:

| Process id | $t_c$  | $t_{io}$ |
|------------|--------|----------|
| A          | 100 ms | 500 ms   |
| B          | 350 ms | 500 ms   |
| C          | 200 ms | 500 ms   |

The processes  $A$ ,  $B$  and  $C$  are started at times 0, 5 and 10 milliseconds, respectively, in a pure time sharing system (Round Robin scheduling) that uses a time slice of 50 milliseconds. The time in milliseconds at which process  $C$  would complete its first I/O operation is \_\_\_\_\_. [2014]

14. A system contains three programs and each requires three tape units for its operation. The minimum number of tape units which the system must have such that deadlocks never arise is \_\_\_\_\_. [2014]
15. An operating system uses *shortest remaining time first* scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in milliseconds):

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| $P_1$   | 0            | 12         |
| $P_2$   | 2            | 4          |
| $P_3$   | 3            | 6          |
| $P_4$   | 8            | 5          |

The average waiting time (in milliseconds) of the processes is \_\_\_\_\_. [2014]

16. Consider a uniprocessor system executing three tasks  $T_1$ ,  $T_2$  and  $T_3$ , each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds, respectively. The priority of each task is the inverse of its period, and the available tasks are scheduled in order of priority, with the highest priority task schedule first. Each instance of  $T_1$ ,  $T_2$  and  $T_3$  requires an execution time of 1, 2 and 4 milliseconds, respectively. Given that all tasks initially arrive at the beginning of the 1st millisecond and task preemptions are allowed, the first instance of  $T_3$  completes its execution at the end of \_\_\_\_\_ milliseconds. [2015]
17. A system has 6 identical resources and  $N$  processes competing for them. Each process can request at most 2 resources. Which one of the following values of  $N$  could lead to a deadlock? [2015]
- (A) 1 (B) 2  
(C) 3 (D) 4
18. The maximum number of processes that can be in Ready state for a computer system with  $n$  CPUs is \_\_\_\_\_. [2015]
- (A)  $n$  (B)  $n^2$   
(C)  $2^n$  (D) Independent of  $n$
19. Consider the following policies for preventing deadlock in a system with mutually exclusive resources. [2015]

- (1) Processes should acquire all their resources at the beginning of execution. If any resource is not available, all resources acquired so far are released.
- (2) The resources are numbered uniquely, and processes are allowed to request for resources only in increasing resource numbers.
- (3) The resources are numbered uniquely, and processes are allowed to request for resources only in decreasing resource numbers.
- (4) The resources are numbered uniquely. A process is allowed to request only for a resource with resource number larger than its currently held resources.

Which of the above policies can be used for preventing deadlock?

- (A) Any one of 1 and 3 but not 2 or 4  
(B) Any one of 1, 3 and 4 but not 2  
(C) Any one of 2 and 3 but not 1 or 4  
(D) Any one of 1, 2, 3 and 4
20. For the processes listed in the following table, which of the following scheduling schemes will give the lowest average turnaround time? [2015]

| Process | Arrival Time | Processing Time |
|---------|--------------|-----------------|
| A       | 0            | 3               |
| B       | 1            | 6               |
| C       | 4            | 4               |
| D       | 6            | 2               |

- (A) First Come First Serve  
(B) Non-preemptive Shortest Job First  
(C) Shortest Remaining Time  
(D) Round Robin with Quantum value two
21. Consider an arbitrary set of CPU - bound processes with unequal CPU burst lengths submitted at the same time to a computer system. Which one of the following process scheduling algorithms would minimize the average waiting time in the ready queue? [2016]
- (A) Shortest remaining time first  
(B) Round-robin with time quantum less than the shortest CPU burst  
(C) Uniform random  
(D) Highest priority first with priority proportional to CPU burst length
22. Consider the following processes, with the arrival time and the length of the CPU burst given in milliseconds. The scheduling algorithm used is preemptive shortest remaining - time first.

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0            | 10         |
| $P_2$   | 3            | 6          |

|                |   |   |
|----------------|---|---|
| P <sub>3</sub> | 7 | 1 |
| P <sub>4</sub> | 8 | 3 |

The average turn around time of these processes is \_\_\_\_\_ milliseconds. [2016]

23. Consider the following CPU processes with arrival times (in milliseconds) and length of CPU bursts (in milliseconds) as given below :

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| P1      | 0            | 7          |
| P2      | 3            | 3          |
| P3      | 5            | 5          |
| P4      | 6            | 2          |

If the pre-emptive shortest remaining time first scheduling algorithm is used to schedule the processes, then the average waiting time across all processes is milliseconds. [2017]

24. A system shares 9 tape drives. The current allocation and maximum requirement of tape drives for three processes are shown below:

| Process | Current Allocation | Maximum Requirement |
|---------|--------------------|---------------------|
| P1      | 3                  | 7                   |
| P2      | 1                  | 6                   |
| P3      | 3                  | 5                   |

Which of the following best describes current state of the system? [2017]

- (A) Safe. Deadlocked  
(B) Safe. Not Deadlocked  
(C) Not Safe. Deadlocked  
(D) Not Safe, Not Deadlocked
25. Consider the set of processes with arrival time (in milliseconds). CPU burst time (in milliseconds). and priority (0 is the highest priority) shown below. None of the processes have I/O burst time.

| Process        | Arrival Time | Burst Time | Priority |
|----------------|--------------|------------|----------|
| P <sub>1</sub> | 0            | 11         | 2        |
| P <sub>2</sub> | 5            | 28         | 0        |

|                |    |    |   |
|----------------|----|----|---|
| P <sub>3</sub> | 12 | 2  | 3 |
| P <sub>4</sub> | 2  | 10 | 1 |
| P <sub>5</sub> | 9  | 16 | 4 |

The average waiting time (in milliseconds) of all the processes using preemptive priority scheduling algorithm is \_\_\_\_\_. [2017]

26. Consider a system with 3 processes that share 4 instances of the same resource type. Each process can request a maximum of  $K$  instances. Resource instances can be requested and released only one at a time. The largest value of  $K$  that will always avoid deadlock is \_\_\_\_\_. [2018]

27. In a system, there are three types of resources:  $E$ ,  $F$  and  $G$ . Four processes  $P_0$ ,  $P_1$ ,  $P_2$  and  $P_3$  execute concurrently. At the outset, the processes have declared their maximum resource requirements using a matrix named Max as given below. For example,  $\text{Max}[P_2, F]$  is the maximum number of instances of  $F$  that  $P_2$  would require. The number of instances of the resources allocated to the various processes at any given state is given by a matrix named Allocation.

Consider a state of the system with the Allocation matrix as shown below, and in which 3 instances of  $E$  and 3 instances of  $F$  are the only resources available.

| Allocation     |   |   |   | Max            |   |   |   |
|----------------|---|---|---|----------------|---|---|---|
|                | E | F | G |                | E | F | G |
| P <sub>0</sub> | 1 | 0 | 1 | P <sub>0</sub> | 4 | 3 | 1 |
| P <sub>1</sub> | 1 | 1 | 2 | P <sub>1</sub> | 2 | 1 | 4 |
| P <sub>2</sub> | 1 | 0 | 3 | P <sub>2</sub> | 1 | 3 | 3 |
| P <sub>3</sub> | 2 | 0 | 0 | P <sub>3</sub> | 5 | 4 | 1 |

From the perspective of deadlock avoidance, which one of the following is true? [2018]

- (A) The system is in safe state.  
(B) The system is not in safe state, but would be safe if one more instance of  $E$  were available.  
(C) The system is not in safe state, but would be safe if one more instance of  $F$  were available.  
(D) The system is not in safe state, but would be safe if one more instance of  $G$  were available.

**ANSWER KEYS****EXERCISES****Practice Problem 1**

- |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 1. C  | 2. A  | 3. C  | 4. C  | 5. B  | 6. D  | 7. A  | 8. C  | 9. A  | 10. B |
| 11. A | 12. C | 13. A | 14. A | 15. B | 16. B | 17. C | 18. B | 19. D |       |

**Practice Problem 2**

- |       |       |       |       |       |      |      |      |      |       |
|-------|-------|-------|-------|-------|------|------|------|------|-------|
| 1. B  | 2. C  | 3. D  | 4. A  | 5. C  | 6. A | 7. B | 8. C | 9. B | 10. A |
| 11. D | 12. A | 13. B | 14. B | 15. B |      |      |      |      |       |

**Previous Years' Questions**

- |       |                |          |       |         |        |          |       |       |       |
|-------|----------------|----------|-------|---------|--------|----------|-------|-------|-------|
| 1. A  | 2. B           | 3. C     | 4. A  | 5. C    | 6. D   | 7. B     | 8. A  | 9. C  | 10. B |
| 11. B | 12. 7.2        | 13. 1000 | 14. 7 | 15. 5.5 | 16. 12 | 17. none | 18. D | 19. D | 20. C |
| 21. A | 22. 8.2 to 8.3 | 23. 3    | 24. B | 25. 29  | 26. 2  | 27. A    |       |       |       |

# Chapter 4

## Memory Management and Virtual Memory

### LEARNING OBJECTIVES

- Basic concepts
- Memory management requirements
- Relocation and memory mapping techniques
- Placement algorithm
- Dynamic partitioning
- Placement algorithm
- Buddy system
- Non-contiguous storage allocation methods
- Paging
- Segmentation
- Page table structure
- Hierarchical page table
- Inverted page table
- Address translation in a segmentation system

### BASIC CONCEPTS

**Uniprogramming system** Main memory is divided into two parts as follows:

1. Operating system (OS) part
2. Program part (which is currently being executed)

**Multiprogramming system** Here the user part of memory must be further subdivided to accommodate multiple processes.

The task of subdivision is carried out dynamically by the OS and is known as memory management.

### Memory Hierarchy

The triangle in Figure 1 gives the hierarchy of memory. The memory hierarchy shows the performance issues.

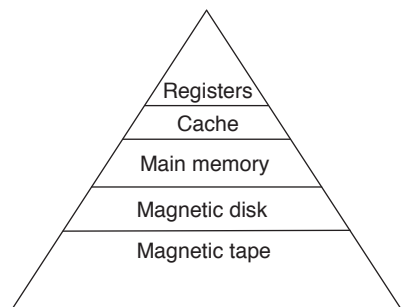


Figure 1 Memory hierarchy.

The memory hierarchy has different types of storage system in computers which are arranged in hierarchy, with respect to speed and cost.

If one moves down the hierarchy, access time increases, the cost per bit decreases, the memory capacity increases and memory access frequency by the processor decreases.

The registers, cache and main memory are volatile, whereas magnetic disc and magnetic tapes are non-volatile storage devices.

### MEMORY MANAGEMENT REQUIREMENTS

Memory management requirements are as follows:

1. Relocation
2. Protection
3. Sharing
4. Logical organization
5. Physical organization

### Relocation

1. The role of relocation, the ability to execute processes independently from their physical location in memory, is central for memory management.
2. In a general purpose multiprogramming environment, a program cannot know in advance what processes will be running in memory when it is executed, nor how much memory the system has available for it, nor where it is located.
3. Hence program relocation is required such that a program must be compiled and linked in such a way that it can later be loaded starting from an unpredictable address in memory, an address that can even change during the execution of the process itself, if any swapping occurs.

4. The basic requirement for program relocation is that all the references to memory it makes during execution must not contain absolute (physical) address of memory cells, but must be generated relatively, that is, as a distance measured in number of contiguous memory words, from some known point.

## Protection

Each process should be protected against unwanted interference by other processes, whether accidental or intentional.

Thus, programs in other processes should not be able to reference memory locations in a process for reading or writing purpose without permission.

## Sharing

1. Any protection mechanism must have the flexibility to allow several processes to access the same portion of main memory.
2. Processes that are cooperating on some task may need to share access to the same data structure.
3. The memory management system must therefore allow controlled access to shared areas of memory without compromising essential protection.

## Logical Organization

Main memory in a computer system is organized as a linear address space consisting of a sequence of bytes or words. But most of the programs are organized into modules.

If the OS and hardware can effectively deal with user programs and data in the form of modules, then there are some advantages.

1. Modules can be written and compiled independently.
2. Different degrees of protection can be given to different modules.
3. It is better to share modules among processes.

## Physical Organization

Computer memory is organized in two levels:

1. Main memory
2. Secondary memory

The flow of information between these two modules is a major concern. If this is assigned to user, then there are some problems:

1. **Overlaying** may be possible. In overlaying concept, the various modules of a program can be assigned to the same region of memory, which causes wastage of programmer time.
2. The programmer does not know at the time of coding how much space will be available or where that space will be. So it must be handled by the system.

**Address binding** Addresses may be represented in different ways during the program execution:

1. Addresses in source program are generally symbolic.
2. A compiler will typically bind these symbolic addresses to relocatable addresses.
3. The linkage editor or loader will in turn bind the relocatable addresses to absolute addresses.

So the binding of instructions and data to memory addresses can be done at any step along the way:

1. Compile time
2. Load time
3. Execution time

**Compile time** If you know at compile time where the process will reside in memory, then absolute code can be generated.

**Load time** If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code.

**Execution time** If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

## Logical versus Physical Address Space

**Logical address** An address generated by the CPU is commonly referred to as a logical address.

**Physical address** An address seen by the memory unit, that is, one loaded into MAR is referred to as a *physical address*.

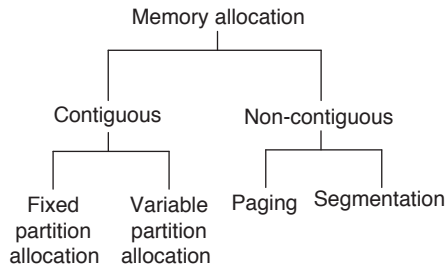
### Notes:

1. Logical and physical addresses differ in execution time address-binding scheme.
2. Logical and physical addresses are same in compile time and load time address-binding schemes.
3. The run-time mapping from logical to physical address is done by a hardware device called the *memory management unit* (MMU).

## MEMORY MAPPING TECHNIQUES

The principle operation of memory management is to bring processes into main memory for execution by the processor. Let's now discuss various memory management techniques as follows:

1. Fixed partitioning
2. Dynamic partitioning
3. Simple paging
4. Simple segmentation
5. Virtual memory paging
6. Virtual memory segmentation.



## Contiguous Storage Allocation

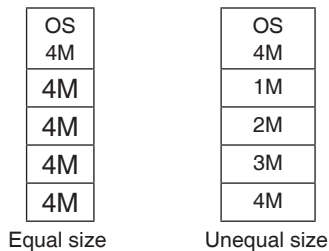
In this allocation, a memory resident program occupies a single contiguous block of memory.

## Fixed/Static Partitioning

The main memory is divided into a number of static partitions at system generation time. Moreover, a process may be loaded into a partition of equal or greater size.

**Partition size** Two alternatives of fixed partition are as follows:

1. Equal-size partitions
2. Unequal-size partitions



**Equal-size partitions:** Any process whose size is less than or equal to the partition size can be loaded into any available partition.

Two problems with this technique are as follows:

1. A program may be too big to fit into a partition. Use overlaying to solve this problem.
2. Main memory utilization is extremely inefficient, as there is a possibility of internal fragmentation.

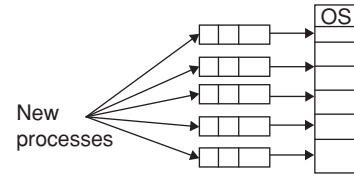
In *internal fragmentation*, there is a space wastage internal to a partition due to the fact that the block of data loaded is smaller than the partition.

**Unequal-sized partition:** Both the problems with equal-size partition can be lessened by using unequal-sized partitions.

**Placement algorithm:** With equal-size partitions, the placement of processes in memory is trivial. As all partitions are of equal size, it doesn't matter which partition is used.

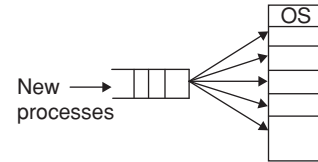
With Unequal-size partitions, there are two possible ways to assign processes to partitions:

1. Assign each process to the smallest partition within which it will fit.



- Figure shows one process queue for partition.
- Minimized internal fragmentation.
- Possibility of unused partitions.

2. Employ a single queue for all processes.



- When it is time to load a process into main memory, the smallest available partition that will hold the process is selected.

## Advantages

1. Simple to implement.
2. Little OS overhead.

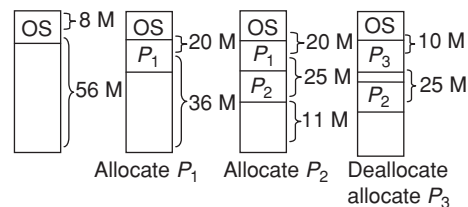
## Disadvantages

1. Inefficient use of memory due to internal fragmentation.
2. Maximum number of active processes is fixed.

## Dynamic Partitioning

With dynamic partitioning, the partitions are of variable length and number. When a process is brought into main memory, it is allocated exactly as much memory as it requires and no more.

### Example:



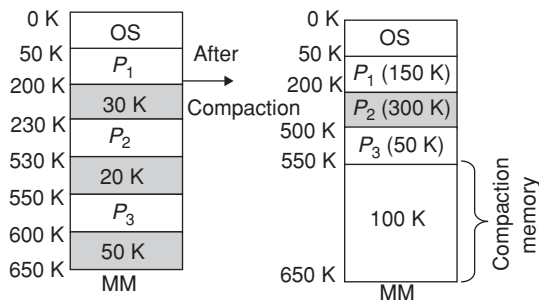
- This method starts out well, but eventually it leads to a situation in which there are a lot of small holes in memory.
- As time goes on, memory becomes more and more fragmented and memory utilization declines. This phenomenon is referred to as *external fragmentation*.

It indicates the memory that is external to all partitions becomes increasingly fragmented.

## Compaction

Compaction is a technique by which the resident program are relocated in such a way that the small chunks of free

memory are made contiguous to each other and clubbed together into a single free partition that may be big enough to accommodate more programs.



It should be noted that compaction involves dynamic relocation of a program.

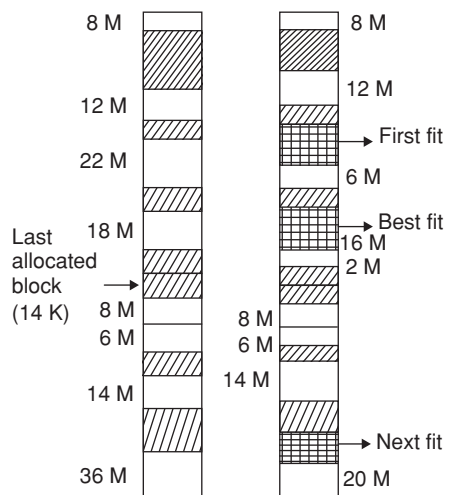
### Placement algorithm

Memory compaction is a time-consuming process, and hence the OS uses some placement algorithms.

The three most common strategies to allocate free partitions to the new processes are as follows:

1. *First fit*: Allocate the first free partition, large enough to accommodate the process. IT executes faster.
2. *Best fit*: Allocate the smallest free partition that meets the requirement of the process. It achieves higher utilization of memory by searching smallest free partition.
3. *Worst fit*: Allocate the largest available partition to the newly entered process in the system.
4. *Next fit*: Start from current location in the list.

**Example:** Consider the following memory configuration after a number of placement and swapping out operations. The last block that was used was a 22 MB block from which a 14 MB partition was created. The figure (b) shows 16 MB allocation request.



(a) Before allocation

(b) After allocation

- Allocated block
- Free block
- Possible new allocation

### Advantages of dynamic partitioning

1. Memory utilization is generally better as partitions are created dynamically.
2. No internal fragmentation as partitions are changed dynamically.
3. The process of merging adjacent holes to form a single larger hole is called coalescing.

### Disadvantages

1. Lots of OS space, time, complex memory management algorithms are required.
2. Compaction time is very high.

**Buddy system:** Both fixed and dynamic partitioning schemes have drawbacks.

In Buddy system, memory blocks are available of size  $2^k$  words,  $L \leq k \leq U$ , where,

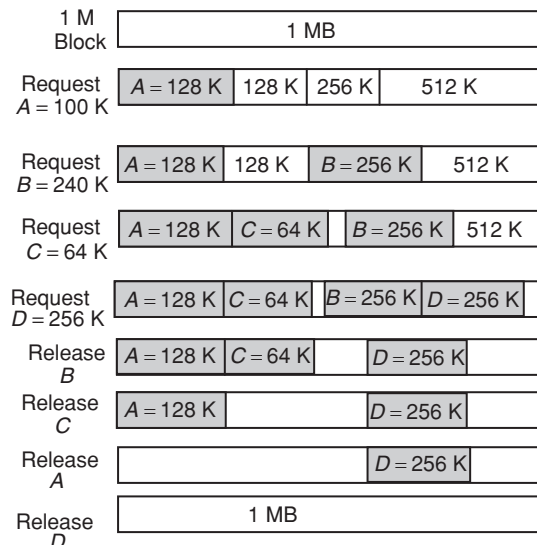
$2^L$  = Smallest-size block that is allocated.

$2^U$  = Largest-size block that is allocated.

Generally,  $2^U$  is the size of the entire memory available for allocation. If a request of size 'S' such that  $2^{U-1} < S \leq 2^U$  is made, then the entire block is allocated. Otherwise the block is split into two equal buddies of size  $2^{U-1}$ . If  $2^{U-2} < S \leq 2^{U-1}$ , then the request is allocated to one of the two buddies. Otherwise, one of the buddies is split in half again. This process continues until the smallest block greater than or equal to 'S' is generated and allocated to the request.

1. At any time, the buddy system maintains a list of holes of each size  $2^i$ .
2. A hole may be removed from the  $(i+1)$  list by splitting it in half to create two buddies of size  $2^i$  in the 'i' list.
3. Whenever a pair of buddies on the  $i$  list both become unallocated, they are removed from the list and coalesced into a single block on the  $(i+1)$  list.

### Example:



## Non-contiguous Storage Allocation Methods

### Paging

In simple paging, the main memory is divided into a number of equal-size frames. Each process is divided into a number of equal-size frames. The chunks of processes are referred as *pages*. A process is loaded by loading all of its pages into available, not necessarily contiguous frames.

**Example:** At a point in time, some of the frames in memory are in use and some are free. A list of free frames is maintained by the OS.

Consider four processes with their pages as displayed below:

|     |     |     |     |
|-----|-----|-----|-----|
| P.0 | Q.0 | R.0 | S.0 |
| P.1 | Q.1 | R.1 | S.1 |
| P.2 | Q.2 | R.2 | S.2 |
| P.3 |     | R.3 | S.3 |
|     |     |     | S.4 |

Process P    Process Q    Process R    Process S

Let the main memory consist of 15 frames:  
Main memory

|    |  |    |     |
|----|--|----|-----|
| 0  |  | 0  | P.0 |
| 1  |  | 1  | P.1 |
| 2  |  | 2  | P.2 |
| 3  |  | 3  | P.3 |
| 4  |  | 4  |     |
| 5  |  | 5  |     |
| 6  |  | 6  |     |
| 7  |  | 7  |     |
| 8  |  | 8  |     |
| 9  |  | 9  |     |
| 10 |  | 10 |     |
| 11 |  | 11 |     |
| 12 |  | 12 |     |
| 13 |  | 13 |     |
| 14 |  | 14 |     |

Load P

|   |     |   |     |   |     |   |     |
|---|-----|---|-----|---|-----|---|-----|
| 0 | P.0 | 0 | P.0 | 0 | P.0 | 0 | P.0 |
| 1 | P.1 | 1 | P.1 | 1 | P.1 | 1 | P.1 |
| 2 | P.2 | 2 | P.2 | 2 | P.2 | 2 | P.2 |
| 3 | P.3 | 3 | P.3 | 3 | P.3 | 3 | P.3 |

|    |     |    |     |    |     |    |     |
|----|-----|----|-----|----|-----|----|-----|
| 4  | Q.0 | 4  | Q.0 | 4  |     | 4  | S.0 |
| 5  | Q.1 | 5  | Q.1 | 5  |     | 5  | S.1 |
| 6  | Q.2 | 6  | Q.2 | 6  |     | 6  | S.2 |
| 7  |     | 7  | R.0 | 7  | R.0 | 7  | R.0 |
| 8  |     | 8  | R.1 | 8  | R.1 | 8  | R.1 |
| 9  |     | 9  | R.2 | 9  | R.2 | 9  | R.2 |
| 10 |     | 10 | R.3 | 10 | R.3 | 10 | R.3 |
| 11 |     | 11 |     | 11 |     | 11 | S.3 |
| 12 |     | 12 |     | 12 |     | 12 | S.4 |
| 13 |     | 13 |     | 13 |     | 13 |     |
| 14 |     | 14 |     | 14 |     | 14 |     |

Load Q    Load R    Swap Q    Load S

1. The OS maintains a page table for each process.
2. The page table shows the frame location for each page of the process.
3. Within a program, each logical address consists of a page number and an offset within the page.
4. Here a logical address is the location of a word relative to the beginning of the program; the processor translates that into a physical address.
5. For this, the processor must know the following details:

- *Logical address:* Consists page number and offset.
- *Page table:* Used to produce physical address (Frame number, offset).

In the previous example, the page tables of each process will be:

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| 0 | 0 | 0 | — | 0 | 7  |
| 1 | 1 | 1 | — | 1 | 8  |
| 2 | 2 | 2 | — | 2 | 9  |
| 3 | 3 |   |   | 3 | 10 |

Process P    Process Q    Process R

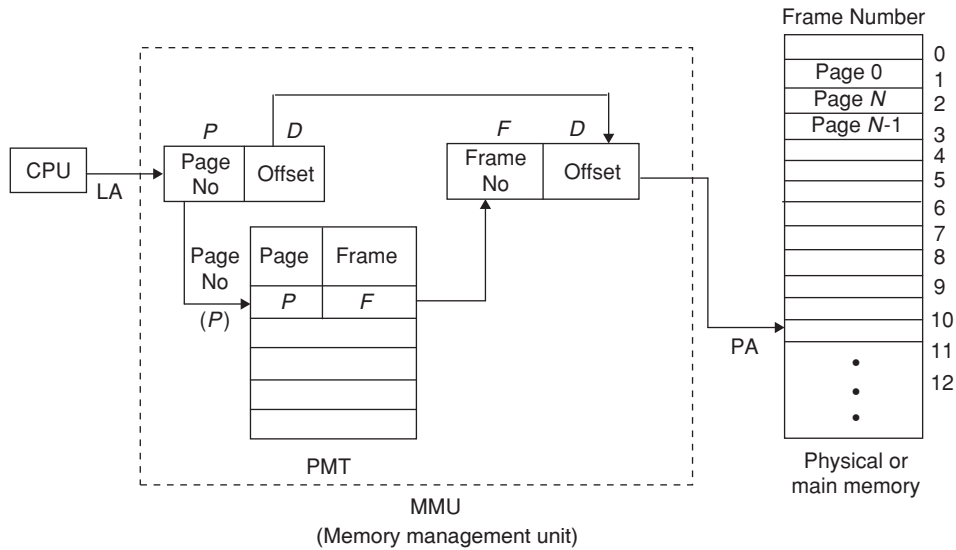
page table    page table    page table

|   |    |    |
|---|----|----|
| 0 | 4  | 13 |
| 1 | 5  | 14 |
| 2 | 6  |    |
| 3 | 11 |    |
| 4 | 12 |    |

Process S

page table    Free frame list.

### Address mapping in paging

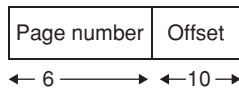


**Example:** Let 16-bit address is used by the processor and the page size is 1 KB.

$$\text{Then number of pages in main memory} = \frac{2^{16}}{2^{10}} = 2^6.$$

∴ Page number = 6-bits

Offset = 10-bits



For the relative address 1502 =

0000010111011110

The page number = 000001 = 1

and offset = 0111011110 = 478, that is, the physical location will be an offset (478)<sub>10</sub> on page 1.

Let the page 1 is present in frame 6. Then the physical address will be 0001100111011110.

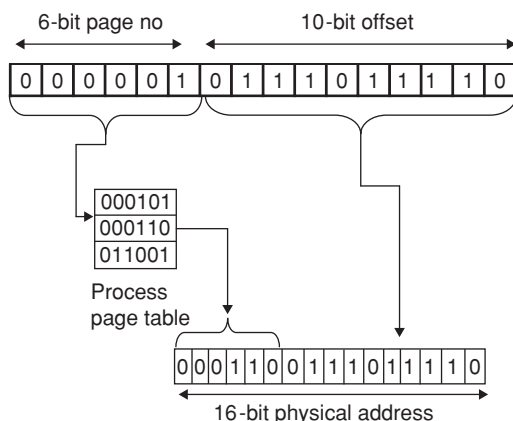


Figure 2 Paging.

### Steps for address translation

1. Extract the page number from the logical address.
2. Use the page number as an index into the process page table to find the frame number.
3. The physical address will be constructed by appending the frame number to the offset.

### Advantage

There is no external fragmentation.

### Disadvantage

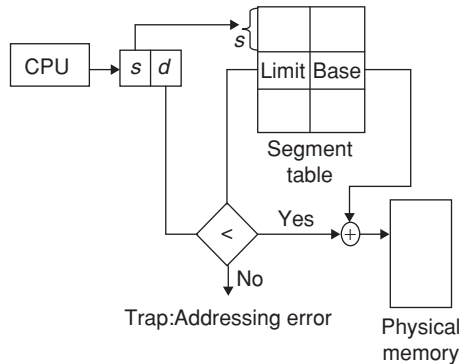
There is a small amount of internal fragmentation.

### Segmentation

1. Each process is divided into a number of unequal-size segments.
2. A process is loaded by loading all of its segments into dynamic partitions that need not be contiguous.
3. The logical address using segmentation consists of two parts: *segment number* and an *offset*.
4. The principle inconvenience of segmentation is that the programmer must be aware of the maximum segment size limitation.
5. It makes use of a segment table for each process and a list of free blocks of main memory.
6. Each segment table entry would have to give the starting address in main memory of the corresponding segment. It also contains the length of the segment.
7. Steps for address translation:
  - Extract the segment number from the logical address.
  - Use the segment number as an index into the process segment table to find the starting physical address of the segment.

- Compare the offset to the length of the segment. If the offset is greater than or equal to the length, the address is invalid.
- The desired physical address is the sum of the starting physical address of the segment plus the offset.

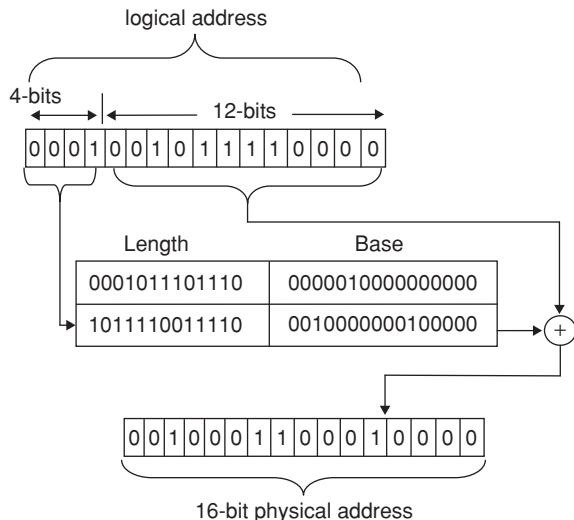
### Hardware support for segmentation



**Example:** Consider the logical address 0001001011110000.

Let the segment number consists of 4-bits. Then segment number = 0001 = 1

Offset = 001011110000 = 752.



### Advantages

1. No internal fragmentation
2. Improved memory utilization.
3. Reduced overhead compared to dynamic partitioning.

### Disadvantage

1. External fragmentation.

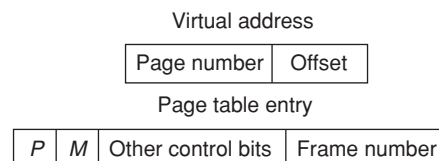
## VIRTUAL MEMORY

1. In simple paging/segmentation, it is not necessary that all of the pages or all of the segments of a process be in main memory during execution.

2. Suppose that it is time to bring new process into memory. The OS begins by bringing in only one or a few pages to include the initial program page and initial data page to which those instructions refer.
3. The portion of a process that is actually in main memory at any time is defined to be the resident set of the process.
4. If the processor encounters a logical address that is not in main memory, it generates an interrupt indicating a memory access fault.
5. Then the OS brings the required page to the main memory.
6. With virtual memory,
  - More processes may be maintained in main memory.
  - A process may be larger than all of main memory, then also it will be executed.
7. Virtual memory is a storage allocation scheme in which secondary memory can be addressed as though it were part of main memory.
8. **Thrashing:** When the OS brings one page in, it must throw another out. If it throws out a page just before it is used, then it will just have to go get that piece again almost immediately. Too much of this leads to a condition known as *thrashing*.  
If the system spends most of its time in swapping rather than executing instructions then that situation refers to thrashing.
9. Principle of locality suggests that a virtual memory scheme may work.
10. Virtual memory will be practical and effective if
  - There is a hardware support for paging/segmentation.
  - The OS includes software for managing the movement of pages/segments.

### Paging with Virtual Memory

1. The main difference between paging and virtual memory paging is that in virtual memory paging concept, not all pages of a process need to be in main memory frames for the process to run. Pages may be read in as needed.
2. A page table is also needed for a virtual memory scheme based on paging. Also it is typical to associate a unique page table with each process.
3. The virtual address and page table entries for virtual memory paging are shown below:



*P*: Present bit. This bit specifies whether that particular page is present in main memory or not.

*M*: Modified bit. This bit indicates whether the contents of the corresponding page have been altered since the page was last loaded into main memory.

### Page table structure

1. To read a word from memory, translate the virtual or logical address consisting of page number and offset into physical address consisting of frame number and offset, using a page table.
2. Page table must be stored in main memory to access it.

### Hardware implementation for virtual memory paging

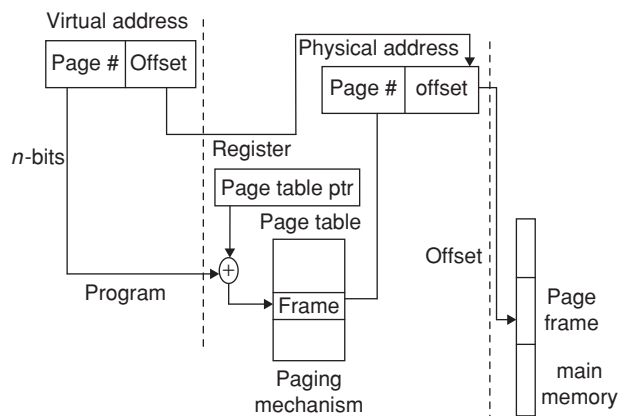
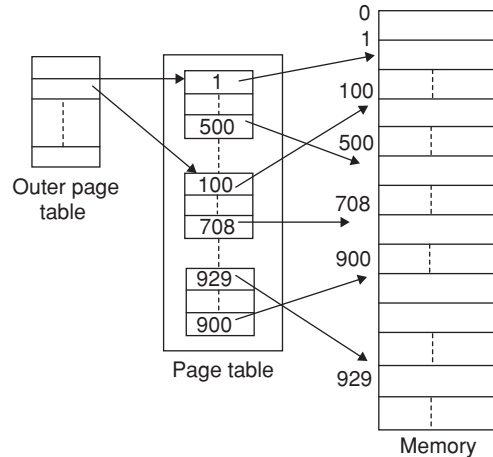


Figure 3 Address translation in paging system.

1. The amount of memory used by the page tables could be high.
2. To overcome this problem, most virtual memory schemes store page tables in virtual memory rather than real memory, that is, page tables are subject to paging just as other pages are.
3. When a process is running, at least a part of its page table must be in main memory, including the page table entry of the currently executing page.
4. Some processors make use of a two-level scheme to organize large page tables.

### Hierarchical page table

1. If page table size is large, then use hierarchical page table.
2. The logical address space is broken up into multiple page tables.

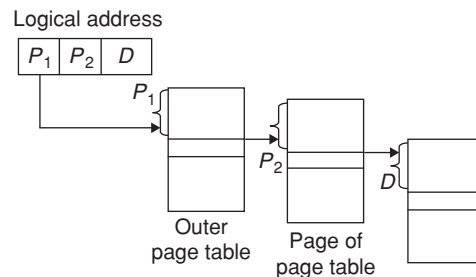


A logical address space (on 32-bit machine with 1 K page size) is divided into a page number consisting of 22-bits, page offset consisting of 10-bits. The page number is paged, the page number is divided into 12-bit page number and 10-bit page offset.

| Page number |       | Page offset |
|-------------|-------|-------------|
| $P_1$       | $P_2$ | $D$         |
| 12          | 10    | 10          |

Here,  $P_1$  is an index into the outer page table,  $P_2$  is the displacement within the page of the outer page table.

### Address translation (diagrammatic)



**Drawback:** Page table size is proportional to that of the virtual address space.

### Inverted page table

1. Here, the page number portion of a virtual address is mapped into a hash value, using simple hash function.
2. The hash value is a pointer to the inverted page table, which contains the page table entries.
3. There is one entry in the inverted page table for each real memory page frame rather than one per virtual page.
4. Thus, a fixed proportion of real memory is required for the tables.
5. One virtual address may map into the same hash table entry, so a chaining technique is used for managing the overflow.
6. The page table's structure is called *inverted*, because it indexes page table entries by frame number rather than by virtual page number.

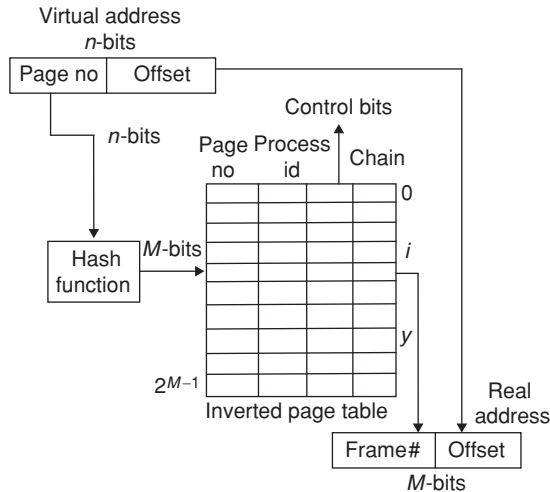
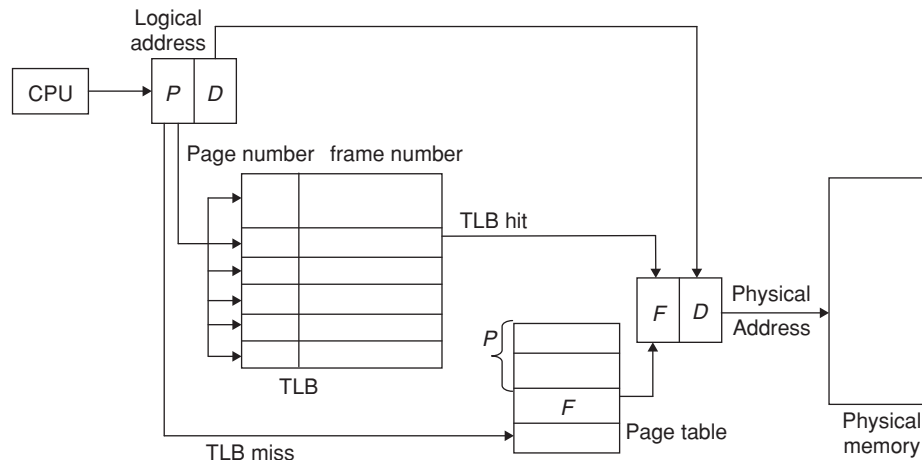


Figure 4 Inverted page table structure.

### Translation look-a-side buffer (TLB)

1. The straight-forward virtual memory scheme would have the effect of doubling the memory access time.
2. To overcome this problem, most virtual memory schemes make use of a special high speed cache for page table entries, usually called TLB.



### Organization of TLB

1. Each entry in TLB must include the page number and the complete page table entry.
2. The TLB may organized its entries either in
  - Direct mapping
  - Associative Mapping
3. The hardware must also consider the ways in which entries are organized and which entry to replace.

**Note:** The virtual memory mechanism must interact with the main memory cache system also.

**Page size** The factors to be considered for page size are as follows:

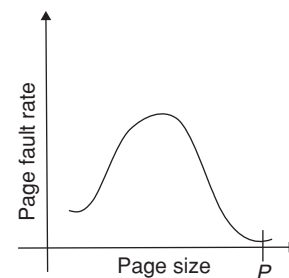
1. If page size is smaller: Then internal fragmentation is less. But it results in larger page tables. For large programs, the page fault rate increases.

3. TLB contains the page table entries that have been most recently used.

### Paging hardware with TLB

1. Given a virtual address, the processor will first examine the TLB. If the desired page table entry is present, that is, TLB hit, then the frame number is retrieved and real address is formed.
2. If there is a TLB miss, the processor uses the page number to index the process page table and examine the corresponding page table entry.
3. If present bit is set, then the page is in main memory and the processor can retrieve the frame number from the page table entry to form the real address.
4. The processor also updates the TLB to include this new page table entry.
5. If the page is not in main memory, then page fault is issued.
6. Then the OS will load the needed page and updates the page table.

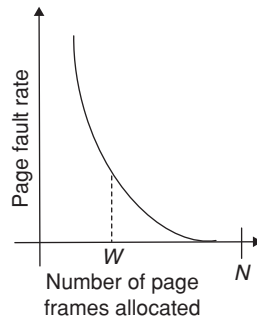
2. Rotational secondary devices favour a larger page size for more efficient block transfer.



where  $P$  = size of entire process.

The above figure shows the relationship between page size and page fault rate.

The page fault rate is also determined by the number of frames allocated to a process. This relation is shown below.



where  $W$  = working set size

$N$  = Total number of pages in process

**Note:** The design issue of page size is related to the size of physical main memory and program size.

### Advantages of virtual memory paging

1. No external fragmentation
2. Higher degree of multiprogramming
3. Large virtual address space

### Disadvantage

Overhead of complex memory management.

### Segmentation with Virtual Memory

1. Memory consists of multiple segments.
2. Segments are of unequal and dynamic in size.
3. It simplifies the handling of growing data structures.
4. It allows programs to be altered and recompiled independently.
5. It lends itself to sharing among processes.
6. It lends itself to protection.
7. A unique segment table is associated with each process.
8. The virtual address and segment table entries are as shown below:

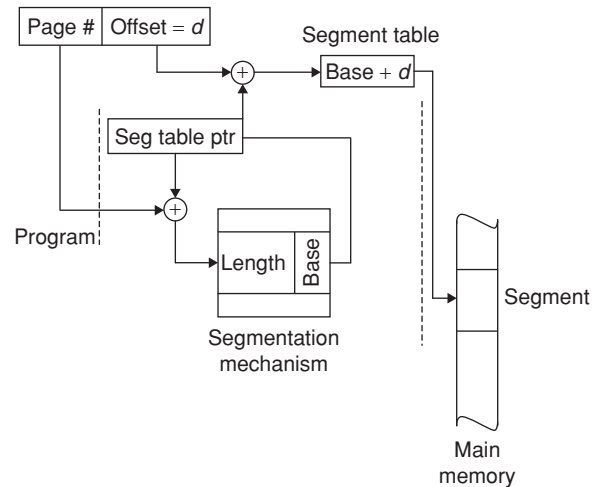
| Virtual address |        |
|-----------------|--------|
| Segment number  | Offset |

| Segment table entry |     |                    |        |              |
|---------------------|-----|--------------------|--------|--------------|
| $P$                 | $M$ | Other control bits | Length | Segment base |

9. Only some of the segments of a process may be in main memory. To identify which segment is present in the main memory, use present bit  $P$ .

10. To know whether the segment is modified or not, use  $M$ -bit.

### Address translation in a segmentation system (using virtual memory)



### Advantages

1. No internal fragmentation
2. Higher degree of multiprogramming
3. Large virtual address space.
4. Protection and sharing support.

### Disadvantage

1. Overhead of complex memory management.

### Combined paging and segmentation

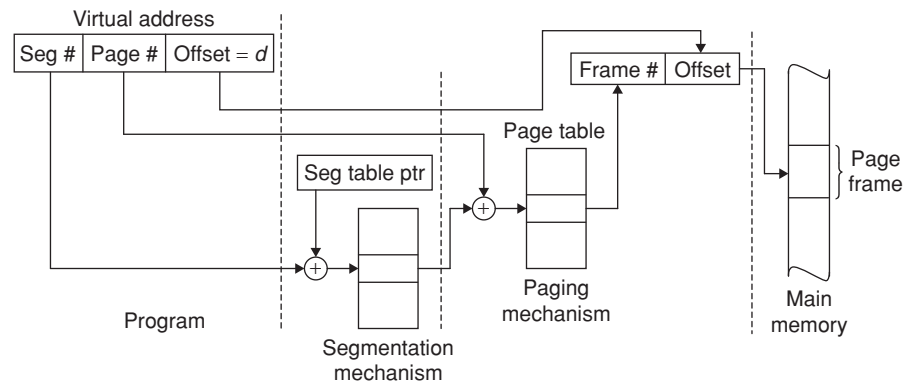
1. Here, the users address space is broken up into a number of segments by the programmer.
2. Each segment is, in turn, broken up into a number of fixed size pages, which are equal in length to a main memory frame.
3. If a segment has length less than that of a page, the segment occupies just one page.
4. The virtual address, segment table and page table entries are as shown below:

| Virtual address |             |        |
|-----------------|-------------|--------|
| Segment number  | Page number | Offset |

| Segment table entry |        |              |
|---------------------|--------|--------------|
| Control bits        | Length | Segment base |

| Page table entry |     |                    |              |
|------------------|-----|--------------------|--------------|
| $P$              | $M$ | Other control bits | Frame number |

### Structure of combined segmentation/paging system



#### Notes:

1. Segmentation lends itself to the implementation of protection and sharing policies.
2. To achieve sharing, it is possible for a segment to be referenced in the segment tables of more than one process.

## OS SOFTWARE FOR MEMORY MANAGEMENT

We consider the following software policies for virtual memory:

1. Fetch policy:
  - Demand
  - Prepaging
2. Placement policy
3. Replacement policy:
  - Optimal
  - LRU
  - FIFO
  - Clock
4. Resident set management
  - Resident set size
    - I. Fixed
    - II. Variable
  - Replacement scope
    - I. Local
    - II. Global
5. Cleaning policy
  - Demand
  - Pre-cleaning
6. Load control
  - Degree of multi-programming

**Fetch policy** Determines when a page should be brought into main memory.

1. **Demand paging:** Here a page is brought into main memory only when a reference is made to a location on that page.

2. **Prepaging:** It is a technique that reduces the large number of page faults at process start up.

- Prepaging is used to get before all or some of the pages a process will need, before they are referenced.
- If prepagged pages are unused, I/O and memory would be wasted.
- Assume 's' pages are prepagged and  $\alpha$  of the pages are used.
- Cost of  $(s * \alpha)$  to save page faults greater or less than the cost of prepagging  $s \times (1 - \alpha)$  unnecessary page. If  $\alpha$  near zero  $\Rightarrow$  prepaging is lost.

### Placement Policy

1. Determines where in real memory a process piece is to reside.
2. In pure segmentation system, the policies like best fit, first fit, etc., are used.
3. For a system that uses either pure paging or paging combined with segmentation, placement is usually irrelevant.

**Replacement policy** This deals with the selection of a page in main memory to be replaced when a new page must be brought in. In a replacement policy, we have to consider the following:

1. How many page frames are to be allocated to each active process.
2. Whether the set of pages to be considered for replacement should be limited to those of the process that caused the page fault or encompass all the page frames in main memory.
3. Among the set of pages considered, which particular page should be selected for replacement.

### Page Fault

1. Whenever a processor needs to execute a particular page and that page is not available in main memory, this situation is said to be *page fault*.

- When the page fault occurs, the page replacement will be done.
- 'Page Replacement' means select a victim page in the main memory, replace that page with the required page from the backing store (disk).
- Some of the replacement algorithms are as follows:
  - FIFO
  - Optimal
  - LRU
  - Clock

### FIFO (First-in-First-Out Algorithm)

- Replace a page that is the oldest page of all the pages of the main memory.
- Focuses on the length of time a page has been in memory rather than how much the page is being used.

#### Example:

Consider the reference string: 0 1 2 3 0 1 2 3 0 1 2 3 1

|        |        |        |        |        |        |  |
|--------|--------|--------|--------|--------|--------|--|
| 0<br>F | 1<br>F | 2<br>F | 3<br>F | 0<br>F | 1<br>F |  |
| 0      | 0      | 0      | 3      | 3      | 3      |  |
|        |        | 1      | 1      | 0      | 0      |  |
|        |        | 2      | 2      | 2      | 1      |  |

|        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|
| 2<br>F | 3<br>F | 0<br>F | 1<br>F | 2<br>F | 3<br>F | 1<br>H |
| 2      | 2      | 2      | 1      | 1      | 1      | 1*     |
| 0      | 3      | 3      | 3      | 2      | 2      | 2      |
| 1      | 1      | 0      | 0      | 0      | 3      | 3      |

Here the symbol 'F' indicates page fault.

The number of page faults = 12

'H' indicates the page is already in the memory. The remaining pages are not present in memory that is why page fault occurs. In general, the more frames there are, the less page fault.

$$\text{Page fault rate} = \frac{\text{Number of page faults}}{\text{Number of bits in reference string}}$$

$$= \frac{12}{13} = 0.923 = 92.3\%$$

### Belady's Anomaly

**Example:** Consider the reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Number of frames = 4

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| F | F | F | F | H | H |
| 1 | 1 | 1 | 1 | 1 | 1 |
|   | 2 | 2 | 2 | 2 | 2 |
|   |   | 3 | 3 | 3 | 3 |
|   |   | 4 | 4 | 4 | 4 |
| 1 | 2 | 3 | 4 | 1 | 2 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| F | F | F | F | F | F |
| 5 | 5 | 5 | 5 | 4 | 4 |
| 2 | 1 | 1 | 1 | 1 | 5 |
| 3 | 3 | 2 | 2 | 2 | 2 |
| 4 | 4 | 4 | 3 | 3 | 3 |
| 5 | 1 | 2 | 3 | 4 | 5 |

The number of page faults = 10

Consider the same reference string with three frames.

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| 1<br>F | 2<br>F | 3<br>F | 4<br>F | 1<br>F | 2<br>F |
| 1      | 1      | 1      | 4      | 4      | 4      |
|        | 2      | 2      | 2      | 1      | 1      |
|        |        | 3      | 3      | 3      | 2      |

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| 5<br>F | 1<br>H | 2<br>H | 3<br>F | 4<br>F | 5<br>H |
| 5      | 5      | 5      | 5      | 5      | 5      |
| 1      | 1      | 1      | 3      | 3      | 3      |
| 2      | 2      | 2      | 2      | 4      | 4      |

Here number of page faults = 9

Here, as the number of frames increases the page fault also increases. This is known as *Belady's anomaly*.

### Optimal page replacement algorithm

- Replace the page that will not be used for the longest period of time.

**Example:** Consider the reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 using four frames.

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| F | F | F | F | H | H |
| 1 | 1 | 1 | 1 | 1 | 1 |
|   | 2 | 2 | 2 | 2 | 2 |
|   |   | 3 | 3 | 3 | 3 |
|   |   | 4 | 4 | 4 | 4 |
| 1 | 2 | 3 | 4 | 1 | 2 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| F | H | H | H | F | H |
| 1 | 1 | 1 | 1 | 4 | 4 |
| 2 | 2 | 2 | 2 | 2 | 2 |
| 3 | 3 | 3 | 3 | 3 | 3 |
| 5 | 5 | 5 | 5 | 5 | 5 |
| 5 | 1 | 2 | 3 | 4 | 5 |

Number of page faults = 6

### Disadvantage

It requires future knowledge of reference string, so used for comparison studies.

### LRU (least recently used) algorithm

- Replace a page that has not been used for the longest period of time.
- It looks backward in time rather than forward.
- It associates with each page the time of that page last used.
- Two methods of implementation:
  - Counters
  - Stack

**Example:** Consider reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Number of frames = 4

| F | H | H | H | F | H |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 |
|   | 2 | 2 | 2 | 2 | 2 |
|   |   | 3 | 3 | 3 | 3 |
| 1 | 2 | 3 | 4 | 1 | 2 |

| H | H | H | F | F | F |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 5 |
| 2 | 2 | 2 | 2 | 2 | 2 |
| 5 | 5 | 5 | 5 | 4 | 4 |
| 4 | 4 | 4 | 3 | 3 | 3 |
| 4 | 4 | 4 | 1 | 2 |   |

Number of page faults = 8 (less than FIFO)

### Approach to Implement LRU Replacement

LRU is a good page replacement policy, but the problem with these is how to determine the frame used for the last time.

This is implemented by using two approaches:

1. Using counters
2. Using stacks

Using counters, LRU is implemented as follows: Every page entry has a counter, whenever a page is referenced, the clock value is copied into the counter. If the page has to be replaced, then it refers to the look up of the counter, which-ever is having oldest time that is changed.

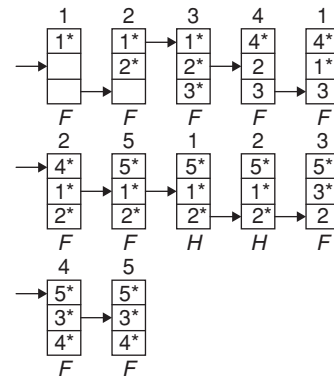
Using stack, LRU is implemented as follows: Form a doubly linked list of page numbers and keep it in stack whenever a page is referenced it is moved to the top of the stack that is top of the stack contains recently referenced page. Bottom of the stack will have least recently used one.

### Clock replacement algorithm

1. The simplest form of clock policy requires the association of an additional bit with each frame, referred to as the use bit.
2. When a page is first loaded into frame in memory, the use bit for that frame is set to 1.
3. Whenever the page is subsequently referenced, its use bit is set to 1.
4. The set of frames that are candidates for replacement is considered to be a circular buffer, with which a pointer is associated.
5. When a page is replaced, the pointer is set to indicate the next frame in the buffer after the one just updated.
6. When it comes time to replace a page, the OS scans the buffer to find a frame with a use bit set to 0.
7. Each time it encounters a frame with a use bit of 1, it resets that bit to 0 and continues on.
8. If any of the frames in the buffer have a use bit of 0 at the beginning of this process, the first such frame encountered is chosen for replacement.

9. If all of the frames have a use bit of 1, then the pointer will make one complete cycle through the buffer, setting all the use bits to 0 and stop at its original position, replacing the page in that frame.

**Example:** Consider the reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, with three main memory frames:



∴ Number of misses = 9

**Note:** The clock algorithm was approximately closer in performance to LRU.

### Effective memory access time

The percentage of times a page number is found in the associative registers is called the *hit ratio*. If we fail to find the page number in the associative registers, then we must first access memory for the page table and frame number, and then access the required byte in memory. To find the effective access time, we should weigh each case by its probability.

EMAT: Is given as  $= p * s + (1 - p) * m$ .

Where

$p$  = Page fault rate

$s$  = Page fault service time

$m$  = Main memory access time

$(1 - p)$  = page hit ratio.

**Frame locking** Some of the frames in main memory may be locked. When a frame is locked, the page currently stored in that frame may not be replaced.

**Page buffering** To improve performance, a replaced page is not lost but rather is assigned to one of two lists:

1. The free page list if the page has not been modified or
2. The modified page list if it has modified.

### Resident set management

**Resident set size** The OS must decide how many pages to bring in, that is, how much main memory to allocate to a particular process. Two policies are there:

1. Fixed allocation
2. Variable allocation

**Fixed allocation** This policy gives to a process a fixed number of frames in main memory within which to execute.

**Variable allocation** This policy allows the number of page frames allocated to a process to be varied over the life time of the process.

**Replacement scope** This is of two types as follows:

**Local page replacement:** When a process requests for a new page to be brought in and there are no free frames in the memory, we choose a frame allocated to only that process for replacement.

**Global replacement:** It allows a process to select a replacement frame from the set of all frames, even if that frame is currently allocated to some other processes. So, one process can take a frame from another.

#### **Fixed allocation, local replacement**

1. Number of frames allocated to a process is fixed.
2. Page to be replaced is chosen from among the frames allocated to that process.

#### **Variable allocation, global scope**

1. Page to be replaced is chosen from all available frames in main memory.
2. Size of resident set of processes varies.

#### **Variable allocation, local scope**

1. The number of frames allocated to a process may be changed from time to time.
2. Page to be replaced is chosen from among the frames allocated to that process.

**Working set:** This strategy is used to determine the resident set size and the timing of changes.

The working set with parameter  $\Delta$  for a process at virtual time  $t$ , which we designated as  $W(t, \Delta)$ , is the set of pages of that process that have been referenced in the last  $\Delta$  virtual time units.

**Virtual time:** Consider a sequence of memory references,  $r(1), r(2), \dots$  in which  $r(i)$  is the page that contains the  $i$ th virtual address generated by a given process.

Time is measured in memory references; thus,  $t = 1, 2, 3, \dots$  measures the processes internal virtual time.

The variable ' $\Delta$ ' is a window of virtual time over which the process is observed.

The working set size will be a non-decreasing function of the window size.

$$W(t, \Delta + 1) \supseteq W(t, \Delta).$$

For the sequence of page references 24, 15, 18, 23, 17, 15, 24, 18, 17, 17, 15. And window size = 2 then working set will be

{24, {24, 15}}, {15, 18}, {18, 23}, {23, 24}, {24, 17}, {17, 18}, {18, 24}, {18, 24}, {18, 17}, {17}, {17, 15}}

#### **Page fault frequency (PFF) algorithm**

1. The algorithm requires a use bit to be associated with each page in memory.
2. The bit is set to 1, when that page is accessed.
3. When a page fault occurs, the OS notes the virtual time since the last page fault for the process.
4. A threshold  $F$  is defined. If the amount of time since the last page fault is less than  $F$ , then a page added to resident set of the process.
5. Otherwise, discard all pages with a use bit of 0 and shrink the resident set according.

**Note:** PFF does not perform well during the transient periods when there is a shift to a new locality.

#### **Variable interval sampled working set (VSWS)**

1. The VSWS policy evaluates the working set of a process at sampling instances based on elapsed virtual time.
2. VSWS considers three parameters:  
 $M$ : The minimum duration of sampling interval  
 $L$ : The maximum duration of sampling interval  
 $Q$ : The number of page faults that are allowed to occur between sampling instances.
3. The policy works as following:
  - If virtual time since the last sampling instance reaches  $L$ , then suspend the process and scan the use bits.
  - If, prior to an elapsed virtual time of  $L$ ,  $Q$  page faults occur,
    - I. If the virtual time since the last sampling instance is less than  $M$ , then wait until the elapsed virtual time reaches  $M$  to suspend the process and scan the use bits.
    - II. If the virtual time since the last sampling instance is greater than or equal to  $M$ , suspend the process and scan the use bits.

**Cleaning policy** It determines when a modified page should be written out to secondary memory. Two approaches are as follows:

1. Demand cleaning
2. Pre-cleaning

**Demand cleaning** A page is written out to secondary memory only when it has been selected for replacement.

**Pre-cleaning** This policy writes modified pages before their page frames are needed so that pages can be written out in batches.

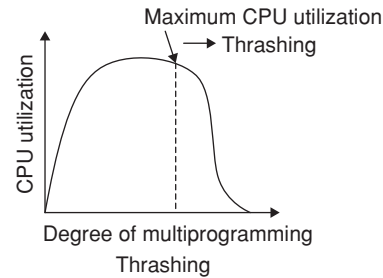
**Load control** It is concerned with determining the number of processes that will be resident in main memory, which has been referred to as the multiprogramming level.

If too few processes are resident at any one time, it leads to swapping. If too many processes are present, thrashing will occur.

**Cause of thrashing** Consider the following scenario:

The OS monitors CPU utilization. If the utilization is too low, we increase the degree of multiprogramming by introducing a new process to the system. A global page replacement algorithm is used, which replaces pages with no regard to the process to which they belong. Now, say that a process enters a new phase in its execution and needs more frames. It starts faulting and taking frames away from other processes. These processes need those pages, however and so they also fault, taking frames from other processes. These faulting processes must use the paging device to swap pages in and out. As they queue up for paging device, the ready queue empties. As processes wait, for the paging device, CPU utilization decreases the CPU scheduler sees the decreasing CPU utilization; so it increases the degree of multiprogramming. The new process tries to get started by taking frames from running processes, causing

more page faults, and a longer queue for paging device. As a result, CPU utilization drops even, further. The CPU scheduler tries to increase the degree of multiprogramming even more. Thrashing occurs, and the system throughput plunges. The page fault rate (FT) increases tremendously. Effective memory access time increases. No work is getting done because the processes are spending all their time in paging.



## EXERCISES

### Practice Problems I

**Directions for questions 1 to 20:** Select the correct alternative from the given choices.

1. Consider a 3-level memory hierarchy shown in the following table, with access times to memory:

| Hierarchy level | Cache hit ratio | Page transfer time |
|-----------------|-----------------|--------------------|
| $M_1$           | 0.55            | 0.003 ms           |
| $M_2$           | 0.92            | 0.3 ms             |
| $M_3$           | –               | 1.0 ms             |

When a miss occurs, data is fetched from the next level. Calculate the average time required for a process to read one word from the memory system.

- (A) 0.379 (B) 0.162  
(C) 0.2798 (D) 0.172
2. Consider a memory system with FIFO page replacement algorithm policy. For an arbitrary page access pattern, increasing the number of page frames in main memory will
- (A) Always decrease the number of page faults  
(B) Always increase the number of page faults  
(C) Sometimes increase the number of page faults  
(D) Never effect the number of page faults
3. Consider the below page address stream generated by executing a program:
- 4 5 4 3 7 4
- Assuming that LRU is used for page replacement and at most three frames are available in the memory for the process, find the number of page faults that can occur (initially all frames empty).
- (A) 0 (B) 1  
(C) 3 (D) 4

4. Consider a logical address space of four pages of 2048 words each mapped into a physical memory of 32 frames. How many bits in logical address?
- (A) 12-bits (B) 14-bits  
(C) 13-bits (D) 11-bits
5. The time taken to service a page fault is on average 10 ms and the memory access time is 20  $\mu$ s. If the hit ratio is 70%, calculate the average access time.
- (A) 3018  $\mu$ s (B) 4014  $\mu$ s  
(C) 3014  $\mu$ s (D) 4024  $\mu$ s
6. Consider the following page trace:
- 4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5
- Number of frames for the Job  $M = 4$ . Then the page fault ratio using FIFO technique will be
- (A) 63% (B) 75%  
(C) 83% (D) 94%
7. The available main memory for loading pages is 64 MB with a frame size of 8 MB. If pages of size 6 MB, and 4 MB are loaded into memory, what is the percentage of the internal fragmentation resulted?
- (A) 42.5 (B) 37.5  
(C) 57.5 (D) 62.45
8. A demand paging system takes 50 time units to handle a page fault and 200 time units to replace a dirty page. Access time of memory is 2 time units. Probability of page fault and dirty page is  $P$ . Average access time is 4 time units. Then what is the value of  $P$ ?
- (A) 0.037 (B) 0.027  
(C) 0.012 (D) 0.042
9. Assume that a total memory 20 KB is available with no partition. If Buddy system technique is used and there are total of four partitions to serve the request, the closest range of the requested size is

- (A) 12.5 and 25                      (B) 14.5 and 30  
(C) 15.5 and 25                      (D) 14.5 and 40
10. A system uses FIFO page replacement algorithm. It has three page frames with no pages loaded. First 50 pages are accessed in some order and the same pages are accessed in the reverse order. What is the number of page faults?  
(A) 98                                      (B) 96  
(C) 97                                      (D) 95
11. If 32-bit addressing is used for pages whose maximum size is 512 KB, what is the maximum number of pages that can be addressed?  
(A) 4096                                      (B) 2048  
(C) 8192                                      (D) 16384
12. Calculate the overhead due to page table if given the average process in bytes is 16-bytes, the page size is 32-bytes and the page entry is 2-bytes.  
(A) 15                                      (B) 17  
(C) 18                                      (D) 19
13. In a system with 32-bit virtual address and 1 KB page size, use of one-level page tables for virtual to physical address translation is not practical because of  
(A) The large amount of internal fragmentation  
(B) The large amount of external fragmentation  
(C) The large memory overhead in maintaining page tables  
(D) The large computation overhead in the translation process
14. If an instruction takes time 10 m sec if there is no page fault and time 20 m sec if there is a page fault, what is the effective instruction time if page fault occurs once every 5 instructions?  
(A) 12.5 msec                                      (B) 12 msec  
(C) 14 msec                                      (D) 15 msec
15. Let an instruction take 10 ms and page fault takes an additional 5 ms. If the average page fault occurs after 20 instructions, the effective instruction time will be  
(A) 10 ms                                      (B) 10.25 ms  
(C) 0.25 ms                                      (D) 10.75 ms
16. A 0.8 MB-sized memory is managed using variable partitions, while the rest of the memory is occupied

by a 0.26 MB partition, 0.27 MB partition and 0.25 MB partitions in the order. Best-fit strategy is being adopted where would be a 0.18 MB allocation request is fulfilled?

- (A) 0.26  
(B) 0.27  
(C) 0.25  
(D) Request will be denied

**Common data for questions 17 and 18:** Suppose that the OS uses variable length partitions for memory management. At some particular time, the running process occupies a partition between physical addresses 20,000 and 40,000.

17. The values of base and limit register are respectively  
(A) 20, 000, 40,000  
(B) 20,000, 20,000  
(C) 0, 10,000  
(D) 0, 40,000
18. What physical address corresponds to a virtual address of 13,000?  
(A) 13,000  
(B) 43,000  
(C) 33,000  
(D) Out of range
19. Consider a page table where translation look ahead buffer is used. TLB hit ratio is 0.95 and generally takes 1 nanosecond to retrieve the frame number. If a miss is recorded by the TLB then an additional overhead of 10 nanoseconds should be taken into consideration, further cache and main memory reference takes 100 ns on average, what is the average memory fetch time using the TLB? Assume main memory accesses are always a success.  
(A) 101 ns                                      (B) 105 ns  
(C) 200 ns                                      (D) 205 ns
20. Consider a 1 MB process which is divided into five segments. Each segment is further divided into pages whose size is 4 KB. What is the maximum segments possible? Assume that the system is byte addressable.  
(A) 64                                      (B) 128  
(C) 256                                      (D) 512

## Practice Problems 2

**Directions for questions 1 to 20:** Select the correct alternative from the given choices.

1. Consider a logical address space of 32 pages of 2048 words mapped into memory of 64 frames. Then the number of bits required for logical address are  
(A) 16-bits                                      (B) 17-bits  
(C) 18-bits                                      (D) 20-bits
2. In which of the page table techniques the logical address space is broken into multiple page table?

- (A) Inverted Page Table  
(B) Hierarchical Page Table  
(C) Hashed Page Table  
(D) None of the above

3. Consider a system with 70% hit ratio, 60 nanoseconds time to search the associative registers, 800 nanoseconds to access memory. What is the effective memory access time?  
(A) 1200 nsec                                      (B) 1100 nsec  
(C) 1300 nsec                                      (D) 2200 nsec

4. On a system using fixed partitions, all of size  $2^8$ , the number of bits used by the limit register is  
(A) 128 (B) 256  
(C) 8 (D) 1024
5. Working set  $(t, k)$  at an instant of time,  $t$ , is the set of  
(A)  $k$  future references that the operating system will make  
(B) future references that the operating system will make in the next ' $k$ ' time units  
(C)  $k$  references with high frequency  
(D) pages that have been referenced in the last  $k$  time units.
6. Cache and interleaved memories are ways of speeding up memory access between CPUs and slower RAM. Which of the following memory models are best suited (i.e., improves performance the most) for which programs?  
(i) Cached memory is best suited for small loops.  
(ii) Interleaved memory is best suited for small loops.  
(iii) Interleaved memory is best suited for large loops.  
(iv) Cache memory is best suited for large sequential code.  
(A) (i) and (ii) are true  
(B) (i) and (iii) are true  
(C) (iv) and (ii) are true  
(D) (iv) and (iii) are true
7. A paging system with a page table in memory every reference to memory takes 100 ns. The TLB hit ratio is 85% and the time needed for searching TLB is almost negligible. What is the effective memory access time?  
(A) 115 ns (B) 135 ns  
(C) 145 ns (D) 125 ns
8. Consider the page sequence 4, 2, 1, 5, 3, 2, 1, 5, 0, 2, 5. If FIFO page replacement algorithm is used and frame size is 3, then the percentage of page fault is  
(A) 99% (B) 90.9%  
(C) 80.8% (D) 89.9%
9. If the page size is 32 KB, primary page table contains 4096 entries and the secondary page table contains 256 entries, then what is the size of logical address in bits?  
(A) 15-bits (B) 20-bits  
(C) 32-bits (D) 35-bits
10. If page size is 2 KB and logical address is 20-bit, then the number of entries in the page table is  
(A) 2048 B (B) 256 B  
(C) 512 B (D) 1 MB
11. Consider a paging system with the page table stored in memory. If a memory reference takes 200 ns, how long does a paged memory reference take?  
(A) 100 ns (B) 200 ns  
(C) 300 ns (D) 400 ns

12. Consider a logical address space of eight pages of 1024 words each mapped onto a physical memory of 32 frames. How many bits are there in the logical address and in the physical address?  
(A) 10, 18 (B) 13, 18  
(C) 13, 15 (D) 10, 5
13. For a paged system, TLB hit ratio is 0.9. Let the RAM access time ' $t$ ' be 20 ns and the TLB access time ' $T$ ' be 100 ns. Then effective memory access (with TLB) will be  
(A) 120 ns (B) 200 ns  
(C) 130 ns (D) 150 ns
14. Assume that a user program is 100 K words and secondary storage device is a fixed hard disk with an average latency of 8 ms and a transfer rate of 2,50,000 words/second. Then find the swap time of a transfer of 100 K words to or from memory.  
(A) 816 ms (B) 408 ms  
(C) 204 ms (D) 8 ms
15. Consider the following segment table:

| Segment | Limit | Base |
|---------|-------|------|
| 0       | 1000  | 1400 |
| 1       | 400   | 6300 |
| 2       | 400   | 4300 |
| 3       | 1100  | 3200 |
| 4       | 1000  | 4700 |

The physical address for a logical address which is in segment 2 with offset 253 is

- (A) 4553 (B) 6353  
(C) 6253 (D) 4453
16. Consider a process of size 2 MB. If the page size is 0.5 KB, what is the size of the page table (assuming that each page is mapped by a 32-bit size page table entry)?  
(A) 8 KB (B) 16 KB  
(C) 24 KB (D) 32 MB
17. A CPU generates 32-bit virtual address. The page size is 2 KB. The translation look-aside buffer (TLB) which can hold 256 page table entries and is two-way set associative mapping. The number of bits in the TLB tag is  
(A) 10-bits (B) 12-bits  
(C) 14-bits (D) 15-bits
18. Assume that a total memory  $M$  is available with no partitions made yet. If Buddy system strategy is being used and a total of  $n$  partitions have been made to serve the request. The closest range of the requested size is

- (A)  $\frac{M}{2^n + 1}$  and  $\frac{M}{2^n}$  (B)  $\frac{M}{n}$  and  $\frac{M}{n-1}$   
(C)  $\frac{M}{2^n}$  and  $\frac{M}{2^{n-1}}$  (D)  $\frac{M}{n+1}$  and  $\frac{M}{n}$

19. The memory has four free blocks of sizes 2K, 6K, 20K, 4K. The request blocks are allocated according to best fit allocation method. The allocation requests are stored in queue as shown:

| Request no.   | $P_1$ | $P_2$ | $P_3$ | $P_4$ | $P_5$ | $P_6$ | $P_7$ |
|---------------|-------|-------|-------|-------|-------|-------|-------|
| Request sizes | 4 K   | 10 K  | 2 K   | 3 K   | 5 K   | 4 K   | 2 K   |
| Usage time    | 1     | 4     | 2     | 6     | 3     | 1     | 8     |

The time at which request for  $P_7$  will be completed is

- (A) 10 unit time  
(B) 14 unit time  
(C) 20 unit time  
(D) 15 unit time
20. A memory page containing a heavily used variable that was initialized very early and is in constant use is removed when \_\_\_\_\_ page replacement is used.
- (A) LRU  
(B) FIFO  
(C) LFU  
(D) Optimal

### PREVIOUS YEARS' QUESTIONS

1. A processor uses 36-bit physical addresses and 32-bit virtual addresses, with a page frame size of 4 K bytes. Each page table entry is of size 4 bytes. A three-level page table is used for virtual to physical address translation, where the virtual address is used as follows:

[2008]

- Bits 30–31 are used to index into the first level page table
- Bits 21–29 are used to index into the second level page table
- Bits 12–20 are used to index into the third level page table, and
- Bits 0–11 are used as offset within the page

The number of bits required for addressing the next level page table (or page frame) in the page table entry of the first, second and third level page tables are, respectively,

- (A) 20, 20 and 20 (B) 24, 24 and 24  
(C) 24, 24 and 20 (D) 25, 25 and 24
2. How many 32 K  $\times$  1 RAM chips are needed to provide a memory capacity of 256 K bytes? [2009]
- (A) 8 (B) 32  
(C) 64 (D) 128
3. In which one of the following page replacement policies, Belady's anomaly may occur? [2009]
- (A) FIFO (B) Optimal  
(C) LRU (D) MRU
4. The essential content(s) in each entry of a page table is/are [2009]
- (A) Virtual page number  
(B) Page frame number  
(C) Both virtual page number and page frame number  
(D) Access right information
5. A multilevel page table is preferred in comparison to a single-level page table for translating virtual address to physical address because [2009]

- (A) It reduces the memory access time to read or write a memory location.  
(B) It helps to reduce the size of page table needed to implement the virtual address space of a process.  
(C) It is required by the translation look-aside buffer.  
(D) It helps to reduce the number of page faults in page replacement algorithms.

6. A system uses FIFO policy for page replacement. It has four-page frames with no pages loaded to begin with. The system first accesses 100 distinct pages in some order and then accesses the same 100 pages but now in the reverse order. How many page faults will occur? [2010]

- (A) 196 (B) 192  
(C) 197 (D) 195

7. Let the page fault service time be 10ms in a computer with average memory access time being 20 ns. If one page fault is generated for every  $10^6$  memory accesses, what is the effective access time for the memory? [2011]

- (A) 21 ns (B) 30 ns  
(C) 23 ns (D) 35 ns

8. Consider the virtual page reference string

1, 2, 3, 2, 4, 1, 3, 2, 4, 1

on a demand paged virtual memory system running on a computer system that has main memory size of three-page frames which are initially empty. Let LRU, FIFO and OPTIMAL denote the number of page faults under the corresponding page replacement policy. Then [2012]

- (A) OPTIMAL < LRU < FIFO  
(B) OPTIMAL < FIFO < LRU  
(C) OPTIMAL = LRU  
(D) OPTIMAL = FIFO

9. A RAM chip has a capacity of 1024 words of 8 bits each ( $1 \text{ K} \times 8$ ). The number of  $2 \times 4$  decoders with

enable line needed to construct a  $16\text{ K} \times 16\text{ RAM}$  from  $1\text{ K} \times 8\text{ RAM}$  is [2013]

- (A) 4 (B) 5  
(C) 6 (D) 7

**Common data for Questions 10 and 11:** A computer uses 46-bit virtual address, 32-bit physical address, and a three-level paged page table organization. The page table base register stores the base address of the first-level table ( $T_1$ ), which occupies exactly one page. Each entry of  $T_1$  stores the base address of a page of the second-level table ( $T_2$ ). Each entry of  $T_2$  stores the base address of a page of the third-level table ( $T_3$ ). Each entry of  $T_3$  stores a page table entry (PTE). The PTE is 32-bits in size. The processor used in the computer has a 1 MB 16-way set associative virtually indexed physically tagged cache. The cache block size is 64 bytes.

10. What is the size of a page in KB in this computer? [2013]

- (A) 2 (B) 4  
(C) 8 (D) 16

11. What is the minimum number of page colours needed to guarantee that no two synonyms map to different sets in the processor cache of this computer? [2013]

- (A) 2 (B) 4  
(C) 8 (D) 16

12. Assume that there are three page frames which are initially empty. If the page reference string is 1, 2, 3, 4, 2, 1, 5, 3, 2, 4, 6, the number of page faults using the *optimal replacement policy* is \_\_\_\_\_. [2014]

13. A computer has 20 physical page frames which contain pages numbered 101 through 120. Now a program accesses the pages numbered 1, 2, ... 100 in that order, and repeats the access sequence THRICE. Which one of the following page replacement policies experiences the same number of page faults as the optimal page replacement policy for this program? [2014]

- (A) Least-recently used (B) First-in-first-out  
(C) Last-in-first-out (D) Most-recently-used

14. A system uses three page frames for storing process pages in main memory. It uses the Least Recently Used (LRU) page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below? [2014]

4, 7, 6, 1, 7, 6, 1, 2, 7, 2

15. Consider a paging hardware with a TLB. Assume that the entire page table and all the pages are in the physical memory. It takes 10 milliseconds to search the TLB and 80 milliseconds to access the physical

memory. If the TLB hit ratio is 0.6, the effective memory access time (in milliseconds) is \_\_\_\_\_. [2014]

16. Consider a system with byte-addressable memory, 32-bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each. The size of the page table in the system in megabytes is \_\_\_\_\_. [2015]

17. Consider a main memory with five page frames and the following sequence of page references: 3, 8, 2, 3, 9, 1, 6, 3, 8, 9, 3, 6, 2, 1, 3. Which one of the following is true with respect to page replacement policies First In First Out (FIFO) and Least Recently Used (LRU)? [2015]

- (A) Both incur the same number of page faults  
(B) FIFO incurs 2 more page faults than LRU  
(C) LRU incurs 2 more pages faults than FIFO  
(D) FIFO incurs 1 more page faults than LRU

18. Consider six memory partitions of sizes 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB, where KB refers to kilobyte. These partitions need to be allotted to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order. If the best fit algorithm is used, which partitions are NOT allotted to any process? [2015]

- (A) 200 KB and 300 KB  
(B) 200 KB and 250 KB  
(C) 250 KB and 300 KB  
(D) 300 KB and 400 KB

19. A computer system implements 8 kilobyte pages and a 32-bit physical address space. Each page table entry contains a valid bit, a dirty bit, three permission bits, and the translation. If the maximum size of the page table of a process is 24 megabytes, the length of the virtual address supported by the system is \_\_\_\_\_ bits. [2015]

20. Consider the following two C code segments.  $Y$  and  $X$  are one and two dimensional arrays of size  $n$  and  $n \times n$  respectively, where  $2 \leq n \leq 10$ . Assume that in both code segments, elements of  $Y$  are initialized to 0 and each element  $X[i][j]$  of array  $X$  is initialized to  $i + j$ . Further assume that when stored in main memory all elements of  $X$  are in same main memory page frame.

Code segment 1: [2015]

```
//initialize elements of Y to 0
//initialize elements X[i][j] of X to i+j
for (i = 0; i < n; i++)
 Y[i] += X[0][i];
```

Code Segment 2:

```
//initialize elements of Y to 0
```

```
//initialize elements X[i] [j] of X
to i + j
for (i = 0; i < n; i++)
 Y[i] += X[i] [0];
```

Which of the following statements is/are correct?

- $S_1$ : Final contents of array  $Y$  will be same in both code segments
- $S_2$ : Elements of array  $X$  accessed inside the for loop shown in code segment 1 are contiguous in main memory
- $S_3$ : Elements of array  $X$  accessed inside the for loop shown in code segment 2 are contiguous in main memory.
- (A) Only  $S_2$  is correct  
 (B) Only  $S_3$  is correct  
 (C) Only  $S_1$  and  $S_2$  are correct  
 (D) Only  $S_1$  and  $S_3$  are correct
21. Consider a computer system with 40-bit virtual addressing and page size of sixteen kilobytes. If the computer system has a one-level page table per process and each page table entry requires 48 bits, then the size of the per-process table is \_\_\_\_\_ megabytes. [2016]
22. Consider a computer system with ten physical page frames. The system is provided with an access sequence  $(a_1, a_2, \dots, a_{20}, a_1, a_2, \dots, a_{20})$ , where each  $a_i$  is a distinct virtual page number. The difference in the number of page faults between the last-in-first-out page replacement policy and the optimal page replacement policy is \_\_\_\_\_. [2016]
23. In which one of the following page replacement algorithms it is possible for the page fault rate to increase even when the number of allocated frames increases? [2016]
- (A) **LRU** (Least Recently Used)  
 (B) **OPT** (Optimal Page Replacement)  
 (C) **MRU** (Most Recently Used)  
 (D) **FIFO** (First In First Out)
24. Recall that Belady's anomaly is that the page-fault rate may *increase* as the number of allocated frames increases. Now, consider the following statements:  
 $S_1$ : *Random page replacement* algorithm (where a page chosen at random is replaced) suffers from Belady's anomaly  
 $S_2$ : *LRU page replacement* algorithm suffers from Belady's anomaly
- Which of the following is CORRECT? [2017]
- (A)  $S_1$  is true,  $S_2$  is true  
 (B)  $S_1$  is true,  $S_2$  is false  
 (C)  $S_1$  is false,  $S_2$  is true  
 (D)  $S_1$  is false,  $S_2$  is false
25. Consider a process executing on an operating system that uses demand paging. The average time for a memory access in the system is  $M$  units if the corresponding memory page is available in memory and  $D$  units if the memory access causes a page fault. It has been experimentally measured that the average time taken for a memory access in the process is  $X$  units.
- Which one of the following is the correct expression for the page fault rate experienced by the process? [2018]
- (A)  $(D - M)/(X - M)$       (B)  $(X - M)/(D - M)$   
 (C)  $(D - X)/(D - M)$       (D)  $(X - M)/(D - X)$

## ANSWER KEYS

### EXERCISES

#### Practice Problems 1

- |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 1. B  | 2. B  | 3. D  | 4. C  | 5. C  | 6. C  | 7. B  | 8. A  | 9. A  | 10. C |
| 11. C | 12. B | 13. C | 14. B | 15. B | 16. C | 17. B | 18. C | 19. A | 20. C |

#### Practice Problems 2

- |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 1. B  | 2. B  | 3. B  | 4. C  | 5. D  | 6. B  | 7. A  | 8. B  | 9. D  | 10. C |
| 11. D | 12. B | 13. C | 14. A | 15. A | 16. B | 17. C | 18. C | 19. A | 20. B |

#### Previous Years' Questions

- |         |       |       |       |         |       |       |       |        |       |
|---------|-------|-------|-------|---------|-------|-------|-------|--------|-------|
| 1. D    | 2. C  | 3. A  | 4. B  | 5. B    | 6. A  | 7. B  | 8. B  | 9. B   | 10. C |
| 11. C   | 12. 7 | 13. D | 14. 6 | 15. 122 | 16. 4 | 17. A | 18. A | 19. 36 | 20. C |
| 21. 384 | 22. 1 | 23. D | 24. B | 25. B   |       |       |       |        |       |

# Chapter 5

## File Systems, I/O Systems, Protection and Security

### LEARNING OBJECTIVES

- 🔗 File systems
- 🔗 File management systems
- 🔗 File system architecture
- 🔗 Device drivers
- 🔗 Basic input/output supervisor
- 🔗 Logical input/output
- 🔗 Access methods

### FILE SYSTEMS

The file system consists of two distinct parts:

1. Collection of files
2. Directory structure

**File** A file is a named collection of related information that is recorded on secondary storage. The files must have

1. Long-term existence
2. Sharable between processes
3. Structure

**File attributes** A typical file attributes are

1. Name
2. Identifier
3. Type
4. Location
5. Size
6. Protection
7. Time, date and user identification

**File operations** The operations that are applied on files are

1. Creation
2. Deletion
3. Closing
4. Reading
5. Writing

**File types** A common technique for implementing file types is to include the type as part of the file name. The name is split into two parts:

1. A name
2. An extension (usually separated by a period character)

The type of a file may be

1. Executable (exe, com, bin)
2. Object (obj, o)
3. Source code (c, cc, java)
4. Batch (bat, sh)
5. Text (txt, doc)
6. Word processor (wp, text, doc)
7. Library (lib)
8. Print or view (ps, pdf, jpg)
9. Archive (zip, tar)
10. Multimedia (mpeg, mov, rm)

**File structure** The four common terms of file systems are

1. Field
  - Basic element of data
  - Contains a single value
  - Has a particular length and data type
2. Record
  - It is a collection of related fields
  - It is treated as a unit
3. File
  - It is a collection of similar records
  - Treated as a single entity
  - Has file names
  - Access to file may be restricted or unrestricted
4. Database
  - Collection of related data.
  - Relationship exists among elements.

## File Management Systems

It is a set of system software that provides services to users and applications in the use of files. Objectives of file management systems are as follows:

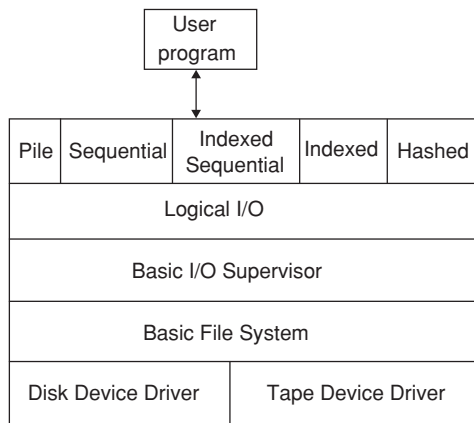
1. To meet the data management needs and requirements of the user including storage of data.
2. To guarantee, that the data in the file are valid.
3. To optimize performance (i.e., throughput, response time).
4. To provide Input/Output (I/O) support for a variety of storage device types.
5. To minimize or eliminate the potential for lost or destroyed data.
6. To provide a standardized set of I/O interface routines.
7. To provide I/O support for multiple users.

The minimal set of requirements from user's point of view for an interactive, general purpose, file system are as follows:

1. Ability to create, delete, read, write and modify files.
2. Controlled access to other users files.
3. Control the type of accesses to files.
4. Restructure the user's files.
5. Able to move data between files.
6. Ability to back up and recover user's files.
7. Able to access a file by name.

## FILE SYSTEM ARCHITECTURE

The file system architecture is shown below:



## Device Drivers

1. It is at the lower part.
2. Communicates directly with peripheral devices.
3. Initiative to start I/O operations on a device.
4. Processes the completion of an I/O request.

## Basic File System

1. Mainly concerned with I/O.
2. Exchanges blocks of data.
3. Deals with placement of blocks.
4. Deals with buffering blocks in main memory.

## Basic I/O Supervisor

1. Responsible for file initiation and termination.
2. Maintain control structures.
3. Selects the device while file I/O is to be performed.
4. Deals with scheduling access to optimize performance.
5. Part of the operating system (OS).

## Logical I/O

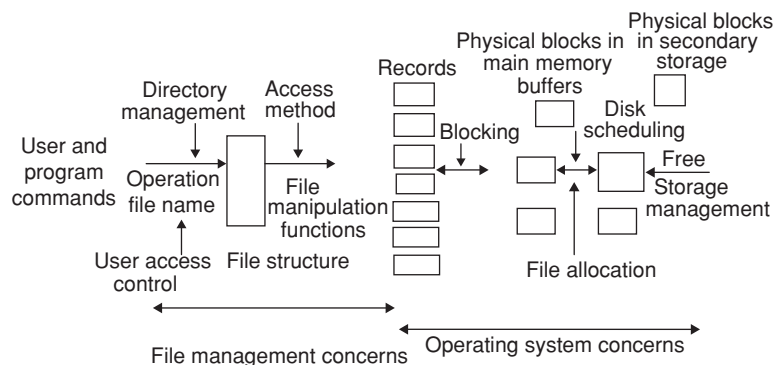
1. Enables users and application to access records.
2. Provides general purpose record I/O capability.
3. Maintains basic data about file.

## Access Methods

1. They reflect different file structures.
2. Provide different ways to access and process data.
3. Access methods are as follows:
  - Sequential: read next, write next
  - Direct: read block n, write block n

## File Management Functions

The functions of a file system is shown below:



## File Organization and Access

File organization refers to the logical structuring of the records as determined by the way in which they are accessed. We choose a particular file organization based on

1. Short access time
2. Ease of update
3. Economy of storage
4. Simple maintenance
5. Reliability

We will discuss five types of file organizations:

1. The pile
2. The sequential file
3. The indexed sequential file
4. The indexed file
5. The direct or hashed file

**The pile** The pile organization is shown in Figure 1:

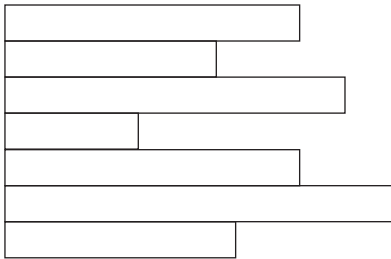


Figure 1 Pile file.

1. Data are collected in order they arrive.
2. The aim is to accumulate mass of data and save.
3. Records may have different fields.
4. There is no structure.
5. Record are accessed by exhaustive search.
6. There can be variable length records.
7. This type of files are encountered when data are collected and stored prior to processing or when data are not easy to organize.

**The sequential file** The sequential file is shown in Figure 2.

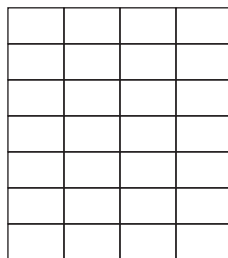


Figure 2 Sequential file.

1. Fixed format used for records.
2. Records are of same length.
3. All fields are of the same order and length.
4. One field is the key field. It uniquely identifies the record. Records are stored in a key sequence.

5. New records are placed in a log file or transaction file.
6. Batch update is performed to merge the log file with the master file.
7. Used in batch applications.
8. Not suitable for interactive applications.

### Indexed sequential file

1. Index provides a look up capability to quickly reach the vicinity of the desired record.
2. It contains key field and a pointer to the main file.
3. Index is searched to find highest key value that is equal to or precedes the desired key value.
4. Search continues in the main file at the location indicated by the pointer.
5. New records are added to an overflow file.
6. Record in main file that precedes it is updated to contain a pointer to the new record.
7. The overflow is merged with main file during a batch update.
8. Multiple indexes for the same key can be set up to increase efficiency.

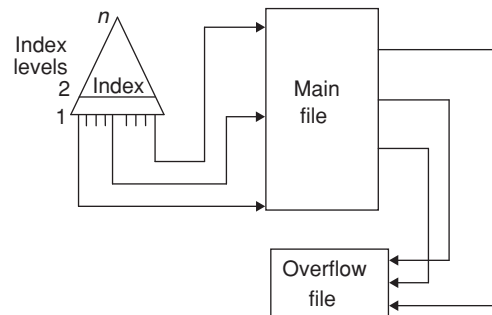
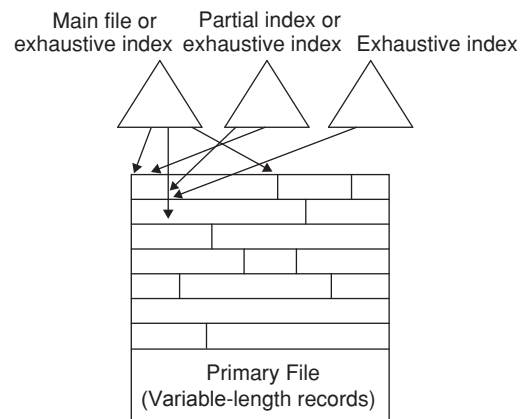


Figure 3 Indexed file.

9. Key field required for each record.
10. It uses multiple indexes for different key fields.
11. It may contain exhaustive index that contains one entry for every record in the main file or partial index.

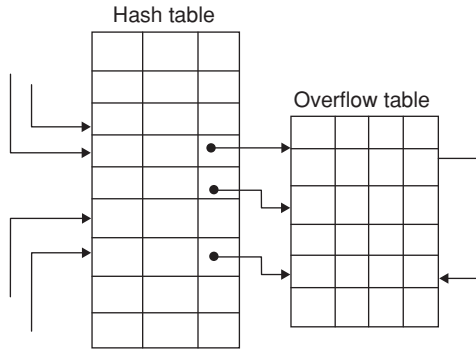


12. Used in the applications where timeliness of information is critical and where data are rarely processed exhaustively.

### Direct or Hashed file

1. This file has the capability to access any block of a known address.
2. Key field required in each record.
3. No concept of sequential ordering.

Figure 4 shows hashed file organization:



**Figure 4** Hashed file organization.

## FILE DIRECTORIES

The collection of files is a file directory.

### Contents

1. Contain information about files, such as attributes, location, and ownership.
2. Itself a file owned by the OS.
3. It provides mapping between file names and the file themselves.

### Directory Structure

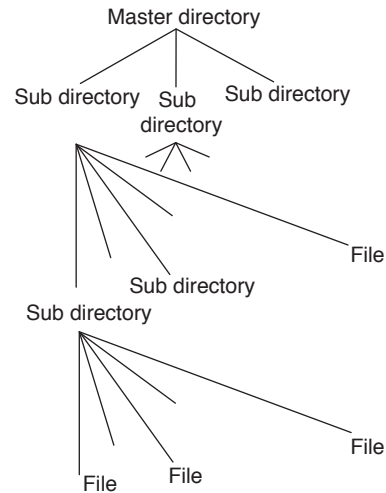
1. It consists of list of entries, one for each file.
2. Sequential file with the name of the file serving as the key.
3. Provides no help in organizing the files.
4. Not scalable (same name cannot be used for two different files).
5. As directory grows in size, searching is too time consuming.

## Two-level Directory Structure

1. Contains one directory for each user and a master directory.
2. The master directory contains entry for each user
3. User directory is a simple list of files for that user.
4. File naming conflict is solved.

## Hierarchical or Tree-structured Directory

1. Contains master directory with user directory underneath it.
2. Each user directory may have subdirectories and files as entries.



3. Files can be located by following a path from the root, or master directory down various branches which is called *pathname* for the file.
4. Current directory is called the *working directory*.
5. Files are referenced relative to the working directory.

**Naming** The use of a tree-structured directory minimizes the difficulty in assigning unique names. Any file in the system can be located by following a path from the root or master directory down various braches until the file is reached.

**Path name** The series of directory names, culminating in the file name itself, constitutes a path name for the file.

**Example:** Time/Gate/Exam/OS

The slash is used to delimit names in the sequence. The name of master directory is implicit, because all paths starts at that directory. Files can also be referenced from the working directory.

**File sharing** It has got two issues, such as:

1. Access rights
2. Management of simultaneous access

**Access rights** Users or groups of users are granted certain access rights to a file. A wide range of access rights has been used. The access rights may be

1. None
2. Knowledge
3. Execution
4. Reading
5. Appending
6. Updating
7. Changing protection
8. Deletion

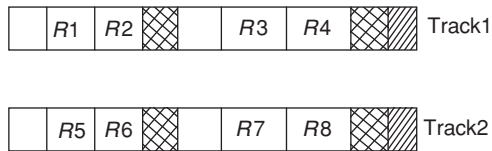
These access rights can be specified to specific users or user groups or all users.

**Record Blocking** For I/O to be performed, records must be organized as blocks. Given the size of a block, there are three methods of blocking that can be used:

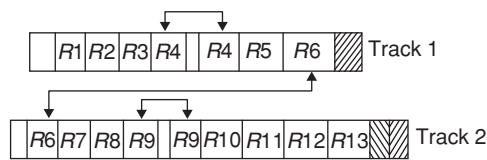
1. Fixed blocking
2. Variable length spanned blocking
3. Variable length unspanned blocking.

**Fixed blocking**

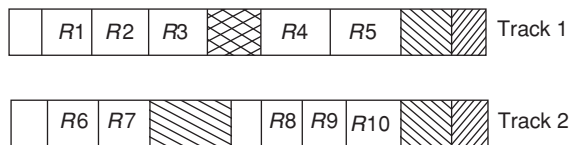
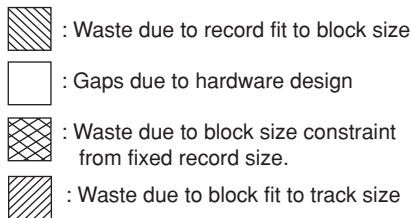
1. Fixed-length records (Figure 5) are used and an integral number of records are stored in a block.
2. Possibility of internal fragmentation.
3. Used for sequential files.

**Figure 5** Fixed blocking.

**Variable length spanned blocking** Variable length records are used and are packed into blocks with no unused space. Some records can span two blocks. These do not limit the size of records.

**Figure 6** Variable blocking: spanned.

**Variable length unspanned blocking** Variable length records (Figure 7) are used, but spanning is not employed. There is wasted space in most blocks and limits record size.

**Figure 7** Variable blocking: unspanned.**SECONDARY STORAGE MANAGEMENT**

1. Space (or blocks) must be allocated to files on disk.
2. Need to keep track of the space available (free blocks) for allocation to files.
3. Preallocation of blocks to files can be used to allocate space for files. For this, it needs to know the maximum size of the file at the time of creation.

**File Allocation****Preallocation Versus Dynamic Allocation**

1. A preallocation policy requires that the maximum size of a file be declared at the time of the file creation request. For many applications, it is difficult to estimate the file size.  
It is better to use dynamic allocation, which allocates space to a file in portions as needed.

**Portion size** The portion size which is allocated to a file may be

1. Variable, large contiguous portions.
2. Blocks

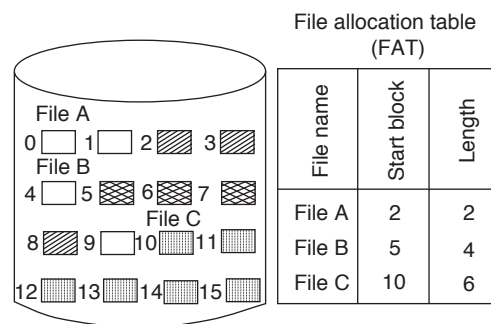
Some strategies for dealing with fragmentation of free space are as follows:

1. *First-fit*: Choose the first unused contiguous group of blocks of sufficient size.
2. *Best-fit*: Choose the smallest unused group that is of sufficient size.
3. *Nearest-fit*: Choose the unused group of sufficient size that is closest to the previous allocation.

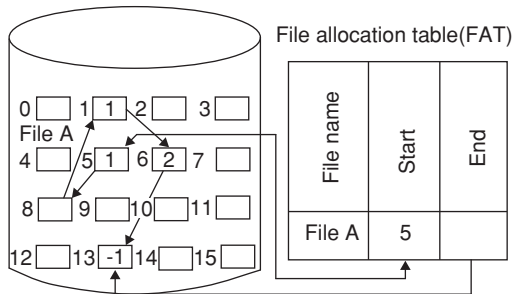
**File allocation methods** It has three methods as follows:

1. Contiguous allocation
2. Chained allocation
3. Indexed allocation

**Contiguous allocation** Here, a single set of blocks is allocated to a file at the time of creation. Only a single entry in the file allocation table is created consisting of starting block and length of the file. It exhibits external fragmentation and performs compaction.

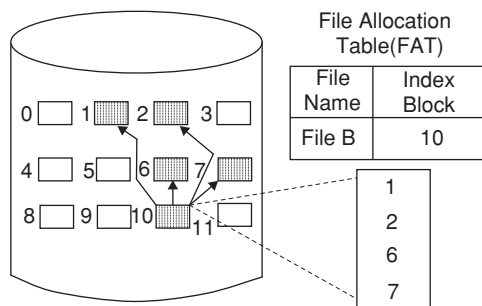


**Linked or chained allocation** The allocation is done on basis of individual block. Each block contains a pointer to the next block in the chain. Only single entry is created in the file allocation table consisting of starting block and length of file. There occurs no external fragmentation and it is best for sequential files. There is no accommodation of principle of locality. If block size is  $n$ , then only  $n - 1$  units of data are stored and 1 unit stores the link information.

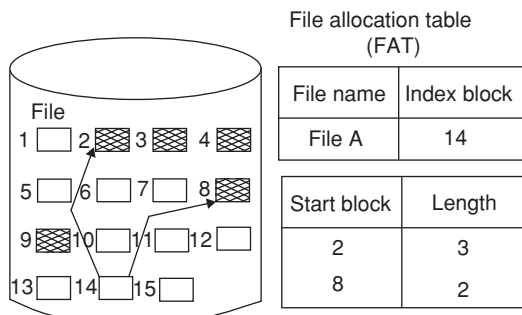


**Indexed file allocation** The file allocation table contains a separate one level index for each file. The index has one entry for each portion allocated to the file. The file allocation table contains block number for the index. If a file requires  $n$  blocks, then  $n + 1$  blocks are used, where the first block contains index information (pointers to data blocks).

### Index Allocation with Block Pointers



### Indexed Allocation with Variable length Portion



**Example 1:** A direct access of file has fixed size 50 byte records. Assuming the first record is record 1, the first byte of record 10 will be at what logical location?

**Solution:**

Total records =  $50 \times 10 = 500$

First record is record 1. This record is already read.

Logical location of first byte =  $500 - 50 = 450$

The correct logical location =  $450 + 1 = 451$ .

**Example 2:** A sequential access file has fixed-size 32-byte records. Assuming that the first record is record 0, the first byte of record 20 will be at what location?

**Solution:** Since the first record is record 0, the first byte of record 20 will be at logical location =  $32 \times 20 = 640$

## FREE SPACE MANAGEMENT

In addition to file allocation table, disk allocation table is also required to know what blocks on the disk are available. Some of the free space management techniques are as follows:

1. Bit tables
2. Chained free portions
3. Indexing
4. Free block list

**Bit tables** This method uses a vector containing, one bit for each block on the disk. Each entry of a '0' corresponds to a free block and each '1' corresponds to a block in use.

### Advantage

1. Easy to find one or a contiguous group of free blocks.
2. Smaller in size.

The amount of memory required for a block bitmap will be

$$\frac{\text{Disk size in (bytes)}}{8 \times \text{file system block size}}$$

**Chained free portion** The free portions may be chained together by using a pointer and length value in each free portion. This method has negligible space overhead. This method is suitable for all file allocation methods. The disk will become quite fragmented, after some use. It is slower for individual block file creation and also for deletion.

**Indexing** It treats the free space as a file and uses an index table (same as in file allocation). The index should be on the basis of variable size portions rather than blocks.

**Free block list** Here, each block is assigned a number sequentially and the list of the numbers of all free blocks is maintained in a reserved portion of the disk.

## Volumes

It is a collection of addressable sectors in a secondary memory that an OS or application can use for data storage. The sectors in a volume need not be consecutive on a physical storage device. (a single disk equals one volume).

## UNIX FILE MANAGEMENT

### I-nodes (Index Node)

UNIX files are administered by the OS by means of i-node. An i-node (index node) is a control structure that contains the key information needed by the OS for a particular file.

The attributes of the file as well as its permissions and other control information are stored in the i-node. The

exact i-node structure varies from UNIX implementation to another. The FreeBSD i-node structure is shown in Figure 8.

File Allocation

- 1. It is done on a block basis.
- 2. Allocation is dynamic.

- 3. The blocks of a file on disk are not necessarily contiguous.
- 4. An indexed method is used to keep track of each file, with i-node includes a number of direct pointers and three indirect pointers.

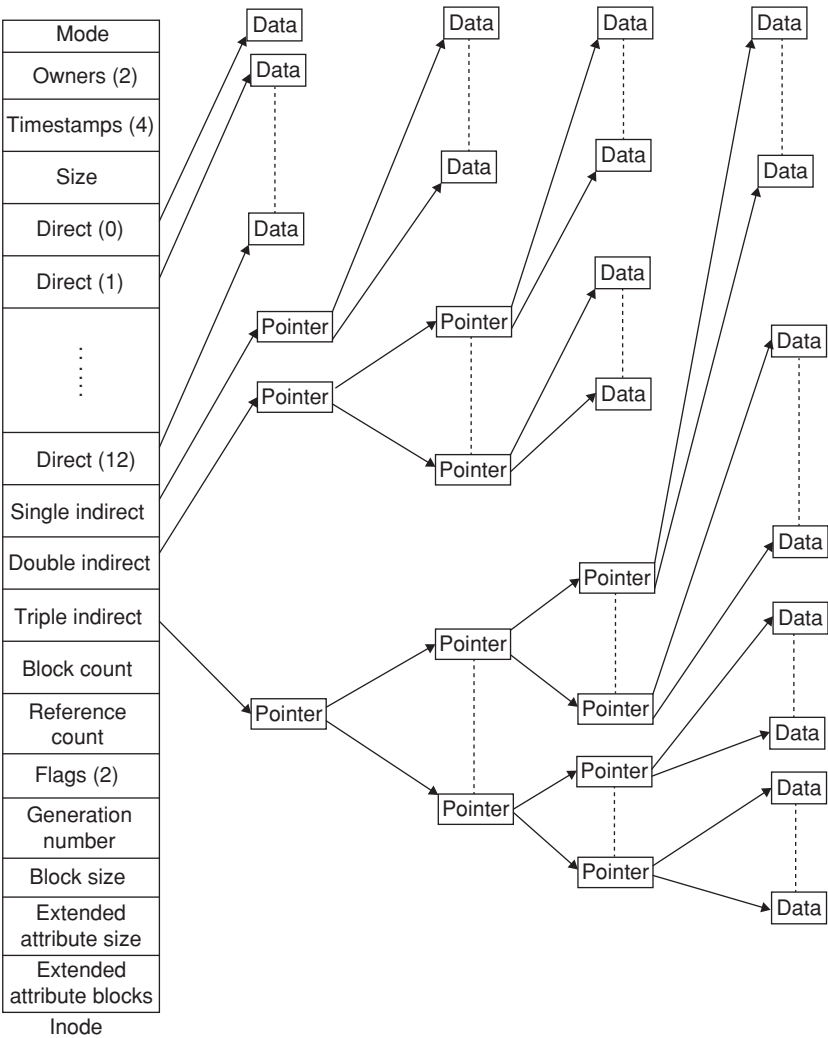


Figure 8 Structure of free BSD i-node and file.

- 5. The free BSD i-node includes 120 bytes of address information that is organized as fifteen 64-bit addresses.
- 6. The first 12 addresses point to the first 12 data blocks of the file.
- 7. If the file requires more than 12 data blocks, one or more levels of indirection are used as follows:
  - The thirteenth address in the i-node points to a block on disk that contains the next portion of the index. This is referred to as the *single indirect block*.
  - If the file contains more blocks, the fourteenth address in the i-node points to a double indirect block. Each block consists of single indirect blocks, each of which contains pointers to file blocks.

- If the file contains still more blocks, the fifteenth address in the i-node points to a triple indirect block that is a third level of indexing. This block points to additional double indirect blocks.

The capacity of FreeBSD file with 4 kB block size is shown below:

| Level           | Number of Blocks   | Number of Bytes |
|-----------------|--------------------|-----------------|
| Direct          | 12                 | 48 K            |
| Single indirect | 512                | 2 M             |
| Double indirect | 512 × 512 = 256 K  | 1 G             |
| Triple indirect | 512 × 256K = 128 M | 512 G           |

The total number of data blocks in a file depends on the capacity of the fixed-size blocks in the system. In FreeBSD, the minimum block size is 4 kB, and each block can hold a total of 512 block addresses. Thus, the maximum size of a file with this block size is over 500 GB.

## Windows NT File System

The windows NT file system provides a combination of reliability, compatibility and performance, which are not available in the FAT file system.

1. It will quickly perform standard file operations, such as write, read and search.
2. It also performs file-system recovery on very large hard disks.
3. NTFS file system formatting on a volume results in the creation of several system files and the master file table (MST), which contains information about all the files and folders on the NTFS (Figure 9).

|                       |                   |              |           |
|-----------------------|-------------------|--------------|-----------|
| Partition boot sector | Master file table | System files | File area |
|-----------------------|-------------------|--------------|-----------|

Figure 9 NTFS after formatting.

4. It includes security features required for high-end personal computers and file servers.
5. It supports data access control and ownership privileges.
6. Folders shared on a Windows NT are assigned specific permissions.
7. It allows users to assign permissions to individual files.

## I/O SYSTEMS

### Design Objectives of OS

Two objectives of OS for I/O systems are as follows:

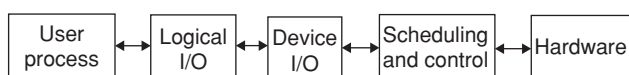
1. Efficiency
2. Generality

### Logical Structure of the I/O Function

The three most important logical structures are as follows:

1. Local peripheral devices
2. Communication ports
3. File system

### Local Peripheral Device



### Logical I/O

Manages general I/O functions on behalf of user processes, allowing them to deal with the device in terms of a device identifier and simple commands, such as open, close, read, write.

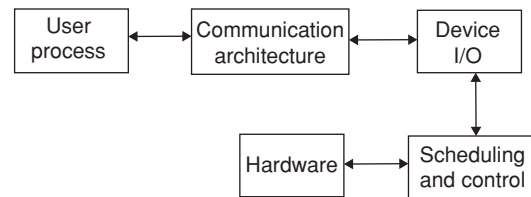
## Device I/O

The requested operations and data are converted into appropriate sequences of I/O instructions, channel commands and controller orders.

## Scheduling and Control

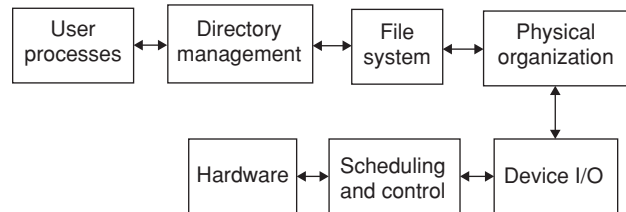
The actual queuing and scheduling of I/O operations occurs at this layer as well as the control of the operations. Interrupts are handled. I/O status is collected and reported.

## Communication Port



Here, the communication architecture may itself consist of a number of layers.

## File System



**Directory management** Here, the symbolic file names are converted to identifiers that either reference the file directly or indirectly through a file descriptor or index table.

**File system** This layer deals with the logical structure of files and with the operations that can be specified by users, such as open, close, read, write.

**Physical organization** Allocation of secondary storage space and main storage buffers is generally treated here.

**I/O buffering** In buffering, we perform input transfers in advance of requests being made and perform output transfers sometime after the request is made. We will discuss

1. Single buffering
2. Double buffering
3. Circular buffering

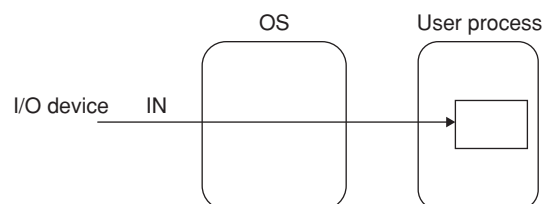


Figure 10 No buffering.

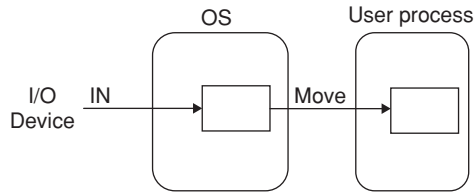


Figure 11 Single buffering.

- When a user process issues an I/O request, the OS assigns a buffer in the system portion of main memory to the operation.
- Provides a speed up compared to the lack of system buffering.
- For block oriented devices, input transfers are made to the system buffer. When the transfer is complete, the process moves the block into user space and immediately requests another block. This is same for block-oriented output.
- Suppose that  $T$  is the time required to input one block and that  $C$  is the computation time that intervenes between input requests. Without buffering, the execution time per block =  $T + C$ . With a single buffer, the execution time per block =  $\max[C, T] + M$ , where  $M$  = Time required to move the data from the system buffer to user memory.
- For stream-oriented I/O, the single buffering scheme can be used in a line-get-a-time fashion or a byte-at-a-time fashion.

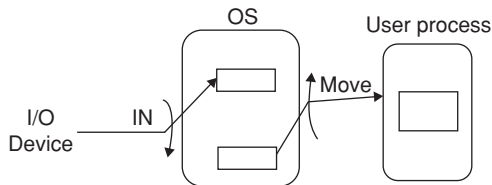


Figure 12 Double buffer

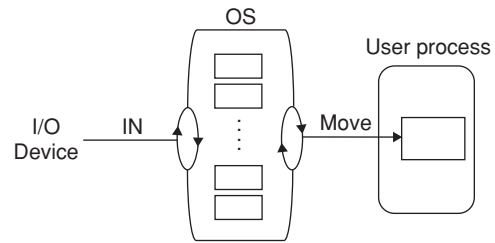
- An improvement over single buffer.
- A process now transfers data to one buffer while the OS empties the other. This is known as *double buffering* or *buffer swapping*.
- For block-oriented transfer, the estimated execution time is  $\max[C, T]$ .
- The stream-oriented transfer may be line-at-a-time or byte-at-a-time.

### Automatic and Explicit Buffering

An indefinite length queue is provided in automatic buffering. Sender never blocks to copy a message. No specifications are present for providing automatic buffering. One method is to reserve large amount of memory. In this method, most of the memory is wasted.

The size of the queue is provided in explicit buffering. Sender will block if the requested space is more than available. In this scheme, memory wastage is less likely to happen.

### Circular Buffer



- Here, more than two buffers are used. Each individual buffer is one unit of circular buffer.

### Kernal I/O Subsystem

Kernel I/O provides I/O-related services. It is built on the hardware and device driver infrastructure.

One of the responsibilities of Kernal I/O subsystem is to protect itself from the erroneous process and malice users.

Services provided by the I/O subsystem are as follows:

- |               |                       |
|---------------|-----------------------|
| 1. Scheduling | 4. Spooling           |
| 2. Buffering  | 5. Device reservation |
| 3. Caching    | 6. Error handling     |

### DISK SCHEDULING

#### Hard Disk Performance Parameters (Terminologies)

**Seek time** It is defined as the time required to move the disk arm to the required track. It consists of

- Initial start-up time
- The time taken to traverse the tracks that has to be crossed once the access arm is up to speed.

$$\text{Seek time, } T_s = m \times n + S$$

where  $T_s$  = Estimated seek time

$n$  = Number of track traversed

$m$  = Constant that depends on the disk drive

$S$  = Start-up time

**Rotational delay** Time required to reach the desired sector by read/write head. Rotational speed ranges from 5400 to 10,000 rpm.

- Floppy disks typically rotate between 300 and 600 rpm.
- For 1,00,000 rpm, the average rotational delay will be 3 ms.

**Transfer time** The transfer time to or from the disk depends on the speed of the disk.

$$T = \frac{b}{rN}$$

where,  $T$  = Transfer time

$b$  = Number of bytes to be transferred

$N$  = Number of bytes on a track

$r$  = Rotation speed in revolutions/second

Total average access time

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

$T_s \rightarrow$  average seek time

### Hard Disk Scheduling Algorithms

Disk bandwidth and fast access time are to be considered. Bandwidth is the total number of bytes transferred divided by the total time between the first request for service and completion of the last transfer.

**FCFS (First Come First Served)** The disk in controller processes the I/O requests the order in which they arrive, thereby moving backwards and forwards across the surface of the disk to get the next requested location each time (Figure 10).

**Example 3:** A disk queue has the following requests to read tracks:

87, 170, 40, 150, 36, 72, 66, 15

Consider the disk head is initially at cylinder 60. Total head movement =  $(87 - 60) + (170 - 87) + (170 - 40) + (150 - 40) + (150 - 36) + (72 - 36) + (72 - 66) + (66 - 15) = 27 + 83 + 130 + 110 + 114 + 36 + 6 + 51 = 557$  cylinders

Average head movement =  $\frac{557}{8} = 69.6$  cylinders

#### Advantage

Improved response time as a request gets response in fair amount of time.

#### Disadvantages

1. Involves a lot of random head movements and disk rotations.
2. Throughput is not efficient.
3. Used in small systems only where I/O efficiency is not very important.

**Shortest Seek Time First (SSTF)** When a disk operation finishes, choose the request that is closer to the current head position or choose the request that has minimum seek time from the current head position.

**Example 4:** Consider the following requests: 87, 170, 40, 150, 36, 72, 66, 15. Find the average head movement for SSTF.

The initial head position is say 60. Now, closest to the head position is the request at cylinder 66. Then, the closest to 66 is 72, closest request to 72 is 87, and so on. Total head movements =  $(66 - 60) + (72 - 66) + (87 - 72) + (87 - 40) + (40 - 36) + (36 - 15) + (150 - 15) + (170 - 150) = 6 + 6 + 15 + 47 + 4 + 21 + 135 + 20 = 254$  cylinders

Average head movements =  $\frac{254}{8} = 31.75$  cylinders

#### Advantages

1. It minimizes latency
2. Better throughput than FIFO method

#### Disadvantages

1. Starvation occurs if some process has to wait for long time until its requests are satisfied.
2. SSTF services requests for those tracks which are highly localized.

**SCAN/elevator algorithm** The disk head constantly moves from the most inner cylinder to the outer cylinder and then it changes its direction back towards the centre. As the head moves, if there is a request for the current disk position then it is satisfied.

1. It is known as *elevator algorithm* because it services all the request of going up and then reaching at the top, it goes downward.
2. It needs two information:
  - Direction of head movement
  - Last position of the disk head

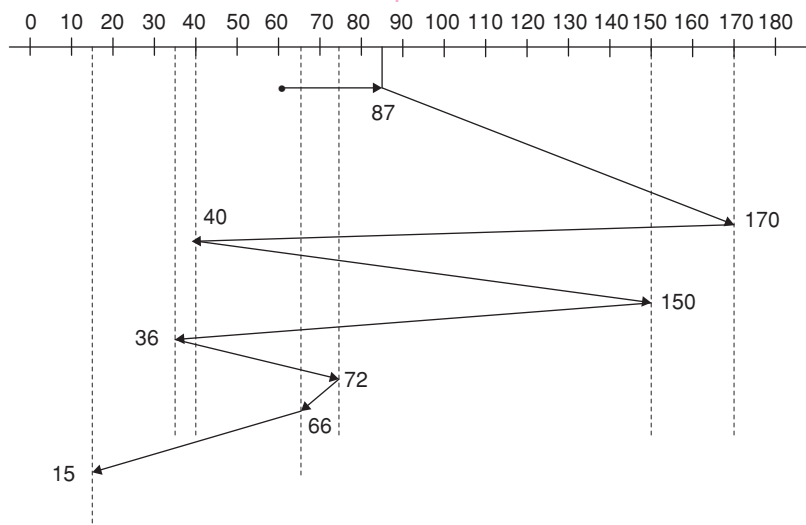


Figure 13 FCFS.

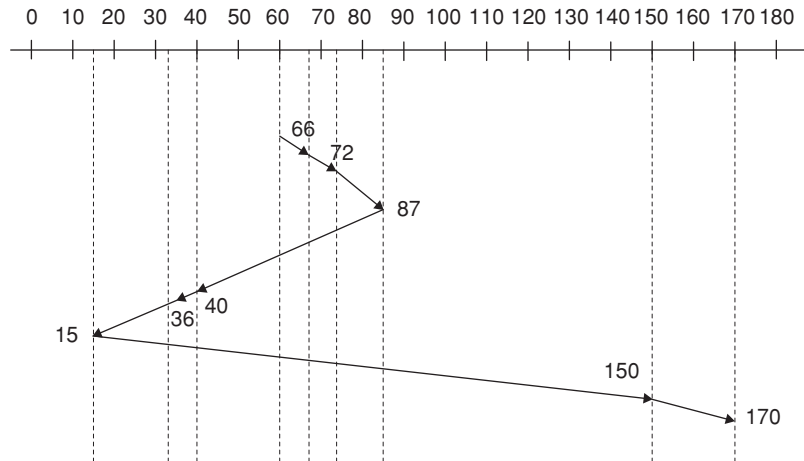


Figure 14 SSTF

**Example 5:** For the following track requests: 87, 170, 40, 150, 36, 72, 66, 15. (Initially head is at track 60 to the arm is moving outwards.

$$\begin{aligned} \text{Total head movement} &= (66 - 60) + (72 - 66) + (87 - 72) + (150 - 87) + (170 - 150) + (180 - 170) + (180 - 40) + (40 - 36) + (36 - 15) \\ &= 6 + 6 + 15 + 63 + 20 + 10 + 140 + 4 + 21 \\ &= 285 \text{ cylinders} \end{aligned}$$

$$\text{Average head movement} = \frac{285}{8} = 35.6 \text{ cylinders}$$

### Advantages

1. Throughput better than FIFO.
2. Basic for most scheduling algorithms.
3. Eliminates the discrimination.
4. No starvation.

### Disadvantages

1. Because of the continuous scanning of disk from end to end, the outer tracks are visited less often than the mid-range tracks.

2. Disk arm keeps scanning between two extremes; this may result in wear and tear of the disk assembly.
3. Certain requests arriving ahead of the arm position would get immediate service but some other requests that arrive behind the arm position will have to wait for the arm to return back.

**C-SCAN algorithm (one-way elevator algorithm)** It treats the cylinder as a circular list. The head sweeps from the innermost cylinder to the outermost cylinder, satisfying the waiting requests in order of their locations. When it reaches the outermost cylinder, it sweeps back to the innermost cylinder without satisfying any requests and then starts again.

**Example 6:** Consider the cylinders requests:

87, 170, 40, 150, 36, 72, 66, 15 Starting cylinder = 60th (arm moving outwards)

$$\begin{aligned} \text{Total head movement} &= (66 - 60) + (72 - 66) + (87 - 72) + (150 - 87) + (170 - 150) + (180 - 170) + (180 - 0) + (15 - 0) + (36 - 15) + (40 - 36) \\ &= 6 + 6 + 15 + 63 + 20 + 10 + 180 + 15 + 21 + 4 = 340 \end{aligned}$$

$$\text{Average head movement} = \frac{340}{8} = 42.5$$

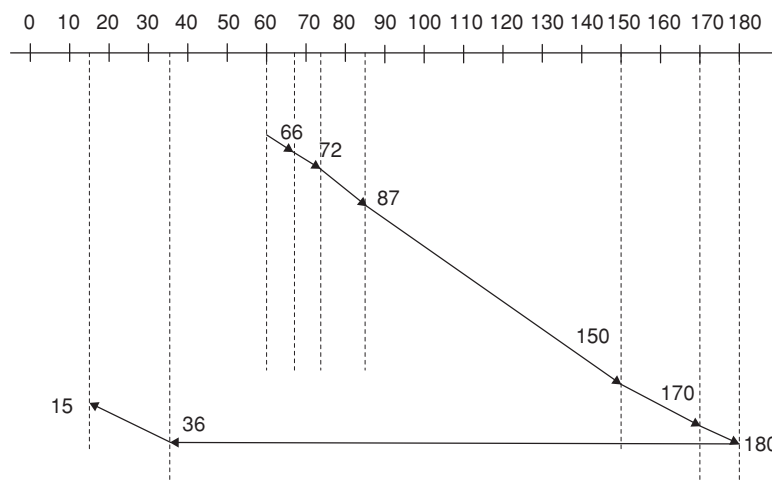


Figure 15 SSTF

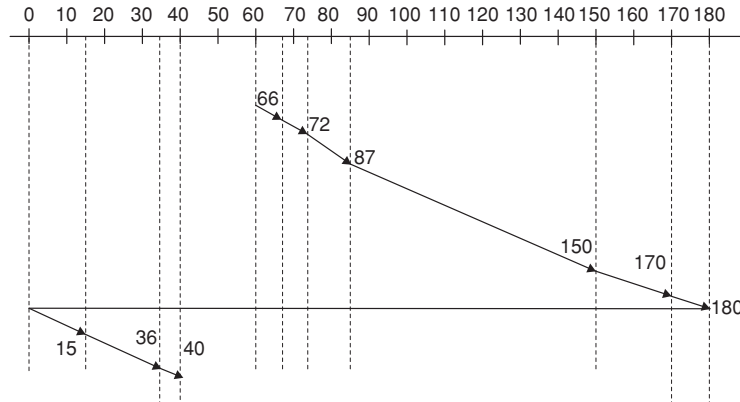


Figure 16 C-SCAN

**Advantage**

Lower service variability.

**Disadvantages**

1. An average head movement is more compared to SCAN algorithm.
2. Increase in the total seek time

**LOOK/SEEK algorithm** Look is similar to SCAN, but stops moving inwards (or) outwards when there are no more requests in that direction.

**Example 7:** Consider the following requests: 87, 170, 40, 150, 36, 72, 66, 15 initially head is at track 60 (moves outwards).

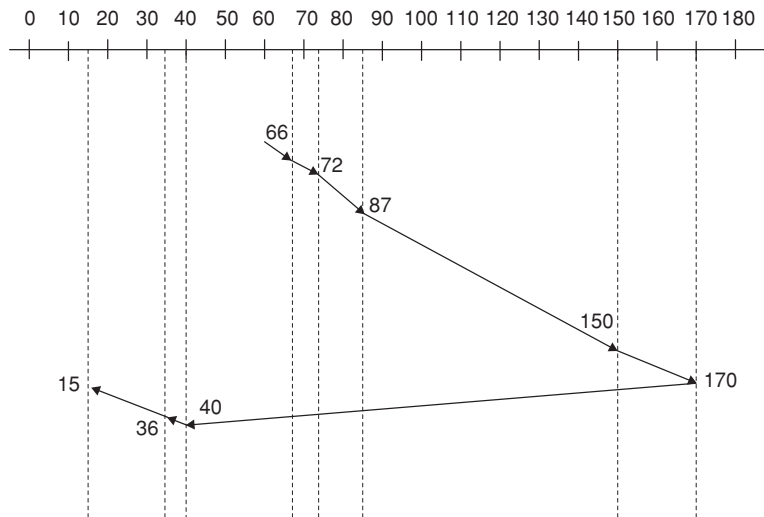


Figure 17 LOOK

$$\text{Average head movement} = \frac{275}{8} = 34.37$$

**C-LOOK/C-SEEK algorithm** The head moves inwards servicing requests until there are no more requests in that direction. Then it jumps to the outer most outstanding request.

### Magnetic Disk and Factors that Determine Access Speed

The factors that determine the access speed are rotational speed, which is measured in revolutions per minute are seek time, the time taken to read or write the particular sector of the disk.

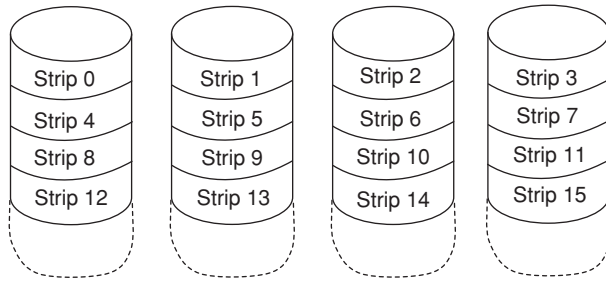
The other factors are sequential read, sequential write, random read and random write. These vary enormously, but for spinning disks one can expect 25 Mbps to 150 Mbps for

sequential read and write it is about 3 Mbps to 50 Mbps for the random read and write.

### RAID (REDUNDANT ARRAY OF INDEPENDENT DISKS)

1. It is multiple disk database design.
2. It consists of seven levels, zero through six.
3. Characteristics of RAID Levels:
  - RAID is a set of physical disk drives viewed by the OS as a single logical drive.
  - Data are distributed across the physical drives of an array in a scheme known as striping.
  - Redundant disk capacity is used to store parity information, which guarantees data recoverability in case of a disk failure.

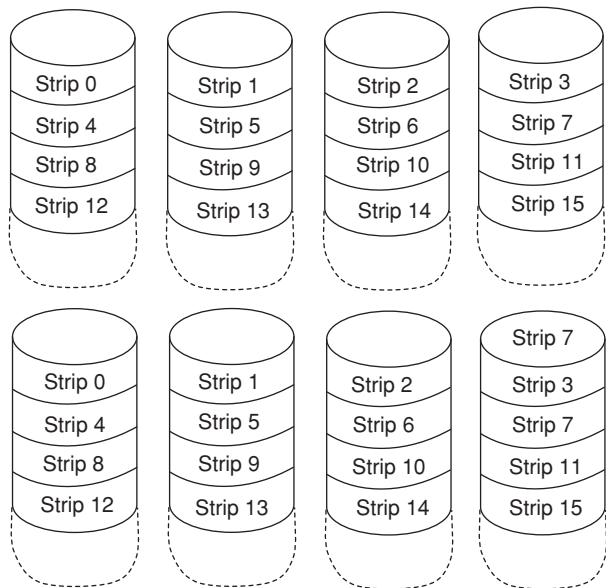
### RAID Level 0:



**Figure 18** Non-redundant (RAID0)

1. It does not include redundancy.
2.  $N$  disks are required.
3. Data available in RAID level 0 is lower than single disk.
4. It has high data transfer capacity.
5. It has high I/O request rate.

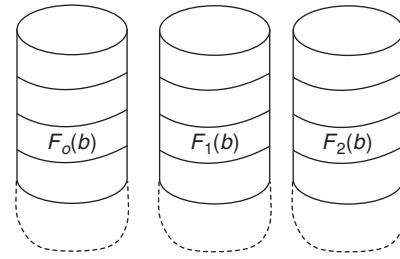
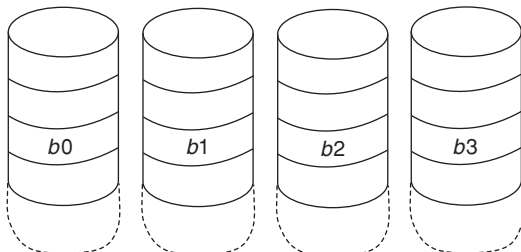
### RAID Level 1:



**Figure 19** RAID 1 (Mirrored)

1. Redundancy is achieved by the simple expedient of duplicating all the data.
2.  $2N$  disks required.
3. Data availability is higher than RAID 2, 3, 4, or 5. But lower than RAID 6.
4. Recovery from failure is simple.
5. RAID1 costs more.

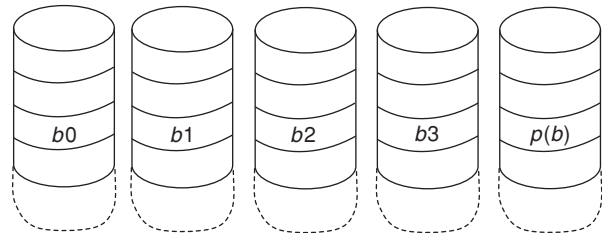
### RAID Level 2:



**Figure 20** RAID 2 (Redundancy through Hamming code).

6. Here redundancy is achieved through hamming code.
7.  $N + m$  disks required.

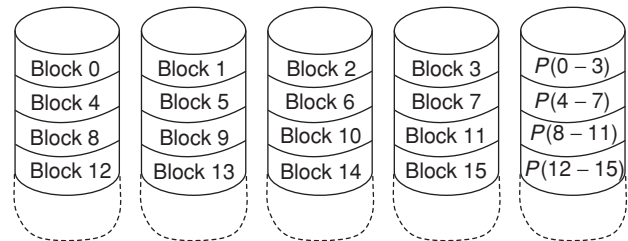
### RAID Level 3:



**Figure 21** RAID 3 (bit-interleaved parity)

1. It has bit interleaved parity.
2. Provides parallel access.
3.  $N + 1$  disks required.

### RAID Level 4:



**Figure 22** RAID 4 (Block-level parity)

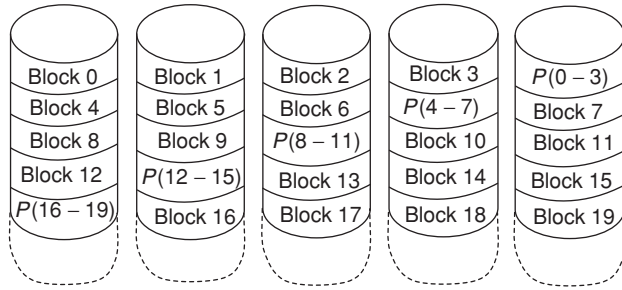
1.  $N + 1$  disks required.
2. Provides independent access.
3. Consider an array of five drives in which  $X_0$  through  $X_3$  contains data and  $X_4$  is the parity disk.

Suppose that a write is performed that only involves a strip on disk  $X_1$ . Initially for each bit  $i$ , we have the following relationship:

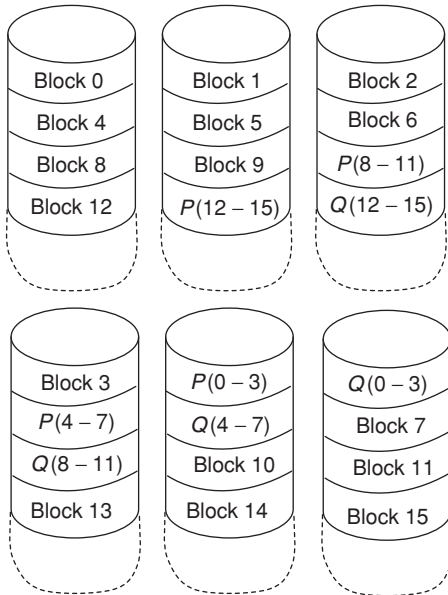
$$X_4(i) = X_3(i) \oplus X_2(i) \oplus X_1(i) \oplus X_0(i)$$

After the update, with potentially altered bits indicated by a prime symbol:

$$\begin{aligned} X_4'(i) &= X_3(i) \oplus X_2(i) \oplus X_1'(i) \oplus X_0(i) \\ &= X_3(i) \oplus X_2(i) \oplus X_1'(i) \oplus X_0(i) \oplus X_1(i) \oplus X_1(i) \\ &= X_4(i) \oplus X_1(i) \oplus X_1'(i) \end{aligned}$$

**RAID Level 5:**

1. It has block level distributed parity.
2. Provides independent access.
3. It has  $N + 1$  disks.

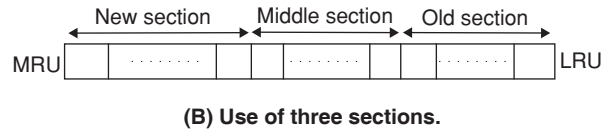
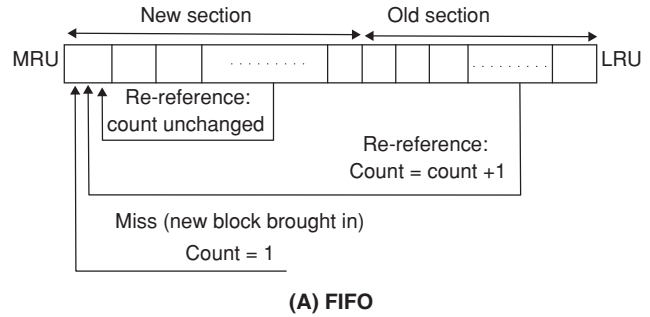
**RAID Level 6:**

1. It has dual redundancy.
2. It provides independent access.
3. It has  $N + 2$  disks.

**Disk Cache:**

1. It is a buffer in main memory for disk sectors.
2. The cache contains a copy of some of the sectors on the disk.
3. Replacement policy:
  - LRU
  - LFU

But these two replacement policies lead to poor performance. So a new technique frequency-based replacement is proposed. Two alternatives of frequency-based technique are shown below:



In FIFO, the blocks are logically organized in a stack; where the top part of the stack is the new section. When there is a cache hit, the referenced block is moved to top of the stack. If the block was already in the new section, its reference count is not incremented; otherwise it is incremented by 1.

In another technique, we divide the stack into three sections: New, middle and old. Here only blocks in the old section are eligible for replacement.

**PROTECTION AND SECURITY**

**System protection** Protection refers to a mechanism for controlling the access of programs, processes or users to the resources defined by a computer system.

**Principles of Protection**

Programs, users and even systems are given just enough privileges to perform their tasks. This is the principle of *least privilege*.

**Domain of protection** A computer system is a collection of processes and objects. The objects may be software or hardware objects. The operations that are possible may depend on the object.

A process should be allowed to access only those resources for which it has authorization. At any time, a process should be able to access only those resources that it currently requires to complete its task. This requirement is referred as *need to know principle* and is useful in limiting the amount of damage, which a faulty process can cause in the system.

**Domain structure** Each domain defines a set of objects and the types of operations that may be invoked on each object.

The ability to execute an operation on an object is an *access right*. A domain is a collection of access rights, each of which is an ordered pair <obj-name, rights-set>

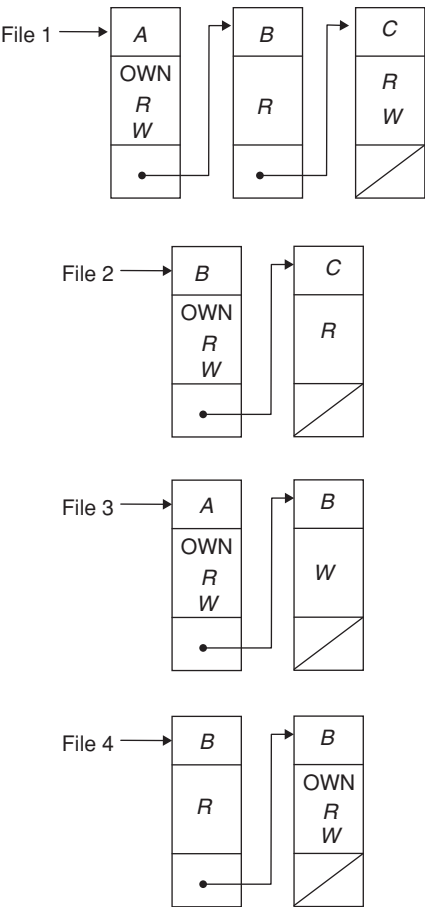
File System Security

- 1. A general model of access control for file management is an access matrix.
- 2. The basic elements of the model are as follows:
  - *Subject*: An entity capable of accessing objects.
  - *Object*: Anything to which access is controlled.
  - *Access right*: The way in which an object is accessed by a subject.

Access matrix format is shown below:

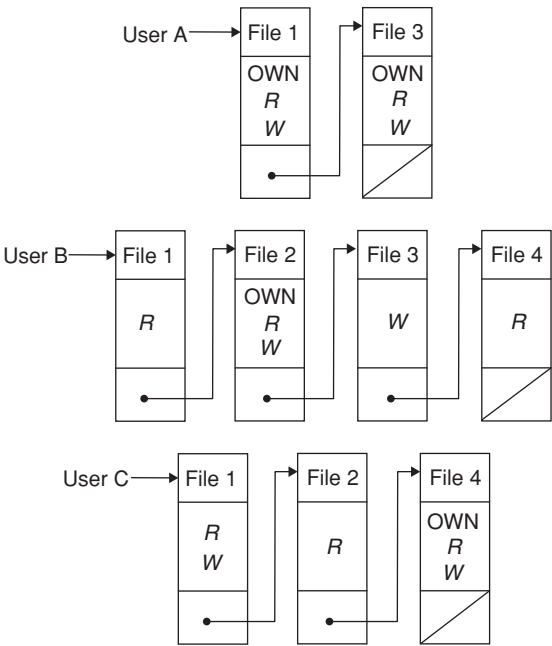
|        | File 1        | File 2        | File 3        | File 4        | Account 1         | Account 2         |
|--------|---------------|---------------|---------------|---------------|-------------------|-------------------|
| User A | OWN<br>R<br>W |               | OWN<br>R<br>W |               | Inquiry<br>credit |                   |
| User B | R             | OWN<br>R<br>W | W             | R             | Inquiry<br>debit  | Inquiry<br>credit |
| User C | R<br>W        | R             |               | OWN<br>R<br>W |                   | Inquiry<br>Debit  |

The access matrix may be decomposed by columns, yielding access control lists. The access control list for above matrix is



**Note:** This allows users that are not explicitly listed as having special rights to have a default set of rights.

Decomposition by row yields. Capability tickets.



**Note:** These tickets would have to be held in a region of memory inaccessible to users.

**System security** This is the protection afforded to an automated information system in order to attain objectives of preserving the integrity, availability and confidentiality of information system resources.

Objectives of Computer Security

**Confidentiality** Preserving authorized restrictions on information access and disclosure, including means for protecting personal privacy and proprietary information.

**Integrity** Guarding against improper information modification or destruction, including ensuring information non-repudiation and authenticity.

**Availability** Ensuring timely and reliable access to and use of information.

**Authenticity** The property of being genuine and being able to be verified and trusted.

**Accountability** The security goal that generates the requirement for actions of an entity to be traced uniquely to that entity.

Threats, Attacks and Assets

**Threats and attacks** Unauthorized disclosure is the threat to confidentiality. The attacks of following this threat are as follows:

- 1. *Exposure*: Sensitive data are directly released to an unauthorized entity.
- 2. *Interception*: An unauthorized entity directly accesses sensitive data traveling between authorized sources and destinations.

3. *Inference*: A threat action where an unauthorized entity indirectly accesses sensitive data by reasoning from characteristics or by-products of communications.
4. *Intrusion*: An unauthorized entity gains access to sensitive data by circumventing a system's security protections.

**Deception** Threat to either system integrity or data integrity. Types of attacks that can result are as follows:

1. *Masquerade*: An unauthorized entity gains access to a system or performs a malicious act by posing as an authorized entity.
2. *Falsification*: False data deceive an authorized entity.
3. *Repudiation*: An entity deceives another by falsely denying responsibility for an act.

**Disruption** A circumstance or event that interrupts or prevents the correct operation of system services and functions. Attacks for this threat are as follows:

1. *Incapacitation*: Prevents or interrupts system operation by disabling a system component.
2. *Corruption*: Undesirably alters system operation by adversely modifying system functions or data.
3. *Obstruction*: A threat action that interrupts delivery of system service by hindering system operation.

**Usurpation** A circumstance or event that results in control of system services or functions by an unauthorized entity. Attacks with this threat are as follows:

1. *Misappropriation*: An entity assumes unauthorized logical or physical control of a system resource.
2. *Misuse*: Causes a system component to perform a function or service that is detrimental to system security.

**Threats and assets** The assets of a computer are as follows:

1. Hardware
2. Software
3. Data
4. Communication lines

**Hardware** A major threat to computer system hardware is the threat to availability (e.g., theft of CD-ROMS).

#### Software

1. A key threat to software is an attack on availability (e.g., deletion of software).
2. A threat to integrity.
3. A threat to confidentiality.

**Data** Threats to data are an attack on

1. Availability
2. Confidentiality
3. Integrity

**Communication lines and networks** Two types of attacks:

1. Passive attacks
2. Active attacks

#### Passive attacks

1. These are in the nature of monitoring of the transmissions.
2. Attackers obtain information that is being transmitted.
3. Two types of passive attacks:
  - Release of message contents.
  - Traffic analysis
4. These are very difficult to detect because they do not involve any alteration of the data.

#### Active attacks

1. These attacks involves some modification of the data stream or the creation of a false stream and can be subdivided into four categories:
  - Replay
  - Masquerade
  - Modification of messages
  - Denial of service
2. It is difficult to prevent active attacks absolutely.

**Intruders** Three types of intruders:

**Masquerader** An individual who is not authorized to use the computer and who penetrates a systems access controls to exploit a legitimate user's account. He is likely to be an outsider.

**Misfeasor** A legitimate user who accesses data, programs or resources for which such access is not authorized or who is authorized for such access but misuses his or her privileges (generally an insider).

**Clandestine user** An individual who seizes supervisory control of the system and uses this control to evade auditing and access controls or to suppress audit collection (either outsider or insider).

**Hackers** Those who hack into computers do so for the thrill of it or for status. Attackers often took for targets of opportunity and then share the information with others.

**Criminals** Organized group of hackers have become a widespread and common threat to internet based systems.

**Malicious software overview** The most sophisticated types of threats to computer systems are presented by programs that exploit vulnerabilities in computing systems. These threats are referred as malicious software (or) malware.

1. It is designed to cause damage to or use up the resources of a target computer.
2. There are two types of malicious software:
  - Those that need a host program.  
**Example:** Viruses, logic bombs.
  - Those that are independent.  
**Example:** Worms, bot programs.
3. We can also differentiate between two types of software threats:
  - That do not replicate. These programs are activated by a trigger.  
**Example:** Logic bombs, backdoors.

- Those that replicate.

**Example:** Viruses, worms

**Backdoor (trapdoor)** It is a secret entry point into a program that allows someone who is aware of the backdoor to gain access without going through usual security access procedures.

**Logic bomb** It is a program inserted into software by an intruder. It lies dormant until a predefined condition is met; the program then triggers an unauthorized act.

**Trojan Horse** It is an apparently useful program, containing hidden code that, when invoked, performs some unwanted or harmful function.

**Mobile code** It is a software that can be shipped unchanged to a heterogeneous collection of platforms and execute with identical semantics.

**Viruses** A computer virus is a piece of software that can infect other programs by modifying them.

### Nature of Viruses

A virus can do anything that other programs do. It attaches itself to another program and executes secretly when the host program is run. Three parts of computer virus are as follows:

1. Infection mechanism
2. Trigger
3. Payload

Phases of computer virus are

1. Dormant phase
2. Propagation phase
3. Triggering phase
4. Execution phase

### Types of Virus

1. Encrypted virus
2. Stealth virus
3. Polymorphic virus
4. Metamorphic virus

**Worms** A worm is a program that can replicate itself and send copies from computer to computer across network connections.

**Bots** A bot is a program that secretly takes over another internet attached computer and then uses that computer to launch attacks that are difficult to trace to the bot's creator.

## EXERCISES

### Practice Problems I

**Directions for questions 1 to 15:** Select the correct alternative from the given choices.

1. Given a system using unspanned blocking and 100 byte blocks. A file contains records 30, 40, 55, 80, 30, 40. What percentage of space will be wasted in the blocks allocated for the file?  
(A) 31.25% (B) 41.25%  
(C) 51.25% (D) 62.15%
2. Disk requests come into the disk driver for cylinders 15, 25, 10, 2, 35, 9, 42 in that order. The disk head is currently positioned over cylinder 15. A seek takes 6 msec per cylinder moved. What is the total seek time using First Come First Served Algorithm?  
(A) 750 msec (B) 650 msec  
(C) 550 msec (D) 450 msec
3. A Java application needs to load 50 libraries. To load each library, one disk access is required. Seek time to access the location is 10 ms. Rotational speed is 6000 rpm. The total time needed to load all libraries is  
(A) 0.65 sec (B) 0.75 sec  
(C) 0.85 sec (D) 1 sec
4. A program has just read the 13th record in a sequential access file. If it wants to read the 10th record next, how many records must the program read to input the tenth record?

- (A) 5 (B) 0  
(C) 10 (D) 13

5. A disk is formatted into 40 sectors and 20 tracks. The disk rotates at 200 ms in one revolution. The time taken by the head to move from the centre to the rim is 10 ms. There are three different files stored on the disk:  
File *P* : Sector 2, track 4  
File *Q* : Sector 5, track 1  
File *R* : Sector 6, track 2  
Calculate the average latency time required for the three files.  
(A) 22.55 ms (B) 32.22 ms  
(C) 21.66 ms (D) 30.22 ms
6. Match the following  
(a) RAID0 (1) Parallel access  
(b) RAID1 (2) Striping  
(c) RAID2 (3) Use hamming code  
(d) RAID3 (4) Mirrored  
(A) a – 2, b – 4, c – 3, d – 1  
(B) a – 1, b – 2, c – 3, d – 4  
(C) a – 3, b – 2, c – 4, d – 1  
(D) a – 4, b – 1, c – 2, d – 3
7. The correct matching for the following pairs is  
(A) Disk scheduling (1) Round Robin  
(B) Batch processing (2) SCAN  
(C) Time sharing (3) LIFO  
(D) Interrupt processing (4) FIFO

- (A) A – 3, B – 4, C – 2, D – 1  
 (B) A – 4, B – 3, C – 2, D – 1  
 (C) A – 2, B – 4, C – 1, D – 3  
 (D) A – 2, B – 1, C – 4, D – 3
8. A program  $P$  reads and processes 2000 consecutive records from a sequential file stored on device  $R$  without using any file system facilities. Given the following:  
 Size of each record = 3500 bytes  
 Access time of  $R$  = 20 ms  
 Data transfer rate of  $R$  =  $500 \times 10^3$  bytes/sec  
 CPU time to process each record = 5 ms  
 What is the elapsed time of  $P$  if  $R$  contains unblocked records and  $P$  does not use buffering?  
 (A) 64 sec (B) 46 sec  
 (C) 34 sec (D) 17 sec
9. A disk has 19456 cylinders, 16 heads and 63 sectors per track. The disk spins at 5400 rpm. Seek time between adjacent tracks is 2 ms. Assuming the read/write head is already positioned at track 0, how long does it take to read the entire disk?  
 (A) 35 min (B) 68 min  
 (C) 58 min (D) 53 min
10. On a disk with 1000 cylinders, numbered 0 to 999, compute the number of tracks the disk arm must move to satisfy all the requests in the disk queue using SCAN algorithm. Assume the last request serviced was at track 345 and the head is moving towards 0. The queue in FIFO order contains requests for the following tracks:  
 123, 874, 692, 475, 105, 376  
 (A) 219 (B) 635  
 (C) 845 (D) 1219

**Common data for questions 11 to 14:** In the operation of a certain disk drive mechanism, a disk is formatted into 20 sectors and 10 tracks. The disk can be rotated either

clockwise or anti-clockwise. The times required to perform certain operations are as follows:

- I. Rotate the disk through one revolution = 200 ms  
 II. Move the disk head from the centre to the rim = 20 ms  
 III. Read and transmit one block of data = 0.3 ms  
 Three files are stored on the disk:  
 File  $A$ : 2 blocks at track 6  
 File  $B$ : 5 blocks at track 2  
 File  $C$ : 1 block at track 5
11. The disk head is initially at sector 0, track 0. If all three files  $A$ ,  $B$  and  $C$  are to be read in the minimum amount of time, they should be read in the following order:  
 (A)  $A, B, C$  (B)  $A, C, B$   
 (C)  $B, C, A$  (D)  $C, A, B$
12. The disk head is initially at sector 0, track 0. The files are read in the order  $C, B, A$ . The total time to read the files is  
 (A) 143.9 ms (B) 100.4 ms  
 (C) 114.0 ms (D) 102.6 ms
13. The most nearly average latency time for the sequence  $CBA$  is:  
 (A) 1 ms (B) 7 ms  
 (C) 27 ms (D) 50 ms
14. The most nearly average seek time for  $CBA$  is  
 (A) 1 ms (B) 8 ms  
 (C) 30 ms (D) 50 ms
15. A CD has 150 tracks rotating at 3500 rpm. Average seek time for consecutive tracks is 0.1 ms, the disk is subjected to read data from the track numbers 89, 75, 112, 5. What is the total seek time if the requests are served unidirectionally (C-Scan) and the first request determines initial direction? Assume that the current position of the head is at track 100.  
 (A) 20.2 ms (B) 16.9 ms  
 (C) 21.3 ms (D) 14 ms

## Practice Problems 2

**Directions for questions 1 to 15:** Select the correct alternative from the given choices.

1. The strategy that allocates the smallest possible chunk of disk space that is sufficient to the file is  
 (A) Nearest fit (B) Best fit  
 (C) Worst fit (D) First fit
2. If a process of 200 kB is transferred from backing store to memory and average disk latency is 10 ms, then what would be the total swap time, if transfer ratio is 2 Mbps?  
 (A) 10 ms  
 (B) 20 ms  
 (C) 30 ms  
 (D) 40 ms
3. Let us assume that the user process is 10 MB in size and backing store is a standard hard disk with a transfer rate of 40 MB per second. Let the average latency is 8 millisecond. Find actual time transfer of the 10 MB process to or from main memory?  
 (A) 8 ms (B) 250 ms  
 (C) 258 ms (D) 516 ms
4. Disk scheduling involves deciding  
 (A) which disk should be accessed next  
 (B) the order in which disk access requests must be serviced.  
 (C) the physical location when files should be accessed in the disk  
 (D) disk access time and an unused space.

5. The root directory of a disk should be placed
  - (A) at a fixed address in the memory
  - (B) anywhere on the disk
  - (C) at a fixed location on the system disk
  - (D) at a location on floppy.
6. Direct access methods are not effectively supported by
  - (A) Contiguous allocation
  - (B) Linked allocation
  - (C) Indexed allocation
  - (D) Sequential allocation
7. In which of the following directory systems, it is possible to have multiple paths for a file, starting from the root directory?
  - (A) Single-level directory
  - (B) Two-level directory
  - (C) Tree-structured directory
  - (D) A cyclic graph directory
8. The most common system's security method is:
  - (A) Passwords
  - (B) Key card systems
  - (C) Surveillance system
  - (D) Lock system
9. Trojan Horse programs
  - (A) are legitimate programs that allow unauthorized access.
  - (B) are hacker programs that do not show up on the system
  - (C) really do not work
  - (D) are immediately discovered
10. Which of the following is a program that spreads throughout the network?
  - (A) Trojan Horse
  - (B) Virus
  - (C) TSR
  - (D) Worm
11. A program has just read the 15th record in a sequential access file. If it wants to read the 10th record next, how many records must the program read to input the tenth record?
  - (A) 0
  - (B) 5
  - (C) 4
  - (D) 10
12. Formatting of a floppy disk refers to
  - (A) Arranging the data on the disk in contiguous fashion
  - (B) Writing the directory
  - (C) Erasing the system area
  - (D) Writing identification information on all tracks
13. Sector interleaving in disks is done by
  - (A) the disk manufacturer
  - (B) the disk controller card
  - (C) the operating system
  - (D) the user
14. Disk I/O is done in terms of
  - (A) Tracks
  - (B) Blocks
  - (C) Bits
  - (D) Bytes
15. How many six-letter passwords can be constructed using lowercase letters and digits?
  - (A)  $26^6$
  - (B)  $10^6$
  - (C)  $36^6$
  - (D)  $35^6$

### PREVIOUS YEARS' QUESTIONS

1. Consider a disk drive with the following specifications: **[2005]**  
 16 surfaces, 512 tracks/surfaces, 512 sectors/track, 1 KB/sector, rotation speed 3000 rpm. The disk is operated in cycle stealing mode whereby whenever one 4 byte word is ready it is sent to memory; similarly, for writing, the disk interface reads a 4 byte word from the memory in each DMA cycle. Memory cycle time is 40 nsec. The maximum percentage of time that the CPU gets blocked during DMA operation is:
  - (A) 10
  - (B) 25
  - (C) 40
  - (D) 50
2. Consider a disk pack with 16 surfaces, 128 tracks per surface and 256 sectors per track. 512 bytes of data are stored in a bit serial manner in a sector. The capacity of the disk pack and the number of bits required to specify a particular sector in the disk are, respectively: **[2007]**
  - (A) 256 Mbyte, 19 bits
  - (B) 256 Mbyte, 28 bits
  - (C) 512 Mbyte, 20 bits
  - (D) 64 Gbyte, 28 bits
3. For a magnetic disk with concentric circular tracks, the seek latency is not linearly proportional to the seek distance due to **[2008]**
  - (A) non-uniform distribution of requests
  - (B) arm starting and stopping inertia
  - (C) higher capacity of tracks on the periphery of the platter
  - (D) use of unfair arm scheduling policies
4. Consider a file of 16384 records. Each record is 32 bytes long and its key field is of size 6 bytes. The file is ordered on a non-key field, and the file organization is unspanned. The file is stored in a file system with block size 1024 bytes, and the size of a block pointer is 10 bytes. If the secondary index is built on the key field of the file, and a multi-level index scheme is used to store the secondary index, the number of first-level and second-level blocks in the multi-level index are, respectively **[2008]**

- (A) 8 and 0                      (B) 128 and 6  
(C) 256 and 4                  (D) 512 and 5

5. Consider a disk system with 100 cylinders. The requests to access the cylinders occur in following sequence: [2009]

4, 34, 10, 7, 19, 73, 2, 15, 6, 20

Assuming that the head is currently at cylinder 50, what is the time taken to satisfy all requests if it takes 1ms to move from one cylinder to adjacent one and shortest seek time first policy is used?

- (A) 95 ms                      (B) 119 ms  
(C) 233 ms                      (D) 276 ms

**Common data for questions 6 and 7:** A hard disk has 63 sectors per track, 10 platters each with 2 recording surfaces and 1000 cylinders. The address of a sector is given as a triple  $\langle c, h, s \rangle$ , where  $c$  is the cylinder number,  $h$  is the surface number and  $s$  is the sector number. Thus, the 0th sector is addressed as  $\langle 0, 0, 0 \rangle$ , the 1st sector as  $\langle 0, 0, 1 \rangle$ , and so on

6. The address  $\langle 400, 16, 29 \rangle$  corresponds to the sector number: [2009]

- (A) 505035                      (B) 505036  
(C) 505037                      (D) 505038

7. The address of the 1039th sector is [2009]

- (A)  $\langle 0, 15, 31 \rangle$                   (B)  $\langle 0, 16, 30 \rangle$   
(C)  $\langle 0, 16, 31 \rangle$                   (D)  $\langle 0, 17, 31 \rangle$

8. A file system with 300 GByte disk uses a file descriptor with 8 direct block addresses, 1 indirect block address and 1 doubly indirect block address. The size of each disk block is 128 bytes and the size of each disk block address is 8 bytes. The maximum possible file size in this file system is [2012]

- (A) 3 Kbytes  
(B) 35 Kbytes  
(C) 280 Kbytes  
(D) dependent on the size of the disk

9. Consider a hard disk with 16 recording surfaces (0–15) having 16384 cylinders (0–16383) and each cylinder contains 64 sectors (0–63). Data storage capacity in each sector is 512 bytes. Data are organized cylinder-wise and the addressing format is  $\langle \text{cylinder no.}, \text{surface no.}, \text{sector no.} \rangle$ . A file of size 42797 KB is stored in the disk and the starting disk location of the file is  $\langle 1200, 9, 40 \rangle$ . What is the cylinder number of the last sector of the file, if it is stored in a contiguous manner? [2013]

- (A) 1281                      (B) 1282  
(C) 1283                      (D) 1284

10. A FAT (File allocation table)-based file system is being used, and the total overhead of each entry in

the FAT is 4 bytes in size. Given a  $100 \times 10^6$  bytes disk on which the file system is stored and data block size is  $10^3$  bytes, the maximum size of a file that can be stored on this disk in units of  $10^6$  bytes is \_\_\_\_\_?

[2014]

11. Consider a disk pack with a seek time of 4 milliseconds and rotational speed of 10000 rotations per minute (RPM). It has 600 sectors per track and each sector can store 512 bytes of data. Consider a file stored in the disk. The file contains 2000 sectors. Assume that every sector access necessitates a seek, and the average rotational latency for accessing each sector is half of the time for one complete rotation. The total time (in milliseconds) needed to read the entire file is \_\_\_\_\_. [2015]

12. Suppose the following disk request sequence (track numbers) for a disk with 100 tracks is given: 45, 20, 90, 10, 50, 60, 80, 25, 70. Assume that the initial position of the R/W head is on track 50. The additional distance that will be traversed by the R/W head when the Shortest Seek Time First (SSTF) algorithm is used compared to the SCAN (Elevator) algorithm (assuming that SCAN algorithm moves towards 100 when it starts execution) is \_\_\_\_\_ tracks. [2015]

13. Consider a typical disk that rotates at 15000 rotations per minute (RPM) and has a transfer rate of  $50 \times 10^6$  bytes/sec. If the average seek time of the disk is twice the average rotational delay and the controller's transfer time is 10 times the disk transfer time, the average time (in milliseconds) to read or write a 512-byte sector of the disk is \_\_\_\_\_. [2015]

14. Consider a disk queue with requests for I/O to blocks on cylinders 47, 38, 121, 191, 87, 11, 92, 10. The C-LOOK scheduling algorithm is used. The head is initially at cylinder number 63, moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_. [2016]

15. In a file allocation system, which of the following allocation scheme(s) can be used if no external fragmentation is allowed? [2017]

- I. Contiguous  
II. Linked  
III. Indexed

- (A) I and III only                  (B) II only  
(C) III only                      (D) II and III only

16. Consider a storage disk with 4 platters (numbered as 0, 1, 2 and 3), 200 cylinders (numbered as 0, 1, ..., 199), and 256 sectors per track (numbered as 0, 1, ..., 255). The following 6 disk requests of the form [sector number, cylinder number, platter number] are

received by the disk controller at the same time:

[120, 72, 2], [180, 134, 1], [60, 20, 0], [212, 86, 3],  
[56, 116, 2], [118, 16, 1]

Currently the head is positioned at sector number 100 of cylinder 80, and is moving towards higher cylinder numbers. The average power dissipation in moving the head over 100 cylinders is 20 milliwatts and for

reversing the direction of the head movement once is 15 milliwatts. Power dissipation associated with rotational latency and switching of head between different platters is negligible.

The total power consumption in milliwatts to satisfy all of the above disk requests using the Shortest Seek Time First disk scheduling algorithm is \_\_\_\_\_.

[2018]

## ANSWER KEYS

### EXERCISES

#### Practice Problems 1

- |       |       |       |       |       |      |      |      |      |       |
|-------|-------|-------|-------|-------|------|------|------|------|-------|
| 1. A  | 2. A  | 3. B  | 4. C  | 5. C  | 6. A | 7. C | 8. A | 9. C | 10. D |
| 11. C | 12. B | 13. C | 14. B | 15. A |      |      |      |      |       |

#### Practice Problems 2

- |       |       |       |       |       |      |      |      |      |       |
|-------|-------|-------|-------|-------|------|------|------|------|-------|
| 1. B  | 2. C  | 3. D  | 4. B  | 5. C  | 6. D | 7. C | 8. A | 9. A | 10. D |
| 11. D | 12. D | 13. C | 14. B | 15. C |      |      |      |      |       |

#### Previous Years' Questions

- |           |      |        |                |      |         |       |        |      |          |
|-----------|------|--------|----------------|------|---------|-------|--------|------|----------|
| 1. B      | 2. A | 3. C   | 4. C           | 5. B | 6. C    | 7. C  | 8. B   | 9. D | 10. 99.6 |
| 11. 14020 |      | 12. 10 | 13. 6.1 to 6.2 |      | 14. 346 | 15. D | 16. 85 |      |          |

## TEST

## OPERATING SYSTEM

Time: 60 min.

**Directions for questions 1 to 30:** Select the correct alternative from the given choices.

- A program is \_\_\_\_\_ entity, while a process is \_\_\_\_\_ entity.  
(A) Active, passive  
(B) Active, sometimes active  
(C) Passive, active  
(D) Both (B) and (C)
- All the information associated with a specific process is contained in:  
(A) Process control block  
(B) Program control block  
(C) TLB  
(D) Heap
- Kernel-level threads and user-level threads are supported, respectively, by \_\_\_\_\_.  
(A) Operating system and operating system  
(B) Operating system and user  
(C) User and user  
(D) None of these
- Which of the following is false about user-level threads?  
(A) User-level threads are visible to the programmer and are unknown to the Kernel.  
(B) These are faster to create.  
(C) Kernel never interferes.  
(D) There is no effect of a system call () on process.
- Which of the following interprocess communication models are implemented using system calls?  
(A) Shared memory  
(B) Message passing  
(C) Both (A) and (B)  
(D) Neither (A) nor (B)
- Peterson's solution  
(i) is restricted to two processes  
(ii) share two data items turn and flag [i]  
(iii) mutual exclusion is achieved  
(iv) is a hardware solution  
Which of the above are true?  
(A) (i), (ii), (iii)  
(B) (ii), (iii), (iv)  
(C) (iv), (i), (ii)  
(D) (i), (ii), (iii), (iv)
- Which of the following requires a mode switch from one thread to another?  
(A) One process multiple thread  
(B) User-level thread  
(C) Kernel-level threads  
(D) Both (B) and (C)
- When a process is created, its state is  
(A) New  
(B) Ready  
(C) Block  
(D) Suspend
- The data section of a process in memory contains  
(A) Local variables, function parameters  
(B) Return addresses  
(C) Global variables  
(D) None of the above
- Which one of the following is true about process states?  
(i) A process which is running must have terminated as next state.  
(ii) From running state process can go to either waiting, ready or terminated state.  
(iii) Only one process can run at any instant.  
(iv) Ready process can go to waiting state.  
(A) (i), (ii), (iii)  
(B) (ii) and (iii) only  
(C) (i) and (iv) only  
(D) (ii), (iii), (iv)
- Message passing model of inter process communication can be  
(A) Blocking only  
(B) Blocking and non-blocking  
(C) Synchronous and asynchronous  
(D) Both (B) and (C)
- The definition of wait( ) is as follows:  

```
wait (S) {
while (S <=0);
S - - ;
}
```

  
The semicolon after while statement, signifies  
(A) Infinite looping  
(B) Blank statement  
(C) Depends on interpretation of compiler  
(D) No operation
- To avoid race condition, the number of processes using the critical sections is/are:  
(A) 1  
(B) 2  
(C) 3  
(D) More than 3
- The 'Critical Section' is the region in which  
(A) Any number of processes can enter without any permission  
(B) Only one process enters at a time and others wait for it.  
(C) Section is very critical  
(D) None of these

15. What does process control block contain?

- (A) Process Identification
- (B) Process state information
- (C) Process control information
- (D) All of the above

16. Match the following

|                                      |                                                                    |
|--------------------------------------|--------------------------------------------------------------------|
| (i) Multiprogramming                 | (x) Managing multiple processes executing on multiple computers    |
| (ii) Multiprocessing                 | (y) Management of multiple processes within a uniprocessor system. |
| (iii) Distributed process Management | (z) Management of multiple processes within a multiprocessor.      |

- (A) (i) –y (ii) –z (iii) –x
- (B) (i) –z (ii) –x (iii) –y
- (C) (i) –y (ii) –x (iii) –z
- (D) Ambiguous

17. For 'n' number of fork( ) system call, how many parent and child processes will be created?

- (A) 1,  $2^n - 1$ , respectively
- (B) 1,  $2^n$ , respectively
- (C)  $2^n - 1$ , 1, respectively
- (D) n, 2n, respectively

18. If the value of binary semaphore is initialized with 1 and three wait( ) operations are performed, how many processes are there in the block list?

- (A) 1
- (B) 0
- (C) 3
- (D) 2

19. A counting semaphore is initialized with the value 3. A list of 'P' and 'V' operations are performed on the semaphore as: 1P, 2V, 2P, 3V, 5P, 7V, 2P, 3V. The final value of semaphore is?

- (A) 5
- (B) 8
- (C) 7
- (D) 6

20. The final value of semaphore after 10 'P' operations and 23 'V' operations is 1. What will be the initial value of this counting semaphore?

- (A) –14
- (B) –13
- (C) –12
- (D) –11

21. For a machine-instruction approach to enforce mutual exclusion, following are its properties:

- (i) starvation and deadlock free
- (ii) it is applicable to any number of processes.
- (iii) it can be used to support multiple critical sections, each defined by its own variable.
- (iv) it is simple, easy to verify and employed with busy waiting

Which of the above is false?

- (A) (iv) only
- (B) (ii), (iii) only
- (C) (i), (ii) only
- (D) (i) only

22. Consider the following code:

```
if (fork() == 0)
{
 a = a + 5;
 printf("%d, %d\n", a, &a);
}
else
{
 a = a - 5;
 printf("%d, %d\n", a, &a);
}
```

Let  $p, q$  be the values printed by the parent process, and  $s, t$  be the values printed by the child process. Which one of the following is true?

- (A)  $p = s + 10$  and  $q = t$
- (B)  $p = s + 10$  and  $q \uparrow t$
- (C)  $p + 10 = s$  and  $q = t$
- (D)  $p + 10 = s$  and  $q \uparrow t$

23. Consider the following statements with respect to user-level threads:

- (i) Context switch is faster with kernel-supported threads.
- (ii) For user-level threads, a system call can block the entire process.
- (iii) Kernel-supported threads can be scheduled independently.
- (iv) User-level threads are transparent to the Kernel.

Which of the above statements are true?

- (A) (i), (iii) and (iv) only
- (B) (ii) and (iii) only
- (C) (i) and (iii) only
- (D) (i) and (ii) only

24. Suppose there are 'n' CPUs and 'm' processes such that  $m > n$ . What will be the minimum and maximum number of ready, running and blocked process, respectively?

- (A) 0, 0, 0 and  $m, n, m$
- (B) 1, m, 1 and  $n, n, n$
- (C) m, 1, 0 and  $m, m, n$
- (D) 0, 0, 0 and  $n, m, m$

25. Consider the following signal semaphore code signal (semaphore \*s)

```
{
 s.value++;
 if (_____ (I) _____)
 {
 remove a process P from S.list;
 _____ (II) _____
 }
}
```

Choose the suitable options for (I) and (II), respectively

- (A) S.value = 0 and wakeup(P);
- (B) S.value <= 0 and wakeup(P);
- (C) S.value < 0 and block( );
- (D) S.value <= 0 and block( );

26. Consider the methods used by processes  $P_1$  and  $P_2$  for accessing their critical sections whenever needed. The initial values of shared Boolean variables  $S_1$  and  $S_2$  are randomly assigned.

Method used by  $P_1$

While ( $S_1 == S_2$ );

Critical section

$S_1 = S_2$ ;

Method used by  $P_2$

While ( $S_1 != S_2$ );

Critical section

$S_2 = !S_1$ ;

Which of the following statements describes the properties achieved?

- (A) Mutual exclusion but not progress
  - (B) Progress only
  - (C) Bounded waiting, progress
  - (D) Mutual exclusion, progress, bounded waiting
27. Consider the following statements regarding spin locks:
- (i) No context switch is required when a process wait on a lock
  - (ii) Spin locks are useful when locks are expected to be held for short times.
  - (iii) They are often employed on multiprocessor systems
  - (iv) Process 'spins' while waiting for a lock
- Choose the correct option:
- (A) (i), (ii), (iii), (iv) are true
  - (B) Only (i) and (ii) are true
  - (C) (iii) is false
  - (D) (ii) is true and (iv) is false

**Common data for questions 28, 29 and 30:** From the Readers-Writers problem, the data structure for reader process is:

```
semaphore mutex, wrt;
int readcount;
```

```
while(1)
{
 wait(mutex);
 readcount++;
 if(readcount == 1)
 wait(wrt);
 signal(mutex);

 - - - - -
 - - - - -
 wait(mutex);
 readcount - -;
 if(readcount == 0)
 signal(wrt); signal(mutex);
}
```

mutex and wrt are initialized to 1 and readcount is initialized to 0.

28. Mutual exclusion for readers is attained by
- (A) Wrt
  - (B) Mutex
  - (C) Readcount
  - (D) Both (A) and (B)
29. Which of the following semaphore or semaphores is used by the first or last reader that enters or exits the critical section?
- (A) Wrt
  - (B) Mutex
  - (C) Readcount
  - (D) Both (A) and (B)
30. The readcount variable keeps track of how many processes are \_\_\_\_.
- (A) Currently reading the object
  - (B) Currently writing the object
  - (C) Waiting in the queue
  - (D) Reading the shared data

## ANSWER KEYS

- |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 1. C  | 2. A  | 3. B  | 4. D  | 5. B  | 6. A  | 7. C  | 8. A  | 9. C  | 10. B |
| 11. D | 12. D | 13. A | 14. B | 15. D | 16. A | 17. A | 18. D | 19. B | 20. C |
| 21. D | 22. D | 23. B | 24. A | 25. B | 26. A | 27. A | 28. B | 29. A | 30. A |